Описание программного обеспечения MotorControlDemo

Руководство программиста/пользователя

Москва 15.04.2019

Оглавление

1. Введение 4
2. Обзор файловой структуры 5
   2.1. Организация каталогов 5
   2.2. Список файлов проекта 5
3. Архитектура программного обеспечения 9
   3.1. Наименование переменных, констант и типов данных 9
   3.2. Модульность программы 9
   3.3. Комментарии к коду программы 10
   3.4. Дискретные автоматы 11
   3.5. Прерывания и их обслуживание 12
   3.6. Управление распределением памяти 13
   3.7. Организация объявления глобальных переменных 14
   3.8. Ход выполнения программы 14
   3.9. Введение в CANopen 15
   3.10. Форматы вычислений и единицы измерения 18
4. Быстрый старт 20
   4.1. Установка среды разработки 20
   4.2. Выбор аппаратной конфигурации 20
      4.2.1. Адаптация ПО для работы с математическими моделями 21
      4.2.2. Адаптация ПО для работы с реальным инвертором и электродвигателем 21
   4.3. Программирование и запуск ПО 22
   4.4. Подключение к ПК через UniCON 26
   4.5. Настройка модели двигателя 29
   4.6. Настройка базовых значений 29
   4.7. Настройка модуля защит 30
   4.8. Настройка модуля АЦП 30
   4.9. Запуск привода 32
5. Описание модулей ПО 35
   5.1. SM_Ctrl: структуры системы управления 35
      5.1.1. Структура скалярного управления (режим работы = 3) 36
      5.1.2. Структура частотно-токового управления (режим работы = 4) 37
      5.1.3. Режим удержания (режим работы = 2) 38
      5.1.4. Структура векторного датчикового управления синхронным двигателем (режим работы = 5) 38
      5.1.5. Структура векторного датчикового управления асинхронным двигателем (режим работы = 55) 41
      5.1.6. Структура векторного датчикового управления для синхронного двигателя с дополнительным внешним контуром положения (режим работы = 10) 44
      5.1.7. Режим автонастройки смещений ДПР (режим работы = 29) 45
      5.1.8. Структура управления двигателем постоянного тока заданием напряжения на обмотках (режим работы = 35) 46
      5.1.9. Структура подчиненного управления двигателем постоянного тока с контуром тока и скорости (режим работы = 36) 46
      5.1.10. Структура управления вентильно-индукторным двигателем с самовозбуждением в режиме автокоммутации с токоограничением и контуром скорости. (режим работы = 101) 48
   5.2. ПИД регулятор 48
   5.3. Кривая U/f 49
   5.4. Задатчик интенсивности (ЗИ) 50
   5.5. Модуль фазных преобразований 50

5.6. Модуль координатных преобразований 50
5.7. Модуль ШИМ 51
5.8. Модуль АЦП 56
5.9. Модуль ДПР на дискретных элементах Холла 56
5.10. Модуль инкрементального энкодера 57
5.11. Модуль цифрового осциллографа (даталоггер) 59
5.12. Модуль защит 60
5.13. Модуль обработки команд управления SM_CmdLogic 62
5.14. Библиотека функций целочисленной математики (IQmath) 62
5.15. Модуль математических моделей электродвигателей (MotorModel) 63
5.16. Модуль работы с пользовательской флеш памятью (UserMemory) 63
5.17. Модуль часов реального времени 64
5.18. Модуль работы с драйвером ключей DRV8301 64
5.19. Модуль драйвера CANopen 64
5.20. Модуль работы с драйвером CANopen через интерфейсы UART, USB (CANtoRS) 64
5.21. Модуль драйвера MODBUS 65
5.22. Отображение текущих параметров привода (модуль CurPar) 65
5.23. Различные важные параметры, не относящиеся к конкретному модулю 65
5.23.1. Настройки системы 65
5.23.2. Переменные для отладки 65
5.23.3. Счетчики 66
6. Лабораторные работы 67
6.1. Синхронный двигатель 67
6.1.1. Скалярное управление 67
6.1.2. Настройка регуляторов тока 67
6.1.3. Частотно-токовое управление 70
6.1.4. Проверка и настройка ДПР 72
6.1.5. Векторное двухконтурное управление 74
6.1.6. Позиционное управление 75
6.2. Асинхронный двигатель 76
6.2.1. Скалярное управление 76
6.2.2. Настройка регуляторов тока 76
6.2.3. Частотно-токовое управление 76
6.2.1. Проверка и настройка ДПР 76
6.2.2. Векторное двухконтурное управление 77
6.2.1. Позиционное управление 80
6.3. Двигатель постоянного тока 80
6.3.1. Разомкнутая система управления 81
6.3.2. Замкнутая система управления 82
6.4. Вентильно-индукторный двигатель 83
6.4.1. Режим автокоммутации 84
6.5. Добавление параметра CANopen в программе COODEdit 85
7. Заключение 91

1. Введение

Данное руководство предназначено для программистов, которые начинают знакомство с программным обеспечением MotorControlDemo для микроконтроллеров серии 1921BK01 фирмы АО «НИИЭТ». Документ описывает основные функции и организацию демонстрационного программного обеспечения (далее ПО), а также содержит руководство по пусконаладке электропривода с применением данного ПО.

ПО реализует различные структуры управления электродвигателями различных типов, а также содержит математические модели электродвигателей, которые могут быть опционально подключены вместо реального объекта управления на время отладки.

ПО может быть сконфигурировано перед компиляцией для различных аппаратных конфигураций (различных контроллеров и отладочных плат на базе 1921BK01). Для некоторых аппаратных конфигураций ПО может работать только с математическими моделями двигателей, а для конфигураций, поддерживающих управление реальным электродвигателем может по выбору пользователя управлять и реальным объектом (например, конфигурация для отладочного комплекта VectorCARD с инвертором).

ПО микроконтроллера предназначено для двух сред разработки:

- VectorIDE (Eclipse+GCC+OpenOCD, загрузить можно по адресу http://motorcontrol.ru/production/soft/vector-ide/).
- CodeMaster++ [ARM] (официальная среда разработки НИИЭТ).

Для настройки системы управления, задания уставок, переключения режимов и наблюдения осциллограмм переходных процессов ПО микроконтроллера использует драйвер CANopen (в виде библиотеки), который обеспечивает связь по интерфейсу CAN с персональным компьютером. Со стороны персонального компьютера используется ПО UniCON для Windows, которое при помощи переходника CAN-USB или USB-UART конвертора взаимодействует через CANopen с ПО микроконтроллера и обеспечивает интерфейс пользователя. Для работы со словарем объектов CANopen используется прогармма COODEdit. Документация на драйвер CANopen и программы персонального компьютера (UniCON, COODEdit) представлена отдельно в виде руководств пользователя.

2. Обзор файловой структуры

2.1. Организация каталогов

Большая часть программного обеспечения (ПО) написана в виде модулей. Модуль обычно состоит из двух файлов – заголовочного файла с расширением *.h и одноименного файла с расширением *.c.

Большинство новых модулей имеют префикс “V_” в именах файлов. Ряд модулей имеет префикс “SM_”, что расшифровывается как State Machine (машина состояний или дискретный автомат): эти модули в своей основе не являются законченными блоками для реализации какой-либо одной функции, а представляют собой логическую функцию, построенную на принципах дискретных автоматов, и служат для отслеживания состояний работы, смены режимов и управляют остальными модулями. Также префикс SM_ имеют некоторые модули-обёртки, которые не содержат своего дискретного автомата, но группируют вызовы других модулей ПО (SM_Net, SM_Sys).

В проект также добавлены стандартные библиотечные файлы для работы с микроконтроллером и его периферией.

Все «стандартные» поставляемые производителем микроконтроллера *.h и *.c файлы находятся в директориях include и src соответственно. Все самостоятельно написанные пользовательские файлы аналогично размещены в Vinclude и Vsrc.

Файлы, относящиеся к определенной среде разработки вынесены в отдельный каталог по имени среды разработки (EclipseIDE и CodeMaster), туда помещены только файлы запуска, распределения памяти и файлы организации «проекта» данной среды разработки, а также библиотека драйвера CANopen (имеет разный формат для разных компиляторов). Все остальные исходные тексты программы делятся между обоими средами разработки.

2.2. Список файлов проекта

Каталог ***EclipseIDE***	
Содержит файлы организации проекта среды разработки EclipseIDE (.cproject, .project, .settings)	
Каталог ***EclipseIDE/asm***	
startup_MCP_gcc.S	стартовый файл, с которого начинается выполнение программы. После небольшой инициализации передает управление функции main.
Каталог ***EclipseIDE/cmd***	
build.ld	файл компоновки памяти проекта.
Каталог ***EclipseIDE/lib***	
libc.a **libgcc.a** **libm.a**	стандартные библиотеки GCC для работы с микроконтроллером (версия float point).
libCANopen_drv.a	библиотека драйвера CANopen. Реализует соответствующий протокол связи при помощи интерфейса CAN.
Каталог ***CodeMaster***	
libCANopen_drv.mcl	библиотека драйвера CANopen. Реализует соответствующий протокол связи при помощи интерфейса CAN.
MotorControlDemo.bat, ***MotorControlDemo.ipp***	Служебные файлы организации проекта среды ***CodeMaster***
startup_K1921BK01T.mca	стартовый файл, с которого начинается выполнение программы. После небольшой инициализации передает управление функции main.
Каталог ***include***	
core_cm4_simd.h **core_cm4.h** **core_cmFunc.h** **core_cmInstr.h**	заголовочные файлы для работы с ядром микроконтроллера.

K1921VK01T.h	заголовочный файл для работы с периферией микроконтроллера. Содержит объявления регистров всех периферийных устройств.
Каталог ***src***	
K1921VK01T_init.c	файл предварительной инициализации периферии микроконтроллера. Содержит функции включения и подготовки микроконтроллера к работе.
Каталог ***Vinclude***	
build.h	конфигурационный заголовочный файл. Выбирает тип аппаратной части.
CANBlockTransferInterface.h	заголовочный файл пользовательских функций интерфейса блочной передачи данных драйвера CANopen.
CANopen_drv.h	главный заголовочный файл драйвера CANopen.
clarke.h	заголовочный файл модуля фазных преобразований.
co_ODvars.h	объявление служебных структур драйвера CANopen.
cood1.h	объявление служебных переменных для словаря объектов 1 драйвера CANopen.
cood2.h	объявление служебных переменных для словаря объектов 2 драйвера CANopen.
DRV8301_SPI.h	заголовочный файл для работы с микросхемой драйвера ключей DRV8301.
DSP.h	файл подключения заголовочных файлов периферии и объявления системных констант.
EPwm_defines.h	макросы для удобства настройки и работы с модулем ШИМ.
filter.h	заголовочный файл инерционного звена первого порядка (фильтр).
ipark.h	заголовочный файл модуля обратных координатных преобразований.
main.h	главный заголовочный файл проекта.
mbod.h	заголовочный файл словаря драйвера MODBUS.
park.h	заголовочный файл модуля координатных преобразований.
SM_CmdLogic.h	заголовочный файл обработки команд и заданий.
SM_Ctrl.h	заголовочный файл основного ДА переключения систем управления.
SM_Net.h	заголовочный файл модуля общего назначения для работы с сетью.
SM_Protect.h	заголовочный файл модуля защит.
SM_Sys.h	заголовочный файл модуля-обертки для расчета остальных модулей.
V_adc.h	заголовочный файл модуля обработки АЦП.
V_AutoOffset.h	заголовочный файл модуля автоподстройки смещения датчиков тока модуля АЦП.
V_bits_to_enum_numbers.h	заголовочный файл модуля "листания" кодов аварий при наличии нескольких аварий одновременно.
V_CANtoRS.h	драйвер для работы CANopen драйвера через RS (UART) и USB.

v_common.h	заголовочный файл с объявлением структур данных различного назначения, которые еще слишком малы для того, чтобы организовывать их в отдельный модуль.
V_CurPar.h	заголовочный файл расчета наблюдаемых текущих параметров.
v_data_log.h	заголовочный файл 4-х канального логгера для осциллографирования в реальном времени.
V_DPR_eCAP.h	заголовочный файл модуля расчёта скорости и положения по трём дискретным датчикам (например, на элементах Холла).
V_Excitation.h	заголовочный файл модуля обмотки возбуждения.
V_hzprof.h	заголовочный файл кривой U(f).
V_IQmath.h	заголовочный файл библиотеки функций целочисленной математики.
V_led.h	заголовочный файл управления светодиодной индикацией.
V_MBVarsConv.h	заголовочный файл вспомогательного модуля драйвера MODBUS для конвертации данных.
V_ModBus.h	заголовочный файл драйвера MODBUS.
V_MotorModel.h	заголовочный файл модуля математических моделей электродвигателей.
V_MotorParams_DCM.h	заголовочный файл с параметрами двигателей постоянного тока для модуля математических моделей электродвигателей.
V_MotorParams_IM.h	заголовочный файл с параметрами асинхронных двигателей для модуля математических моделей электродвигателей.
V_MotorParams_SM.h	заголовочный файл с параметрами синхронных двигателей для модуля математических моделей электродвигателей.
V_MotorParams_SRM.h	заголовочный файл с параметрами вентильно-индукторных двигателей с самовозбуждением для модуля математических моделей электродвигателей.
V_MotorParams.h	заголовочный файл с подключением параметров разных двигателей модуля математических моделей.
V_pid_reg3.h	заголовочный файл ПИД-регулятора.
V_PWM_Module.h	заголовочный файл модуля реализации ШИМ.
V_QEP.h	заголовочный файл модуля оценки скорости и положения при помощи eQEP.
V_relay_reg.h	заголовочный файл релейного регулятора.
V_rmp_ctrl.h	заголовочный файл задатчика интенсивности.
V_rotor_observer.h	заголовочный файл модуля наблюдателя потокосцепления ротора для асинхронного двигателя.
V_RTC_Clock.h	заголовочный файл модуля работы с часами реального времени.
V_SSI_Encoder.h	заголовочный файл модуля работы с датчиком положения ротора с SSI интерфейсом.
V_UserMemory.h	заголовочный файл работы с энергонезависимой пользовательской памятью.
Каталог ***Vsrc***	
CANBlockTransferInterface.c	файл с пользовательскими функциями интерфейса блочной передачи данных драйвера CANopen.

CANopenUDfuncs.c	пользовательские функции драйвера CANopen (обращение к памяти, callback и т.п.).
clarke.c	модуль фазных преобразований.
cood2.c	файл словаря объектов драйвера CANopen второго модуля CAN.
DRV8301_SPI.c	файл работы с микросхемой драйвера ключей DRV8301.
filter.c	модуль инерционного звена первого порядка (фильтр).
ipark.c	модуль инверсных координатных преобразований.
main.c	главный файл проекта. Содержит main(), а также обработчики прерываний.
mbod.c	файл словаря объектов драйвера MODBUS.
park.c	модуль координатных преобразований.
SM_CmdLogic.c	обработка команд и заданий.
SM_Ctrl.c	основной дискретный автомат переключения систем управления.
SM_Net.c	модуль общего назначения для работы с сетью.
SM_Protect.c	модуль защит.
SM_Sys.c	модуль-обертка для расчета остальных модулей.
V_adc.c	модуль обработки АЦП.
V_AutoOffset.c	модуль автоподстройки смещения датчиков тока модуля АЦП.
V_bits_to_enum_numbers.c	модуль "листания" кодов аварий при наличии нескольких аварий одновременно.
V_CANtoRS.c	драйвер для работы CANopen драйвера через RS (UART) и USB
V_CurPar.c	расчет наблюдаемых текущих параметров.
v_data_log.c	4-х канальный логгер для осциллографирования в реальном времени.
V_DPR_eCAP.c	драйвер датчика положения ротора на трех дискретных элементах (например, Холла).
V_excitation.c	модуль управления обмоткой возбуждения.
V_hzprof.c	кривая U(f).
V_IQmath.c	библиотека функций целочисленной математики.
V_led.c	управление светодиодной индикацией.
V_MBVarsConvert.c	файл вспомогательного модуля драйвера MODBUS для конвертации данных.
V_ModBus.c	драйвер MODBUS.
V_MotorModel.c	модуль математических моделей электродвигателей.
V_pid_reg3_pos.c	ПИД-регулятор для контура положения.
V_pid_reg3.c	ПИД-регулятор общего назначения.
V_PWM_Module.c	модуль реализации ШИМ.
V_QEP.c	модуль оценки скорости и положения при помощи eQEP.
V_relay_reg.c	модуль релейного регулятора.
v_rmp_ctrl.c	задатчик интенсивности.
V_rotor_observer.c	модуль наблюдателя потокосцепления ротора для асинхронного двигателя.
V_RTC_Clock.c	модуль работы с часами реального времени.
V_SSI_Encoder.c	модуль работы с датчиком положения ротора с SSI интерфейсом.
V_UserMemory.c	модуль работы с энергонезависимой пользовательской памятью.

3. Архитектура программного обеспечения

3.1. Наименование переменных, констант и типов данных

Имена переменных и функций, как правило, пишутся строчными буквами, каждое новое слово с заглавной буквы (за исключением, возможно, первой). Другой вариант – с использованием подчеркивания для разделения слов. Например:

```
long DebugW1;
TSM_Ctrl sm_ctrl;
void AdcDrv_init(TAdcDrv *p)
```

Константы всегда объявляются заглавными буквами с разделением слов подчеркиванием:

```
#define CTRL_RUN_VECTOR_HALL                    5
```

Для установки принадлежности функции или константы к какому-либо модулю используются префиксы:

```
void DPReCAP_Init(TDPReCAP*);     // Функция относится к модулю DPReCAP
void AdcDrv_slow_calc(TAdcDrv*); // Функция относится к модулю AdcDrv
```

Типы данных, таких как типы структур, именуются аналогично переменным и функциям, но с префиксом "T". Например:

```
typedef struct
{
   long speed_ref;//!< Задание скорости
   long Iq_ref;//!< Задание токоограничения для векторной системы управления
} TRefs;
```

3.2. Модульность программы

В проекте не используются классы Си++ из-за различных причин, таких как:

1) Меньшая прозрачность генерируемого компилятором ассемблерного кода.
2) Проблемы переносимости ПО между разными компиляторами, проблемы с операцией взятия адреса метода класса (реализуется в разных компиляторах по-разному, с разным синтаксисом, где-то не поддерживается вообще).
3) Нежелательность использования операций динамического выделения памяти во встраиваемых системах управления: одна из сильных сторон Си++ оказывается не задействована.

Вследствие этого модульность программы достигается средствами языка Си. "Модуль" в терминах языка Си представляет собой структуру, внутри которой размещаются необходимые переменные модуля, а также указатели на функции модуля, предназначенные для "внешнего" использования – имитация класса Си++ средствами Си. Сама реализация функций находится в отдельном файле, а имена функций имеют префиксы, содержащими имя модуля, чтобы гарантированно не пересекаться в названии с другими функциями проекта. В саму же структуру модуля указатели на эти функции добавляются с короткими и простыми именами.

Как уже было сказано, обычно модуль состоит из двух файлов – заголовочного *.h файла, в котором определяется структура, её инициализатор, а также *.c файла, в котором реализуются «методы» модуля, т.е. содержатся функции работы с этим модулем. В каждую такую функцию помимо остальных аргументов, обязательно передается указатель на экземпляр структуры модуля, чтобы обращаться к членам конкретного экземпляра такого «самодельного» класса. Пример:

Заголовочный файл модуля цифрового фильтра:

```
struct SFilter
{
```

```
    long input;//! Вход
    long output;//! Выход
    long T; //! T=Ts/Tfiltra где - Tfiltra постояннаЯ времени фильтра
    void (*calc)(volatile struct SFilter*); //указатель на функцию расчета
  };

  typedef volatile struct SFilter TFilter;

//Инициализатор структуры
#define FILTER_DEFAULTS {0,0,_IQ(0.01),TFilter_Calc} //Функция расчета
void TFilter_Calc(TFilter*);
```

Файл с реализацией функций модуля:

```
#include "V_filter.h"

//! Инерционное звено 1/(Tp+1)
//! Используется как фильтр первого порядка
void TFilter_Calc(TFilter *p) {
      p->output = _IQmpy(p->T,(p->input-p->output)) + p->output;
}
```

Для использования модуля фильтра пользователь должен объявить в основной программе переменную (или несколько, если необходимо) с типом `TFilter` и присвоить ей инициализатор `FILTER_DEFAULTS`:

```
      TFilter myFilter1 = FILTER_DEFAULTS;
      TFilter myFilter2 = FILTER_DEFAULTS;
```

Затем для работы фильтра нужно с определенной частотой вызова фильтра (например, 10кГц) передавать фильтруемые данные в переменную `input`, вызывать функцию расчета через указатель `calc`, а затем забирать отфильтрованный сигнал с `output`:

```
      myFilter1.input=ADCInput1;
      myFilter1.calc(&myFilter1);
      filteredADCInput1=myFilter1.output;
```

Каждый модуль может содержать различные функции, но имеется следующее соглашение на именование: указатель на функцию (далее – «метод» в терминологии объектно-ориентированного программирования) инициализации должен называться `init`, а методы, выполняющие однотипные действия и вызывающиеся с заданной дискретностью по времени должны иметь название `calc`. Например, так может называться метод обработки датчика положения ротора, обработки модуля защит, обработки АЦП и так далее. Данные методы ничего не принимают (кроме указателя на экземпляр своего модуля) и не возвращают в явном виде, но результаты их работы как-то отражаются в переменных структуры их модуля или в работе периферии. Если в модуле имеется несколько методов, которые надо вызывать с разной дискретизацией по времени, то имя метода дополняется пояснением, с какой частотой его необходимо вызывать. Например, метод `calc_10k`, `calk_1k` и т.п. Если разработчик модуля не имеет строгих требований к частоте вызова, но хочет подчеркнуть, что одна функция расчета выполняется значительно дольше или реже другой, то допустимы наименования `fast_calc` и `slow_calc`. Если метод модуля выполняет одну конкретную операцию, например, отправку данных или расчет частоты вращения, то допускается более конкретное наименование метода, например, `send` или `rotorSpeedCalc`.

3.3. Комментарии к коду программы

Большая часть ПО документирована в системе doxygen. Придерживаясь синтаксиса комментариев doxygen при помощи соответствующей утилиты

(https://ru.wikipedia.org/wiki/Doxygen) возможна автоматическая генерация документации на основе комментариев к программе.

В терминах doxygen, чтобы обозначить принадлежность Си-функций к определенной структуре, используется следующая схема. Структура, содержащая в себе переменные и ссылки на функции модуля, названа классом. А функции, реализация которых находится в *.c файле, названы методами с принадлежностью к соответствующему классу. Это сделано специальными директивами doxygen, такими как «\class», «\addtogroup» «\memberof».

3.4. Дискретные автоматы

Дискретные автоматы (ДА) – или, как их чаще называют, машины состояния, конечные автоматы, state machines – широко применяются в данном проекте. Простыми словами, дискретный автомат – это такой подход к программированию, где программу разбивают на фиксированное число состояний, в каждом из которых выполняются одни и те же однотипные действия, а переход между состояниями выполняется по каким-то событиям. Программа дискретного автомата ни в коем случае не зацикливается внутри каждого состояния, а, выполнив нужные действия, возвращает управление, чтобы через некоторое время быть вызванной снова. В зависимости от сложности ПО в системе управления может работать сразу несколько ДА.

Легко представить дискретный автомат защит, который в состоянии «норма» каждый раз проверяет наступление аварийных событий, а при их наступлении переводит автомат в состояние «авария». В состоянии «авария» дискретный автомат выключает и обесточивает электрооборудование и ожидает команду сброса аварии от пользователя. Получив такую команду ДА переходит в состояние «сброс аварии», где производит действия по обнулению флагов аварии, после чего автоматически переходит в состояние «норма». В зависимости от требуемого быстродействия реакции на аварию подобный дискретный автомат должен вызываться с определенной частотой, скажем 10кГц, выполнен в виде отдельного программного модуля и выполнятся постоянно независимо от работы других модулей программы.

Помимо действий, которые выполняются в стационарном состоянии дискретного автомата, часто требуется выполнить какие-то действия однократно при смене состояния дискретного автомата. Например, при попадании в состояние «норма» дискретного автомата защит можно произвести сброс команд на запуск привода, если они были поданы за время нахождения привода в состоянии аварии, чтобы предотвратить неожиданный пуск. Для этой цели используется переменная entry (для краткости E), которая служит флагом первого вхождения в состояние. Типичный для данного проекта синтаксис дискретного автомата на два состояния приведен ниже, где p это указатель на текущий модуль:

```
//считаем дискретный автомат
if (p->state_prev != p->state) //смена состояния?
      p->E = 1;//Возводим флаг первого вхождения "entry"
else
      p->E = 0;//иначе сбрасываем
p->state_prev = p->state;

switch (p->state) {//в зависимости от текущего состояния
case PROT_ON_OK: {//Норма
      if (p->E == 1) {  //Первое вхождение
            cmd.all = 0;// обнулим командное слово
      }
      //есть аварии?
      if ((p->masked_bit_fault1 | p->masked_bit_fault2) != 0) {
            p->state = PROT_FAIL; //переходим в состояние аварии
      }
      break;
}

case PROT_FAIL: { //состояние аварии (сработала защита)
```

```
        if (p->E == 1) {//Первое вхождение
              pwm12.Off(&pwm12); //Выключение ШИМ.
        }
        if (cmd.bit.trip_reset == 1)//команда на сброс аварии
              p->state = PROT_ON_OK;//идем в "норма"
        break;
}
}
```

Подробнее о дискретных автоматах можно прочитать в общеобразовательных статьях, которые можно в большом количестве найти в сети Интернет по запросу «конечные автоматы на практике». Например, статья Козаченко В.Ф. «Эффективный метод программной реализации дискретных управляющих автоматов во встроенных системах управления» (http://motorcontrol.ru/wp-content/uploads/2015/11/state_mashine.pdf).

3.5. Прерывания и их обслуживание

В проекте ПО используется ряд прерываний микроконтроллера. Все прерывания системы (за исключением CAN), конфигурируются в модуле SM_Sys. Существует два прерывания для выполнения задач реального времени. В них выполняется большая часть программного кода. Эти два прерывания следующие:

1. **Прерывание «10кГц».** Таймер общего назначения NT_TIMER0 используется для создания прерывания с частотой 10кГц. В обработчике этого прерывания рассчитывается основная часть системы управления: модули, реализующие саму структуру системы управления, модуль ШИМ, модуль защит, модуль датчика положения ротора и т.п. При этом частота ШИМ инвертора может быть любой и не привязана к данному прерыванию.
2. **Прерывание «1кГц».** Используется таймер общего назначения NT_TIMER1. Прерывание необходимо для задач, не требующих большой скорости выполнения, однако требующих строгую дискретизацию по времени. В нем обрабатываются различные сетевые интерфейсы и различные дискретные автоматы.

Третий таймер общего назначения NT_TIMER2 используется для расчета длительности обработки прерываний. Счетчик таймера тактируется с частотой работы ядра микроконтроллера. Значения счетчика таймера в начале и конце процедуры вычитаются, в результате получается длительность обработки прерывания в тактах.

Кроме этих прерываний в системе присутствует прерывание, возникающее при аппаратной аварии инвертора, три прерывания модулей захвата, прерывание модуля квадратурного энкодера и два прерывания драйвера CANopen.

В системе прерываний ПО реализована явная иерархия по приоритету прерываний. Например, прерывание 10кГц должно быть строго приоритетнее прерывания 1кГц. Если бы было наоборот, то, например, если во время обработки прерывания «10кГц» возникнет прерывание «1кГц», то во все время выполнения программного кода из обработчика «1кГц» прерывание 10кГц будет заблокировано, т.е. ключи инвертора «заморозятся», модуль защит прекратит работать и т.п. Поэтому все прерывания распределены по приоритетам, и более приоритетное прерывание не может быть приостановлено менее приоритетным. Приоритеты задаются в виде констант в файле DSP.h. Прерывания в порядке убывания приоритета представлены в списке:

1. Прерывание по аппаратной аварии инвертора
2. Прерывание модуля захвата или модуля квадратурного энкодера (имеют одинаковый приоритет)
3. Прерывание с частотой 10 кГц
4. Прерывание модуля CAN
5. Прерывание с частотой 1 кГц

На Рис. 3.1 показана диаграмма прерываний, их приоритетов и решаемых в них задач.

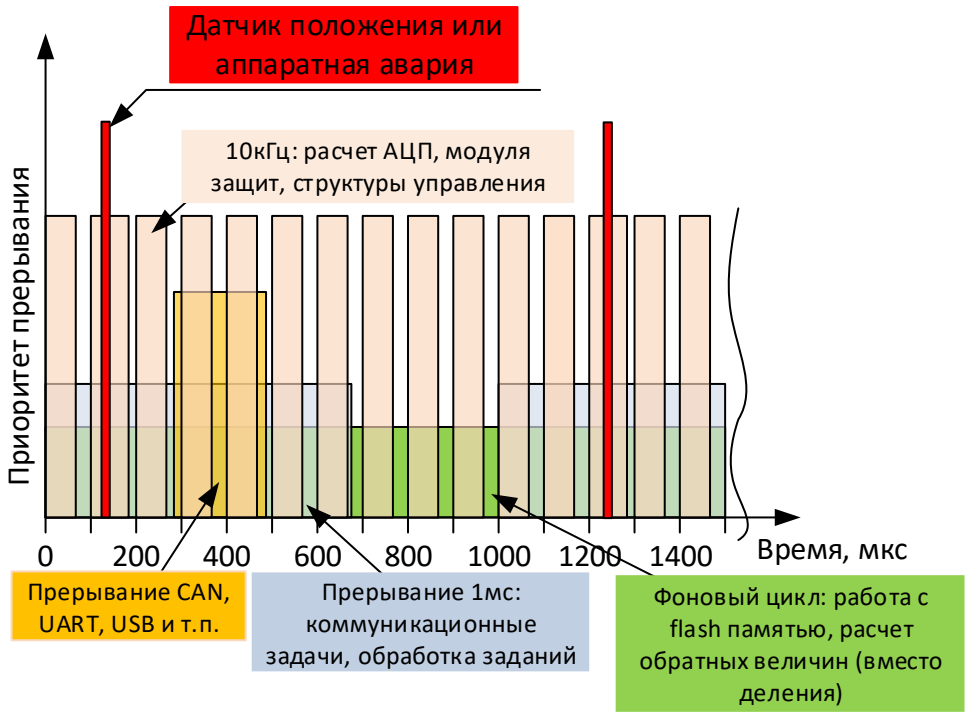


Рис. 3.1 Распределение задач по прерываниям

Существует немного другой, также широко распространенный подход для организации прерываний для систем управления электродвигателями: создается прерывание на таймере, ответственном за генерирование ШИМ, в которое (вместо прерывания 10кГц данного ПО) помещается весь программный код структуры управления электродвигателем. В этом подходе есть как плюсы, так и минусы.

Плюсы:

1) Управляющее воздействие на задание напряжения формируется с той же частотой, с которым его может воспроизвести инвертор: это гарантирует максимально возможное быстродействие системы управления.
2) Отсутствие биений на квазичастоте по сравнению с решением асинхронного прерывания. В случае близких, но не равных частот ШИМ и расчета системы управления формирование задания напряжения будет все время «плыть» относительно частоты ШИМ инвертора, внося переменную фазовую ошибку в реализацию вектора напряжения. При синхронной частоте расчета СУ и ШИМ такой проблемы нет.

Минусы:

1) Строгие ограничения на допустимую частоту ШИМ, чтобы гарантированно «успеть» произвести расчет системы управления на этой частоте. Такой подход часто невозможен для высоких частот ШИМ порядка нескольких десятков кГц.
2) Проблемы с изменением частоты ШИМ при работе: необходимо пересчитывать все коэффициенты интегрирования, коэффициенты регуляторов и производных дифференциальных уравнений, так как частота расчета модулей меняется.
3) Сложность в отладке при возможности изменения частоты ШИМ: необходимо проводить специальные тесты на максимальной частоте ШИМ, чтобы убедиться в достаточной производительности ПО при расчете системы управления на такой частоте.

Так как данный проект ориентирован на общую задачу электропривода без привязки к конкретному инвертору, частоте ШИМ и требований, то был выбран асинхронный подход в реализации прерываний расчета системы управления и ШИМ.

3.6. Управление распределением памяти

В проекте принципиально не используется динамически выделяемая во время работы память из кучи (heap), т.е. не используются функции языка Си malloc, free и т.п. Работа с

динамической памятью требует от программиста особого внимания для слежения за переполнением и исключения утечек памяти, а последствия такой ошибки для электротехнической установки могут быть катастрофическими. Кроме того, проблема с утечкой памяти может проявляться в очень специфических условиях и сложно воспроизводиться и отлаживаться. С другой стороны, система управления приводом обычно не содержит задач, требующих динамического выделения памяти. Поэтому в любом месте, где может потребоваться динамическое выделение памяти строго рекомендуется заменять её на предварительно выделенный на этапе компиляции статический блок памяти (массив на N элементов) и работа с ним. Если же свободной памяти микроконтроллера не хватает, то нужно реорганизовать работу с данными или перенести такие сложные задачи на другое, более высокоуровневое процессорное устройство, не управляющее напрямую элементами электротехнической установки.

В проекте не используются операционные системы и какие-либо библиотеки абстракции, библиотеки работы с периферией. Авторы данного ПО придерживаются идеи, что управление периферией микроконтроллера через битовый доступ к регистрам уже достаточно простое, а использование библиотеки абстракции введет лишь новую сущность для изучения и снизит гибкость в использовании периферии. Операционные же системы реального времени хотя и могут быть использованы, но не столь необходимы для управления электроприводами, при этом они снизят производительность за счет своих накладных расходов. Основные плюсы операционных систем раскрываются при использовании динамического выделения памяти, которое по указанным выше причинам не используется.

Для разметки памяти микроконтроллера в среде VectorIDE используется файл редактора связей build.ld, написанный на скриптовом языке «линкера» комплекта утилит GCC (The GNU linker). Файл содержит большое число комментариев, но полное понимание может быть достигнуто только после знакомства с документацией на утилиту ld, которая может быть найдена, например, в директории со средой разработки по адресу VectorIDE/CodeBench/ CodeBench\share\doc\arm-arm-none-eabi\pdf\ld.pdf. Также рекомендуется ознакомиться с общеобразовательными статьями по данной теме, которые можно найти в сети Интернет, например, по запросу «ld GCC ARM».

В среде разработки CodeMaster распределение памяти не конфигурируется отдельно и оставлено заданным «по умолчанию».

3.7. Организация объявления глобальных переменных

В начале файла main.c объявляются все программные модули и их инициализаторы. В комментариях написано назначение каждого модуля. Эти модули объявлены как со спецификатором extern в заголовочном файле main.h. Подключив такой заголовочный файл во все файлы программы можно обращаться к любому модулю программы из любого места – модулям АЦП, ШИМ, модулю защит и т.п.

Такой подход имеет свой плюс в простоте, однообразности, но имеет минус во времени компиляции и зависимости каждого файла от общего заголовочного файла main.h (теряется локальность модулей).

3.8. Ход выполнения программы

Выполнение программы для проекта в среде VectorIDE начинается с файла asm\startup_MCP_gcc.S. Это стартовый файл на языке Ассемблера, метка `Reset_Handler` является точкой входа (Entry point), с которой начинается выполнение программы. Далее управление передается на вспомогательные ассемблерные подпрограммы, выполняющих инициализацию переменных, включается поддержка плавающей точки, после чего управление передается на функцию main языка Си в файле main.c. Аналогично для среды CodeMaster выполнение начинается с startup_K1921BK01T.mca.

В начале функции main вызывается функция инициализации тактирования, а затем вызывается функция инициализации главного модуля-обертки всего проекта `sm_sys.init`, в котором вызываются функции инициализации всех остальных модулей проекта.

Внутри данной функции вызываются необходимые функции инициализации всех других модулей программы. Далее разрешаются прерывания макросом EINT, после чего в основном файле запускается бесконечный фоновый цикл **`while(1)`**. После этого этапа ход основной программы идет главным образом тремя путями: в прерывании 1кГц, 10кГц, и вне их в фоновом цикле. Вызовы всех модулей этих трех «потоков» также осуществляются через модуль-обертку *`sm_sys`*. По событиям могут также возникать другие прерывания (CAN, датчик положения и т.п.).

В файле main.c после функции **`main`** также определяются функции-обработчики, вызываемые практически по всем прерываниям в системе (исключение – драйвер CANopen, который содержит обработчик прерывания в своем файле).

Все структуры управления электродвигателями реализованы в файле SM_Ctrl.c в виде дискретного автомата, каждому состоянию которого соответствует та или иная структура управления. Например, структура скалярного управления синхронным и асинхронным двигателем реализуется в состоянии, определяемом константой **`CTRL_RUN_U2F`**, принимает на вход задание частоты вращения, модулем задатчика интенсивности формирует плавную выходную частоту, модулем кривой U(f) находит соответствующее этой частое напряжение, формирует интегратор углового положения для вектора напряжения, при помощи модуля координатных преобразований переводит амплитуду и угол вектора в задание напряжения в декартовых осях и передает задания в модуль ШИМ. Все эти состояния и структуры управления более подробно описаны. Сейчас необходимо отметить, что структура управления двигателем набирается из отдельных готовых модулей, как из конструктора, и не содержит в своем теле других сервисных функций (защиты, обработки датчиков, дискретных входов и т.п.) – эти функции реализованы отдельно и независимо. После этапа инициализации состояние дискретного автомата равно **`CTRL_STOP`**, что соответствует отключенному приводу.

Для обработки команд управления пользователя используется дискретный автомат команд управления, реализованный в файле SM_CmdLogic.c. Его обработка происходит в прерывании 1кГц, он принимает команды запуска и останова, а также следит за состоянием ДА защит (SM_Protect.c), управляя при этом автоматом структур управления SM_Ctrl, запуская его и останавливая по необходимости.

Другие модули программы по большей части работают автономно и рассчитываются постоянно, независимо от того, работает привод или нет. К таким модулям можно отнести:

- Коммуникационные драйверы протоколов CANopen и MODBUS.
- Модуль АЦП (файл V_adc.c).
- Модуль защит (SM_Protect.c).
- Модуль расчета текущих параметров (V_CurPar.c).
- Модуль цифрового осциллографа (V_data_log.c).
- Модули обработки датчиков положения (V_DPR_eCAP.c, V_SSI_Encoder.c, V_QEP.c).
- Модуль управления светодиодами (V_led.c).
- Модели электродвигателей (V_MotorModel.c).
- Модуль часов реального времени (V_RTC_Clock.c).

3.9. Введение в CANopen

В ПО данного контроллера используется протокол CANopen для связи с ПК (программа UniCON) или другими устройствами. Связь может осуществляться штатно через CAN интерфейс, однако также в данном ПО существует драйвер V_CANtoRS.c, осуществляющий взаимодействие по CANopen через последовательные интерфейсы UART и USB. Подробное описание драйвера CANopen приведено в документе **«Описание драйвера CANopen»**. Здесь будут описаны основные положения, которое необходимо знать для начала работы с ПО. Протокол CANopen, используемый в данном ПО, расширен по сравнению с общепринятым стандартом и обладает следующими характеристиками:

- Позволяет соединить и адресовать в одной сети до 127 устройств;
- Реализует сервис expedited SDO – подтверждаемые посылки с 8-ю байтами данных. С помощью них можно как читать, так и записывать данные. Посылки SDO можно послать любому узлу в сети. Посылки содержат номер адресата, данные, контрольную

сумму, адрес объекта (индекс, подындекс) и ряд служебных данных. Используются для конфигурирования и пусконаладки устройства.

- Реализует сервис PDO – неподтверждаемые посылки с 8-ю байтами данных. Данный сервис используется для обмена оперативной информацией между устройствами в рабочем режиме (реализации распределенной системы управления).
- Реализует сервис Heartbeat – не подтверждаемые посылки, сигнализирующие о наличии устройства в сети и передающие его статус.
- Посылки SDO можно послать любому узлу в сети. Посылки содержат номер адресата, данные, контрольную сумму, адрес объекта (индекс, подындекс) и ряд служебных данных.
- Каждое устройство содержит словарь объектов – переменных, к которым имеется доступ через протокол CANopen. Посылки могут адресоваться к любому из объектов. SDO может адресовать только один объект, PDO может адресовать до 4х объектов.
- Каждый объект имеет свой адрес в словаре: индекс и подындекс (двухуровневое дерево). Объект-индекс может как содержать, так и не содержать подындексы. Если он их содержит, он является корневым элементом и не содержит данных пользователя. Объекты - подындексы могут содержать любые данные пользователя. Т.е. индекс либо корневой объект и его подындексы – объекты словаря с данными пользователя, либо он не содержит подындексов и содержит только данные пользвателя.
- Каждый объект имеет формат данных, в котором содержится информация о том, редактируемый это параметр или только для чтения, тип данных: числовой, битовый и т.п., визуализация: числовой параметр, текстовое отображение и т.п., разрядность данных, масштабирующий коэффициент и другое.
- У каждого объекта, в зависимости от формата, могут иметься дополнительные атрибуты (помимо значения и самого формата): максимум, минимум, значение по умолчанию, номер текста. Эти атрибуты хранятся в структуре словаря объектов и принадлежат данному объекту.
- У объекта хранится только идентификационный (уникальный) номер текста. Сами тексты должны храниться в памяти устройства управления данным устройством (пульт управления (далее ПУ), компьютер со специальной программой (UniCON) и т.п.). Это сделано для экономии свободной памяти контроллера. Сам текст – это «название» объекта, понятное пользователю и отображаемое как имя параметра.
- Для дополнительной группировки индексов словаря служащих одной задаче были введены группы параметров. Поэтому были введены группы параметров. Каждая группа имеет свое название. Каждый индекс словаря принадлежит той или иной группе. Если индекс содержит подындексы, то они автоматически попадают в данную группу. Например, группы параметров «Задатчик интенсивности», «Температуры» и т.п. Внутри этих групп, соответственно, будут все объекты словаря, им принадлежащие. Принадлежность определяется в словаре CANopen.
- Вся структура словаря объектов содержится в памяти контроллера. Чтобы внешнее устройство могло адекватно управлять им, необходимо, чтобы в нем также была структура этого словаря. Поэтому был создан специальный интерфейс, позволяющий скачивать (при подключении к устройству) структуру словаря устройства.
- После загрузки словаря объектов он сохраняется на управляющем устройстве (программа UniCON сохраняет словарь в файл), чтобы не требовалась повторная загрузка после.
- Так как вся система управления построена с использованием относительных единиц, а пользователю требуется отображать реальные величины (вольты, амперы), введена идеология масштабирующих коэффициентов. Масштабирующие коэффициенты – отдельные индексы словаря объектов, который обязательно должны присутствовать в словаре объектов. Значение коэффициента – число, на которое требуется умножить число в относительных единицах, чтобы получить реальные единицы. Так, если номинальному току соответствует 100 А, в относительных единицах переменная равна

0.5, то отобразится значение 50А. Кроме того, масштабирующий коэффициент задает отображаемые префикс и размерность: амперы (А), миллисекунды (мс), вольты (В) и т.п. Каждому объекту можно задать любой масштабирующий коэффициент. Всего масштабирующих коэффициентов в настоящий момент 31, чего достаточно для реализации любого проекта.

- Кроме текстов, символизирующих имена параметров, существует такое понятие, как перечисление. Перечисление – массив текстов. Они используются, если значение объекта должно быть текстовым. Например, тип торможения может быть: Выбег, Динамическое торможение, Частотное торможение. На самом деле переменная, отвечающая за тип торможения, будет меняться от 0 до 2, где 0 – Выбег, 1 - Динамическое торможение и т.п. А на ПУ будут отображаться соответствующие тексты перечисления. То, что для отображения переменной необходимо использовать перечисление, также задается в формате объекта.
- Драйвер CANopen несет также функцию загрузки и восстановления параметров из энергонезависимой памяти. Все параметры, которые входят в словарь объектов и не имеют тип «только для чтения» в функции инициализации драйвера CANopen загружаются из энергонезависимой пользовательской флеш памяти. После какой-либо перенастройки привода пользователь с помощью ПУ должен дать команду сохранения, после которой все значения словаря объектов сохраняются на флеш. Кроме того, существует команда загрузки параметров по умолчанию, при которой драйвер CANopen каждому элементу словаря присваивает значение, заданное у него в атрибутах как «значение по умолчанию». Технически эти команды от ПУ передаются через объекты интерпретатора команд.
- В первый момент времени, после соединения устройств в сеть (например, ПК и контроллера) происходит следующее. Каждое из устройств каждую секунду посылает сообщение сервиса Heartbeat (о том, что оно в сети). Как только специальная программа на ПК (UniCON) принимает HeartBeat от контроллера, она запрашивает информацию о нем с помощью сервиса SDO. А именно читает данные с двух объектов (которые обязательно должны присутствовать в словаре), в которых закодирована информация о названии устройства и версии его ПО. После этого либо происходит скачивание словаря объектов, либо, если он уже был скачан для этого устройства, ПК становится готов к работе с устройством.
- При включении контроллера при загрузке параметров из пользоваельской энергонезависимой памяти драйвер CANopen проверяет общую контрольную сумму загруженных данных. Если она не сходится, то драйвер загружает параметры по умолчанию. Не сходиться она может по двум причинам: произошла какая-то ошибка (данные перетерты, отказ работы модуля памяти), а также когда в результате прошивки контроллера структура словаря изменилась. Таким образом, при добавлении, удалении или другом изменении параметров словаря объектов остальные значения параметров после включения контроллера не смогут загрузиться из энергонезависимой памяти и «сбросятся» на стандартные настройки.
- CANopen драйвер имеет возможность вызова заданной пользовательской функции при обращении (чтение или запись с помощью SDO) к любым заданным индексам словаря (или подындексам, в них входящим). Например, если есть необходимость выполнять какую-либо переинициализацию некоторого модуля при изменении его настроек, возможен вызов заданной пользователем функции при обращении ПУ к его параметрам. Функции сообщается, произошло событие чтения или записи. Такие пользовательские функции, вызываемые драйвером, называются callback-функциями.
- Словари объектов для первого и второго CAN определяются в файлах cood1.c и cood2.c. Определяются они в виде нескольких таблиц (массивов), в которых задаются: адреса переменных контроллера, которые соответствуют объектам словаря; форматы, номера текстов, максимумы, минимумы и др.; адреса для сохранения словаря в энергонезависимую память; принадлежность к группам; таблица callback функций. Для

генерирования этих файлов разработано специальное ПО COODEdit редактор словарей объектов CANopen. Описание на данное ПО можно найти в файле «**Описание программы COODEdit»** или в директории с самой программной.

Если требуется найти в ПО контроллера переменную, соответствующую какому-либо параметру на пульте управления (или программе UniCON), это легко сделать с использованием файла cood1.c или cood2.c. Напротив каждой переменной, выводимой в словарь CANopen, в данном файле стоит комментарий с названием параметра. Таким образом, для нахождения соответствия переменной и параметра нужно выполнить поиск в данном файле текста имени параметра или по номеру индекса.

Для уверенного использования программы UniCON рекомендуется ознакомиться с соответствующим руководством пользователя, расположенному в корневой директории с программой (файл **«Руководство UniCON.pdf»**). Без изучения этого документа некоторая часть возможностей программы останется «за кадром», так как не все функции работают интуитивно-понятным образом.

3.10. Форматы вычислений и единицы измерения

Практически весь проект построен на использовании целочисленных вычислений с фиксированной точкой в формате IQ8.24 со знаком, где 8 разрядов отводится под целую часть, а 24 под дробную. Для обеспечения таких вычислений реализована библиотека IQmath, содержащая функции умножения, деления, синуса и т.п. в указанном формате.

Весь проект реализован с использованием относительных единиц (о.е.) измерения. Это означает, что для каждой физической величины – тока, частоты вращения, напряжения и т.п. выбирается некоторая базовая величина, относительно которой нормируются все значения переменных в программе. Например, базовый фазный амплитудный ток по умолчанию в данном проекте равен 5А. Таким образом, если переменная, в которой хранится значение тока в программе равна единице (в о.е.), это обозначает ток в 5А. Такой подход является классическим для встраиваемых систем управления и выбран для исключения проблем переполнения переменных или нехватки точности. При изменении мощности преобразователя достаточно переопределить базовые параметры (скажем, увеличить базовый ток в 100 раз), при этом значения переменных внутри системы управления останутся в тех же рабочих диапазонах.

Обычно базовая величина выбирается близкой к номинальному значению параметра, чтобы обеспечить как запас "вверх" по переполнению, так и вниз по точности. Т.е. стремятся, чтобы относительные единицы были близки к единице при работе системы управления с номинальными значениями. Однако в силу формата 8.24 вполне допустимо в 10-20 раз отойти от базы без последствий в виде переполнения или значительной потери точности.

Программа UniCON, которая осуществляет управление приводом по CANopen, автоматически преобразует относительные единицы в физические и обратно. Так, прочитав из параметра величины тока значение, равное 0.5, программа автоматически умножит его на базовую величину для тока 5А и отобразит пользователю 2.5А. При вводе значения параметра пользователем UniCON, наоборот, поделит введенное значение на базовую величину и отправит данные в микроконтроллер. Посмотреть истинное значение в IQ формате можно выбрав пункт «Инфо» через контекстное меню интересующего параметра в UniCON.

Список базовых значений можно посмотреть в группе параметров UniCON **Базов. парам**. После изменения базовых параметров необходимо дать команду на сохранение значений параметров в энергонезависимую память (через программу UniCON), а затем перезагрузить ПО микроконтроллера, чтобы при инициализации были применены новые базовые коэффициенты. Также необходимо дать команду UniCON на обновление словаря объектов CANopen, чтобы UniCON перечитал новые значения базовых параметров и корректно отображал физические единицы.

Нужно помнить, что при изменении базового коэффициента все уставки и защиты в физических единицах пропорционально изменятся. Например, если защита по максимальному току была равна 2.0 в о.е. (удвоенное значение базы), то при повышении базового тока в десять

раз она все равно останется равной 2.0 в о.е., но будет соответствовать в 10 раз большему току в физической величине. Тоже самое касается частот вращения и остальных величин.

Благодаря применению вычислений в относительных единицах система управления практически нечувствительна к изменению базовых единиц и не использует их в своих расчетах. Исключением является ввод аналоговых данных (оцифрованных данных АЦП: измерение токов, напряжений). Для преобразования аналоговых сигналов в относительные единицы системы управления в модуле АЦП (V_adc.c) используются как значение базовых величин параметра, так и коэффициенты настройки аналоговых датчиков, показывающих, сколько единиц физической величины соответствует максимальному измеряемому коду АЦП. Кроме того, базовые единицы частоты вращения используются при инкрементировании углового положения для формирования вращения вектора тока/напряжения.

Альтернативным способом (по отношению к использованию относительных единиц) является использование физических единиц и формата переменных с плавающей точкой (float). Этот подход имеет ряд преимуществ и недостатков.

Плюсы:

1) Переменные представлены в физических единицах, при отображении их не нужно масштабировать и учитывать относительные единицы, задавать базовые величины и их выбирать.
2) Не нужно использовать специальные целочисленные библиотеки для вычислений.
3) В микроконтроллерах серии 1921ВК01 есть аппаратная поддержка многих операций с плавающей точкой, включая деление и квадратный корень.

Минусы:

1) Вычисления с плавающей точкой в ряде случаев медленнее целочисленных. Особенно это касается отказа от многих «ускоряющих» целочисленных трюков, таких как использование сдвига вместо деления, накладывания маски для ограничения величины переменной и т.п.
2) Реализованные в стандартных библиотеках языка Си тригонометрические и другие функции относительно медленные, необходимо искать им замену: более быстродействующую, но менее точную.
3) Требуется строгое внимание к генерируемому компилятором коду: в ряде случаев может быть использована не ожидаемая программистом медленная реализация вычислений, которая займет в несколько раз больше тактов, чем могла бы (например, при использовании константы 0.123L вместо 0.123f, или при вызове функции sqrt() вместо sqrtf() и т.п.).
4) Требуется строгое внимание к операциям, затрагивающим в одном выражении очень большие и очень малые величины. Например, цифровой интегратор может перестать интегрировать, если к большому накопленному значению пытаться прибавлять маленькое приращение. Такого типа ошибки более сложны для поиска и отладки, чем более «грубое» переполнение значения целочисленной математики, которое проще обнаружить.

В проекте MotorControlDemo в формате с плавающей точкой реализованы математические модели двигателей (модуль TMotorModel, файлы V_MotorModel.h и c). Плавающая точка использована из-за сложностей выбора базовых единиц для разных типов промежуточных вычислений внутри модели, а также большого разброса параметров моделируемых двигателей.

4. Быстрый старт

Следование данной главе должно в конечном итоге привести к возможности запустить электродвигатель (или его модель) в простейшем режиме работы (скалярном управлении) – без пояснения принципа работы программных модулей и описания системы управления. Подробное описание модулей ПО будет приведено главе 5, и читатель выборочно сможет ознакомиться с интересными для его задачи частями документа.

4.1. Установка среды разработки

Проект MotorControlDemo выполнен в средах разработки VectorIDE и CodeMaster. Далее пошаговая инструкция по запуску проекта приведена для VectorIDE, для среды CodeMaster следует пользоваться его руководством по эксплуатации для частей касательно компиляции проекта и программирования flash памяти.

VectorIDE является сборкой из свободных продуктов: IDE Eclipse, компилятора GCC, плагина GNU ARM Eclipse и отладчика OpenOCD. Полнофункциональная среда может быть скачана в сети Интернет по адресу http://motorcontrol.ru/production/soft/vector-ide/ . Среда по умолчанию поддерживает два типа наиболее популярных и доступных JTAG адаптеров для ARM: JTAG-эмулятор J-link (или совместимый) и ST-Link V2 (используемый JTAG выбирается при установке). При желании пользователь может осуществить перенос проекта (портировать) на любую другую среду разработки. Проще всего это сделать для среды, основанной на компиляторе GCC.

Сама работа интуитивно понятна и схожа с функционалом других популярных сред, однако при возникновении специфичных проблем, смене JTAG адаптера, отладочной конфигурации и т.п. рекомендуется обратиться к руководству, находящемуся в корневой директории с установленной средой (файл «**VectorIDE_user_manual.pdf**»).

Проект необходимо открыть в среде разработки. Eclipse поддерживает концепцию рабочих пространств (workspace), в которые импортируются проекты. Поэтому (если этого не было сделано при запуске) необходимо создать свой workspace (или открыть стандартный «C:\WorkSpaces\VectorIDE_Examples»), импортировать проект MotorControlDemo через меню «File->Import->General->Existing Projects into Workspace», указать путь до проекта в поле “Select root directory”, выбрать проект MotorControlDemo, при желании поставить галочку копирования проекта в workspace и нажать кнопку Import. Следует проверить, что после импорта директории src, Vsrc и другие не пусты. Если они пусты, следует повторить процедуру импорта без установки галочки копирования проекта в workspace.

4.2. Выбор аппаратной конфигурации

В файле Vinclude/build.h необходимо выбрать версию аппаратной части путем комментирования или раскомментирования соответствующей константы. Поддерживаемый список аппаратных конфигураций растет, на момент написания данного руководства поддерживаются отладочные платы VectorCARD и отладочная плата НИИЭТ. Для некоторых аппаратных конфигураций можно указать сборку проекта для управления как моделью электродвигателя, так и реальным электрооборудованием (например, для платы VectorCARD и различным доступным инверторам для нее), а для некоторых конфигураций доступна только работа с математической моделью. Необходимо ознакомиться с доступными на текущий момент опциями конфигурации в начале файла build.h и выбрать одну желаемую.

Если необходимо осуществить перенос проекта на свою неподдерживаемую в текущий момент версию аппаратной части, то есть два варианта:

1) Сделать адаптацию проекта для работы только математических моделей, это проще всего и затронет всего несколько строк программного кода.
2) Сделать адаптацию проекта для управления реальным инвертором и электродвигателем – в этом случае придется переопределить множество мест программы.

Таким образом, следующие пункты нужны только тем пользователям, которые желают запустить данное ПО на собственном аппаратном обеспечении (собственной плате контроллера).

4.2.1. Адаптация ПО для работы с математическими моделями

Для адаптации проекта MotorControlDemo под неподдерживаемую аппаратную конфигурацию для работы с математическими моделями двигателей необходимо выполнить следующий порядок действий:

1) Выбор наиболее близкой из существующих аппаратной конфигурации, использующей работу с моделями двигателей.
2) Настройка тактирования в функции InitCLK файла src\K1921VK01T_init.c с учетом установленного на используемой плате модуля тактирования.
3) При работе с программой UniCON через UART – настройка выводов желаемого UART модуля в функции CANtoRS_init файла V_CANtoRS.c.
4) При работе с программой UniCON через CAN – настройка выводов желаемого CAN модуля в файле CANopenUDfuncs.c согласно руководству «Описание драйвера CANopen».
5) При желании настройка управления светодиодами в файле V_led.c. Как минимум необходимо проверить, что управление светодиодами не конфликтует с каким-либо другим предназначением выводов, либо просто отключить (закомментировать) модуль.

4.2.2. Адаптация ПО для работы с реальным инвертором и электродвигателем

Рекомендуется предварительно сконфигурировать ПО для работы с моделями двигателей, научиться работать с моделью, запустить желаемую структуру управления на модели, а затем только переходить к работе с реальным оборудованием.

В текущем описании планируется, что будет осуществлено управление стандартным шестиключевым инвертором напряжения с двумя датчиками тока и одним датчиком напряжения Udc. Если необходима более сложная аппаратная часть, пользователю необходимо проводить доработку ПО самостоятельно.

Для переконфигурирования ПО под реальное оборудование сначала необходимо выполнить действия, необходимые для работы математических моделей (перечислены в пункте выше), но за основу взять какую-либо существующую аппаратную конфигурацию для реального инвертора и электродвигателя (например, конфигурацию `HW_VECTORCARD_DRV8301_EVM`).

Далее выполнить следующие действия:

1) Перейти к следующим пунктам данного описания и проверить работоспособность интерфейсов связи и программы UniCON.
2) Модифицировать модуль АЦП в файле V_adc.c. В конце функции AdcDrv_init выбрать номера каналов АЦП для датчиков тока и напряжения согласно требуемой схеме по аналогии с уже существующими конфигурациями. Проверить работу внесенных изменений, проимитировав аналоговые сигналы и наблюдая изменение соответствующих величин в программе UniCON. Воспользоваться рекомендациями п.п. 4.8 Настройка модуля АЦП.
3) Модифицировать модуль ШИМ в файле V_PWM_Module.c. В текущем проекте используются аппаратные блоки ШИМ NT_PWM0, NT_PWM1, NT_PWM2 для управления фазами A,B,C соответственно. Необходимо изменить номера блоков ШИМ во всем файле при необходимости, если на пользовательской аппаратной конфигурации задействованы другие блоки. В конце функции PWM_Module_Init необходимо переконфигурировать выводы ШИМ микроконтроллера в соответствии с необходимыми. В файле SM_Protect.h переназначить константу DRV_FAULT на вход микроконтроллера, соответствующий приему аппаратной аварии от инвертора. Внимание! Это программная обработка аварии, если нужна быстрая аппаратная реакция, то необходима дополнительная активация функции TZ блока ШИМ. Перед запуском на реальном оборудовании работу ШИМ рекомендуется тщательно проверить на осциллографе в разных режимах работы, убедиться в правильно генерировании мертвого времени, реакции на аппаратную аварию и т.п.

4) При необходимости работы инкрементального энкодера требуется модифицировать файл V_QEP.c: заменить модуль NT_QEP1 на тот, который использован в пользовательской аппаратной конфигурации. Также требуется соответствующим образом переинициализировать выводы микроконтроллера в конце функции TposspeedEqep_init. Проверить работу датчика можно наблюдением переменных скорости и углового положения в группе ДПР-Энкодер через программу UniCON.
5) При необходимости работы датчика положения на трех дискретных элементах (Холла или другого типа) в файле V_DPR_eCAP.c необходимо заменить текущие модули NT_CAP0, NT_CAP1, NT_CAP2 на желаемые, аналогичную замену провести в файле main.c в обработчиках прерываний, а также переконфигурировать соответствующие прерывания в функции SM_Sys_Init файла SM_Sys.c. Также необходимо переинициализировать выводы микроконтроллера в конце функции DPReCAP_Init. Проверить работу датчика можно наблюдением переменных скорости и углового положения в группе «ДПР-Холл» через программу UniCON.

Для управления реальным оборудованием, скорее всего, потребуется еще множество доработок, которые не описаны в данном документе. Но базовый функционал – работа АЦП, инвертора, датчиков положения – должен быть обеспечен.

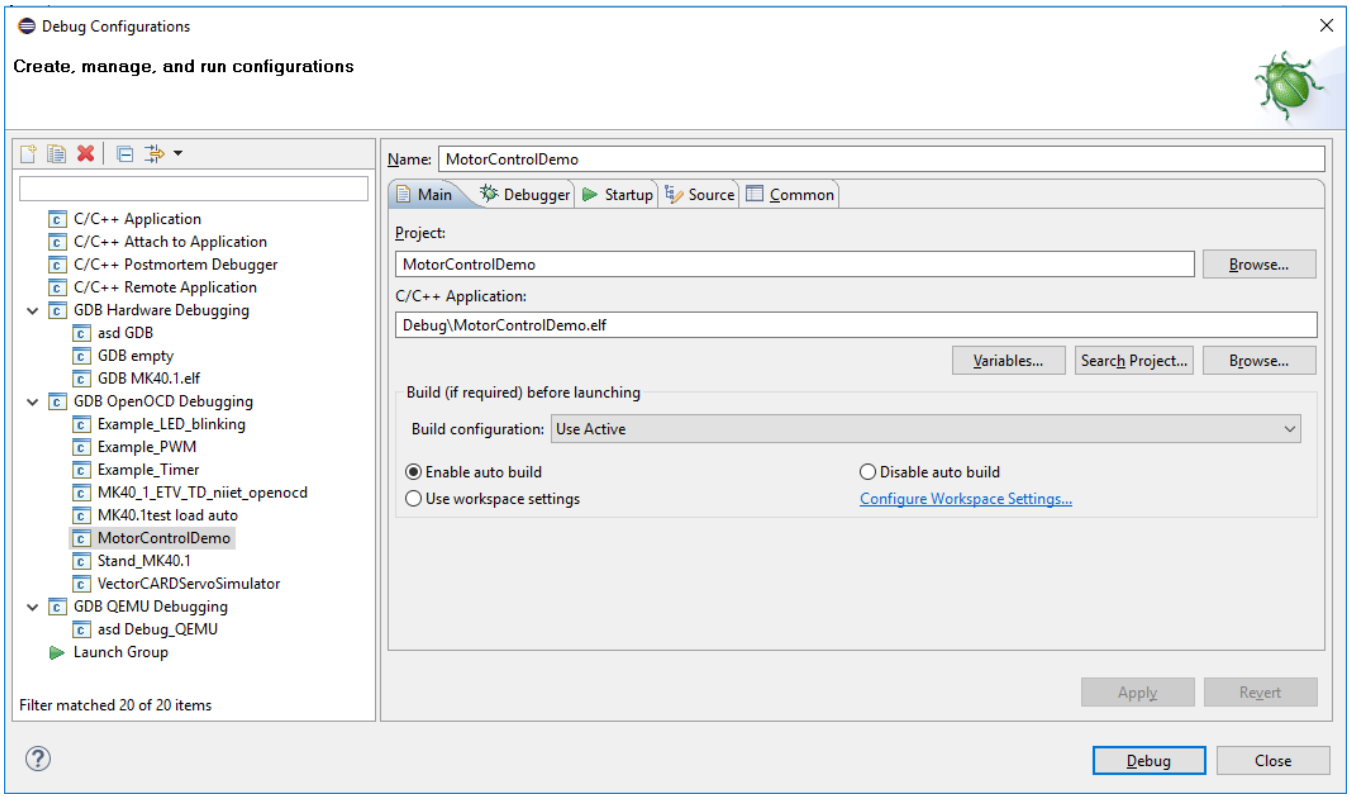
4.3. Программирование и запуск ПО

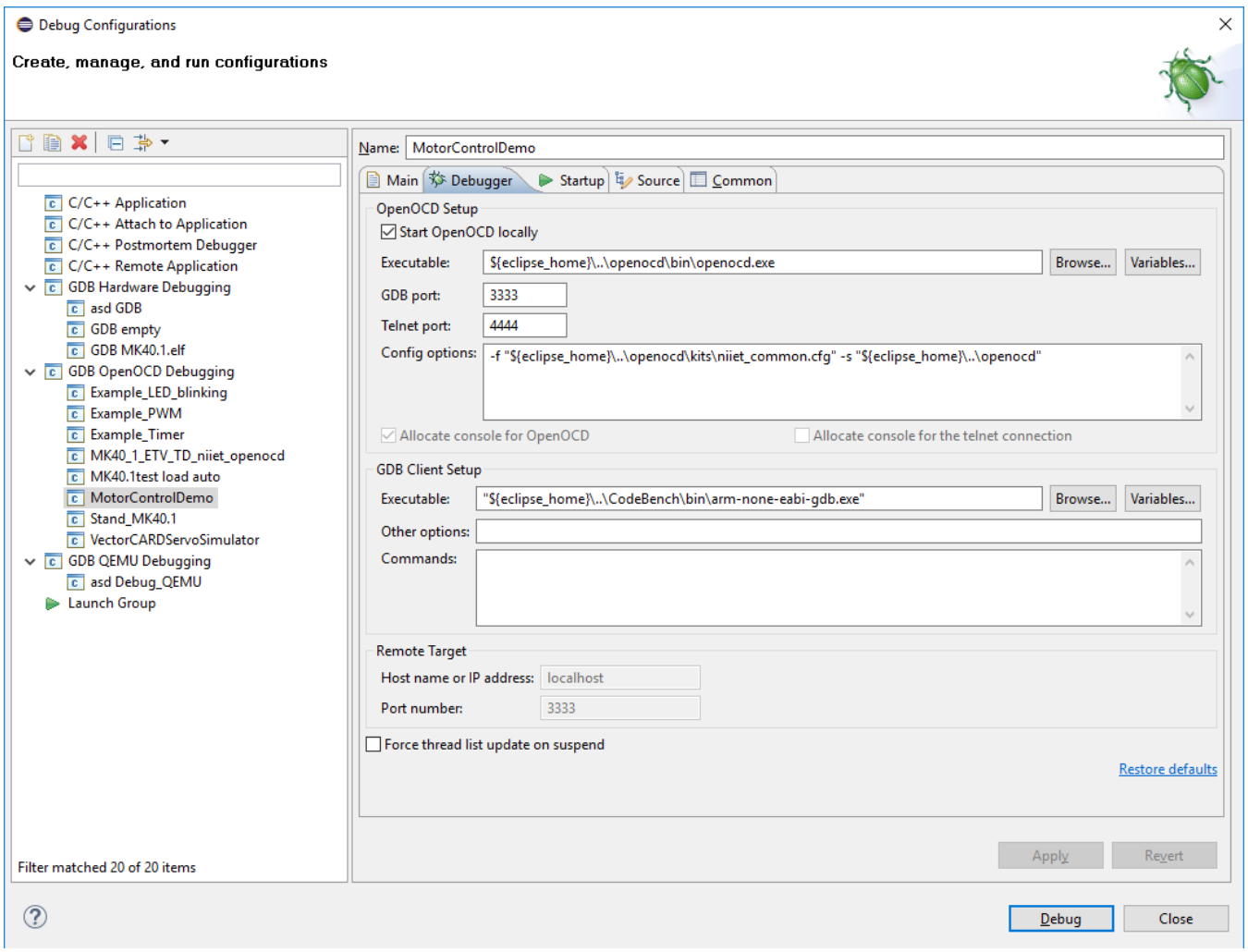
После конфигурации аппаратной части нужно запустить сборку проекта (компиляцию) в среде разработки. Необходимо убедиться, что проект собран без ошибок. В консоли среды разработки должна отображаться информация, похожая на приведенную:

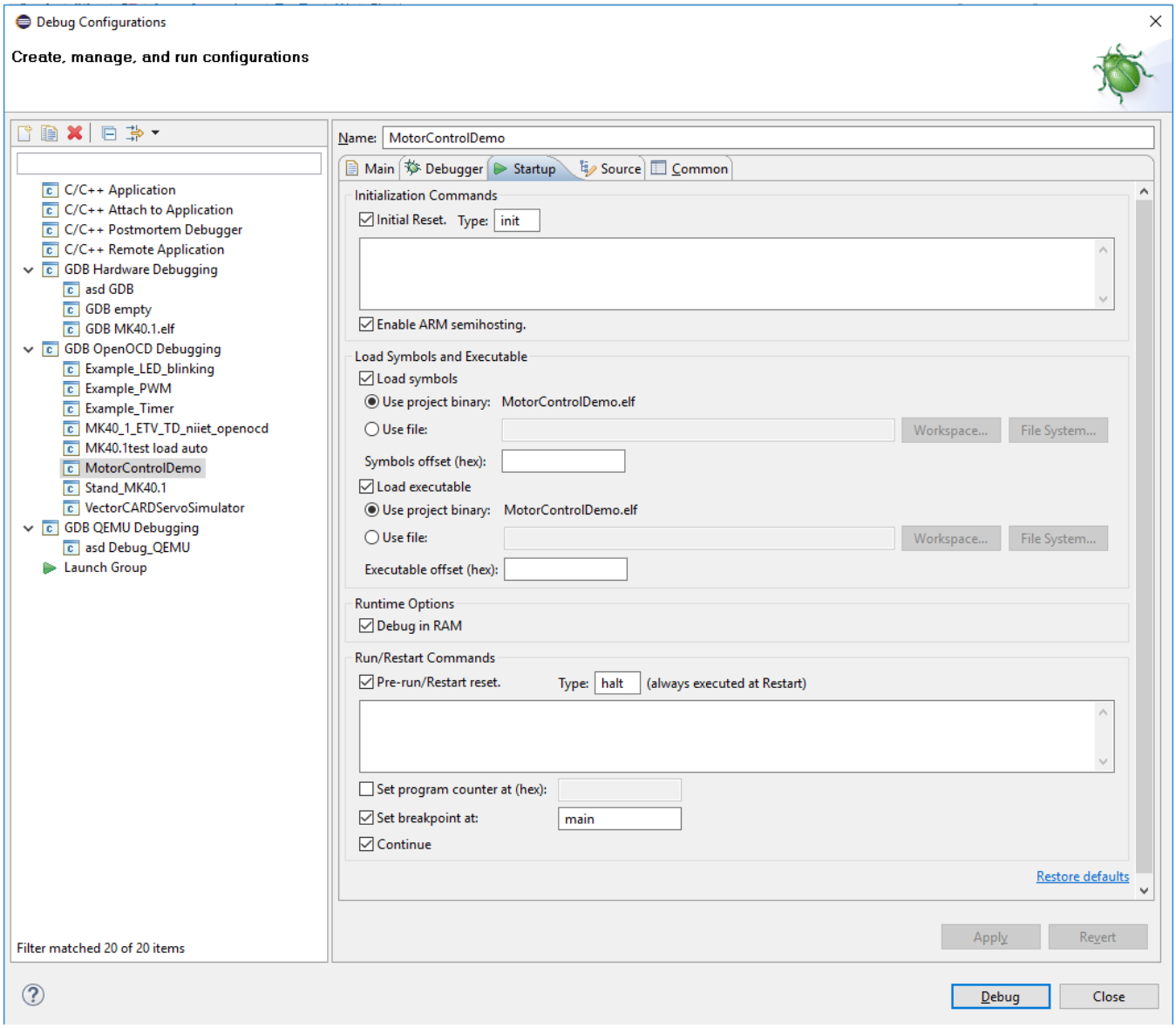
```
   text        data         bss         dec         hex     filename
  98232        9572      129148      236952       39d98     MotorControlDemo.elf
Finished building: MotorControlDemo.bin
Finished building: MotorControlDemo.siz
```

Если сборка происходит с ошибками, то необходимо обратиться к руководству пользователя на VectorIDE – там описаны рекомендации для исправления самых встречаемых проблем (файл «**VectorIDE_user_manual.pdf»** в директории с установленной средой разработки).

Убедившись, что процесс завершился без ошибок, необходимо выполнить программирование (прошивку) контроллера. Для этого нужно создать отладочную конфигурацию, нажав на угловую стрелочку с пиктограммой жука, там выбрав «Debug Configurations…», там в разделе «GDB OpenOCD Debugging» найти готовую конфигурацию от другого проекта и скопировать её пунктом «Dublicate» контекстного меню. В скопированной конфигурации изменить на вкладке Main название проекта и путь до выходного elf файла. Если по какой-то причине готовой конфигурации для проекта нет, необходимо создать новую. Ниже на рисунках показано содержимое всех окон с настройкой отладочной конфигурации.


Debug Configurations
Create, manage, and run configurations
C/C++ Application
C/C++ Attach to Application
C/C++ Postmortem Debugger
C/C++ Remote Application
GDB Hardware Debugging
asd GDB
GDB empty
GDB MK40.1.elf
GDB OpenOCD Debugging
Example_LED_blinking
Example_PWM
Example_Timer
MK40_1_ETV_TD_niiet_openocd
MK40.1test load auto
MotorControlDemo
Stand_MK40.1
VectorCARDServoSimulator
GDB QEMU Debugging
asd Debug_QEMU
Launch Group
Filter matched 20 of 20 items
Name: MotorControlDemo
Main
Debugger
Startup
Source
Common
Project:
MotorControlDemo
Browse...
C/C++ Application:
Debug\MotorControlDemo.elf
Variables...
Search Project...
Browse...
Build (if required) before launching
Build configuration: Use Active
Enable auto build
Disable auto build
Use workspace settings
Configure Workspace Settings...
Apply
Revert
Debug
Close


Debug Configurations
Create, manage, and run configurations
C/C++ Application
C/C++ Attach to Application
C/C++ Postmortem Debugger
C/C++ Remote Application
GDB Hardware Debugging
asd GDB
GDB empty
GDB MK40.1.elf
GDB OpenOCD Debugging
Example_LED_blinking
Example_PWM
Example_Timer
MK40_1_ETV_TD_niiet_openocd
MK40.1test load auto
MotorControlDemo
Stand_MK40.1
VectorCARDServoSimulator
GDB QEMU Debugging
asd Debug_QEMU
Launch Group
Filter matched 20 of 20 items
Name: MotorControlDemo
Main
Debugger
Startup
Source
Common
OpenOCD Setup
Start OpenOCD locally
Executable:
${eclipse_home}\..\openocd\bin\openocd.exe
Browse...
Variables...
GDB port:
3333
Telnet port:
4444
Config options:
-f "${eclipse_home}\..\openocd\kits\niiet_common.cfg" -s "${eclipse_home}\..\openocd"
Allocate console for OpenOCD
Allocate console for the telnet connection
GDB Client Setup
Executable:
"${eclipse_home}\..\CodeBench\bin\arm-none-eabi-gdb.exe"
Browse...
Variables...
Other options:
Commands:
Remote Target
Host name or IP address:
localhost
Port number:
3333
Force thread list update on suspend
Restore defaults
Apply
Revert
Debug
Close


Debug Configurations
Create, manage, and run configurations
C/C++ Application
C/C++ Attach to Application
C/C++ Postmortem Debugger
C/C++ Remote Application
GDB Hardware Debugging
asd GDB
GDB empty
GDB MK40.1.elf
GDB OpenOCD Debugging
Example_LED_blinking
Example_PWM
Example_Timer
MK40_1_ETV_TD_niiet_openocd
MK40.1test load auto
MotorControlDemo
Stand_MK40.1
VectorCARDServoSimulator
GDB QEMU Debugging
asd Debug_QEMU
Launch Group
Filter matched 20 of 20 items
Name: MotorControlDemo
Main
Debugger
Startup
Source
Common
Initialization Commands
Initial Reset. Type: init
Enable ARM semihosting.
Load Symbols and Executable
Load symbols
Use project binary: MotorControlDemo.elf
Use file:
Workspace...
File System...
Symbols offset (hex):
Load executable
Use project binary: MotorControlDemo.elf
Use file:
Workspace...
File System...
Executable offset (hex):
Runtime Options
Debug in RAM
Run/Restart Commands
Pre-run/Restart reset. Type: halt (always executed at Restart)
Set program counter at (hex):
Set breakpoint at: main
Continue
Restore defaults
Apply
Revert
Debug
Close

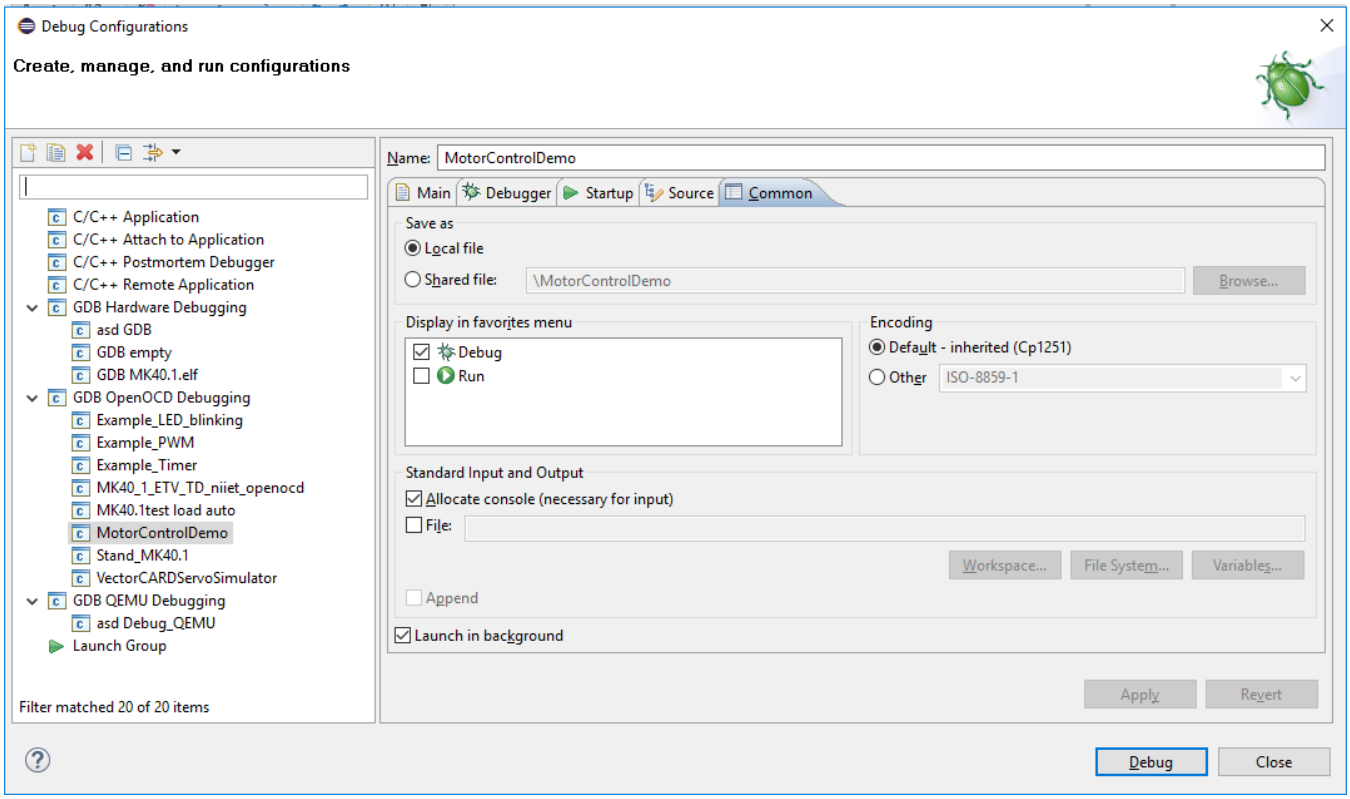


Рис. 4.1 Содержимое окон настройки отладочной конфигурации

Содержимое текстовых полей во вкладке «Debugger»:

```
${eclipse_home}\..\openocd\bin\openocd.exe
-f "${eclipse_home}\..\openocd\kits\niiet_common.cfg" -s "${eclipse_home}\..\openocd"
"${eclipse_home}\..\CodeBench\bin\arm-none-eabi-gdb.exe"
```

Далее нужно подключить JTAG-эмулятор, установить на него драйвера, при необходимости сконфигурировать файл настройки JTAG в среде разработки (`VectorIDE \openocd\kits\niiet_common.cfg`, подробнее смотрите в инструкции на среду разработки) запустить процесс отладки, щелкнув в VectorIDE по стрелочке рядом с кнопкой Debug (с зеленым жучком) и выбрав Debug-конфигурацию «MotorControlDemo Debug». При этом среда автоматически выполнит подключение к микроконтроллеру, стирание и программирование его флеш памяти, запуск проекта. В результате должна открыться отладочная сессия, указатель текущей выполняемой инструкции должен быть установлен на начале функции `main`:

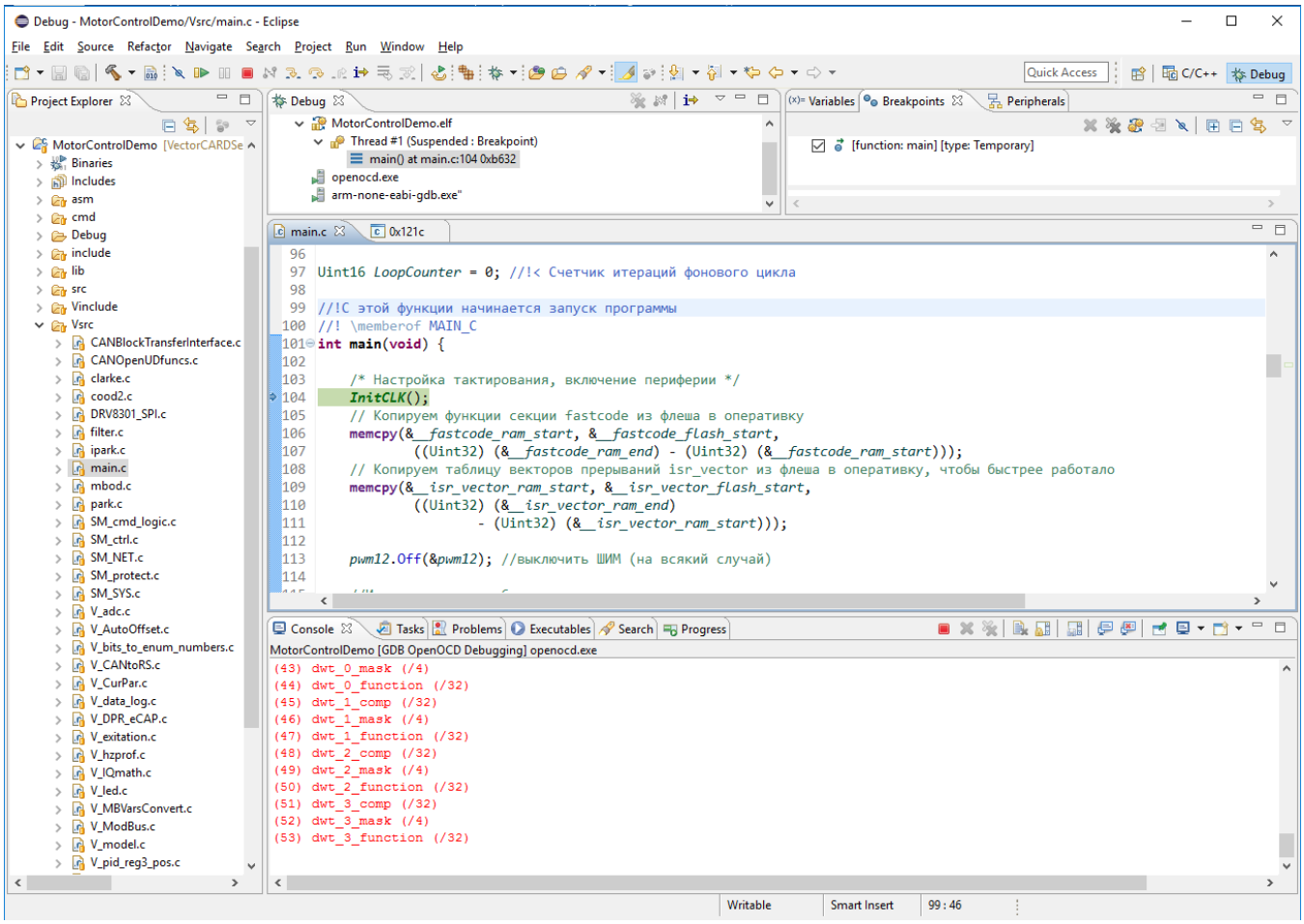


Рис. 4.2 VectorIDE с запущенной отладочной сессией проекта MotorControlDemo

Далее необходимо запустить программу на постоянную работу зеленой кнопкой Resume на панели отладки.

Если с подключением возникли проблемы, изучите инструкцию на VectorIDE или обратитесь с вопросом в службу поддержки.

4.4. Подключение к ПК через UniCON

Программа ПК UniCON, работающая под ОС Windows, используется как пульт управления приводом, средство настройки параметров системы управления и просмотра осциллограмм. Есть два способа подключения UniCON:

1) Штатно через CAN интерфейс. Для этого используемая плата должна содержать CAN выход, а у пользователя иметься один из CAN адаптеров, поддерживаемых программой UniCON (см. руководство пользователя в директории с программой). При этом драйвер CANopen будет работать полнофункционально, загрузка осциллограмм проходить быстро, в сеть CAN можно подключить несколько устройств одновременно.
2) Через UART интерфейс (RS232, USB-UART переходник и т.п.), представляемый в компьютере как COM порт (настоящий или виртуальный). В этом случае драйвер CANopen и UniCON работают с ограничениями: используется только SDO сервис CANopen обмена сообщениями, что негативно влияет на время загрузки осциллограмм. Кроме того, в сети может быть только одно устройство. Также в программе UniCON нужно указывать номер виртуального COM порта в параметрах модуля связи, который может произвольно меняться после перезагрузки компьютера или смене USB порта.
3) Через USB интерфейс, представляемый в компьютере как виртуальный COM порт. В этом случае драйвер CANopen и UniCON имеют те же возможности и ограничения, что и при подключении через UART-интерфейс, однако скорость работы через USB выше (примерно на 40%).

Для подключения через интерфейс CAN необходимо:

1) Подключить используемый USB-CAN переходник к плате согласно документации на переходник и плату через трехпроводной кабель (CAN-H, CAN-L, GND). При небольшой длине кабеля в раде случаев допустимо использовать двухпроводную связь. В общем случае на линии также должен быть установлен минимум один терминальный резистор (резистор параллельно CAN-H, CAN-L номиналом порядка 120 Ом).
2) Установить драйвера для ПК на используемый USB-CAN переходник (с официального сайта производителя, для МАРАФОН это http://can.marathon.ru/page/prog/chai).
3) Запустить программу UniCON, открыть в меню Сервис – Параметры модуля связи, выбрать из списка сверху используемый модуль связи, нажать Ок.

Для подключения через интерфейс UART необходимо:

1) Подключить ПК к интерфейсу UART используемой платы любым доступным способом: через интегрированный на плату USB-UART переходник, через внешний переходник, через COM порт компьютера и преобразователь уровней и т.п. Достаточно трехпроводного интерфейса: RX, TX, GND.
2) Установить драйвера для ПК на используемый USB-UART переходник. Убедиться, что в диспетчере устройств Windows появился соответствующий виртуальный или настоящий COM порт. Запомнить номер порта.
3) Запустить программу UniCON, открыть в меню Сервис – Параметры модуля связи, выбрать из списка сверху «USB-RS». Задать слева внизу параметр «Номер COM порта для альтернативных модулей» равным номеру используемого COM порта, нажать Ок.

Для подключения через интерфейс USB необходимо:

1) Подключить ПК к интерфейсу USB соответствующим кабелем.
2) При использовании операционной системы Windows ниже версии 10, необходимо установить драйвер устройства (драйвер доступен в репозитории проекта на BitBucket в разделе Downloads: https://bitbucket.org/niietcm4/motorcontroldemo/downloads/ , название файла – «niietvcp_x32x64»). Устройство определится как «Niiet USB RS-232 Emulation Driver». На системах Windows 10 отдельной установки драйвера не требуется, устройство автоматически определяется как COM-порт.
3) Запустить программу UniCON, открыть в меню Сервис – Параметры модуля связи, выбрать из списка сверху «USB-RS». Задать слева внизу параметр «Номер COM порта для альтернативных модулей» равным номеру используемого COM порта, нажать Ок.

Далее, если программа в микроконтроллере работает, подключение выполнено правильно, модуль связи настроен согласно инструкции, то UniCON должен отобразить текущие устройства в сети, т.е. показать в верхней левой части окна вкладку «Узел N» с названием «ARM_MCP», станет доступна кнопка «Подключиться». После нажатия на неё можно приступать к работе. На Рис. 4.3 показано окно программы UniCON после успешного подключения.

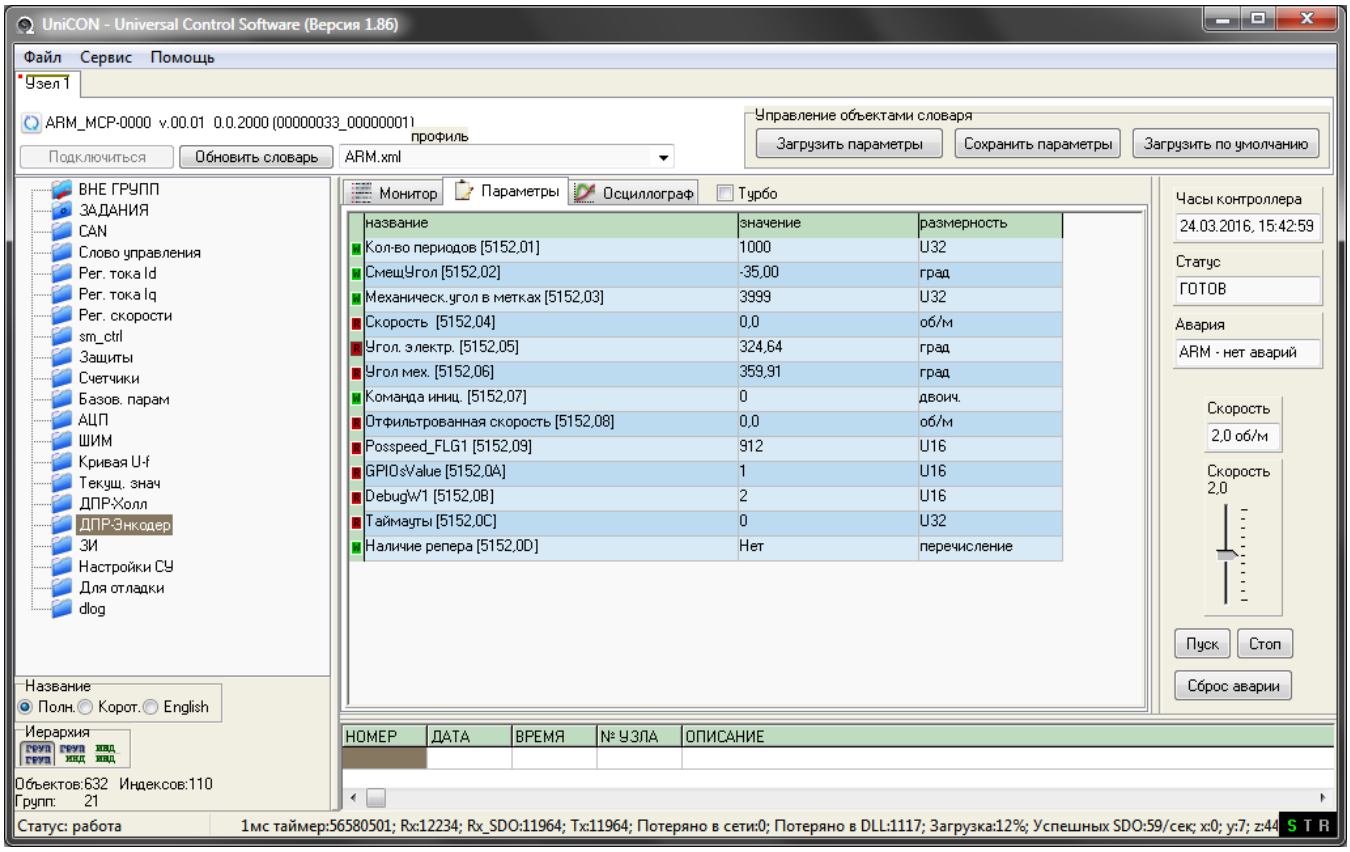


Рис. 4.3 Окно программы UniCON после подключения к устройству

Если подключение не удается, то следующие рекомендации могут помочь в диагностике проблемы:

1) Если программа UniCON не запускается, то можно попробовать временно отключить антивирусные программы (по опыту разработчиков один раз был замечен конфликт с антивирусной программой Avast). Также при первом запуске брандмауэр может спросить о разрешении доступа к сети TCP IP, доступ можно запретить, так как он нужен для предоставления внешнего доступа к сети CAN на чтение другим программам (опцию TCP сервера можно в будущем отключить в настройках).
2) Следует обратить внимание на статус программы UniCON слева снизу. Статус «Работа» информирует об успешном открывании модуля связи и его работе. Если же идет последовательный перебор статусов «ожидание»/«запуск», то необходимо проверить корректность установки драйверов на модуль связи, проверить, что он правильно выбран в параметрах модуля связи в меню UniCON, что модуль связи не задействован в данный момент другим приложением (или другой запущенной копией UniCON).
3) Если устройство не отображается на вкладке устройств, то необходимо проверить работу программы в микроконтроллере, корректность настройки выводов используемого интерфейса связи в микроконтроллере (если используется не поддерживаемая по умолчанию аппаратная конфигурация), корректность подключения кабелей. Микроконтроллер должен каждую секунду отправлять статусные сообщения сервиса Heartbeat CANopen, можно воспользоваться внешним осциллографом и проверить наличие посылок в линии связи.
4) Если устройство отображается как «Неизвестное устройство», то это говорит об успешной передаче посылок от микроконтроллера в UniCON, но об отсутствии связи в обратном направлении.
5) Если вместо одного узла в сети с названием «ARM_MCP» появилось два с названием «Эмулятор», это значит в настройках UniCON вместо переходника связи CAN выбран эмулятор. Откройте в программе UniCON Сервис-Параметры модуля связи. Выберите нужный вам модуль связи.
6) Если ничего из перечисленного не помогает, обратитесь в поддержку.

После успешного подключения к устройству необходимо ознакомиться с интерфейсом программы UniCON. Подробное описание доступно в справке, которую можно открыть как из меню Справка, так и найти файл «**Руководство UniCON.pdf**» в директории с программой. В данном руководстве приведено краткое описание приёмов работы с программой по мере необходимости.

В левой части окна программы Рис. 4.3 расположен список объектов CANopen (параметры устройства), в центре – значения параметров или осциллограф, а в правой – панель управления устройства. Для простоты последующего указания параметра будет применяться такое именование: **Группа.Параметр**. Например, **АЦП.Ток фазы А**. В данном случае параметр – это некоторый объект словаря CANopen. Открывая группы с параметрами можно видеть список параметров в них в средней части окна. Параметры с зелеными пиктограммами можно менять (параметры для записи), с красными – только для чтения. После изменения значения параметра нужно нажимать клавишу Enter.

4.5. Настройка модели двигателя

Этот пункт необходим к прочтению в том случае, если пользователь работает с моделью двигателя (соответствующий #define в файле build.h), а не с реальным оборудованием. В этом случае нужно выбрать, какой именно двигатель будет промоделирован. Выбор типа электродвигателя производится в параметре **Модель общие данные.Тип двигателя модели [5181,01]**, а предустановленный набор параметров двигателя выбирается числом в переменной **Модель общие данные.Параметры двигателя [5181,02]**. Рекомендуется оставить двигатель по умолчанию: асинхронный двигатель с набором параметров 1. При смене типа двигателя или его параметров требуется сохранение параметров контроллера кнопкой «Сохранить параметры» в UniCON и перезагрузка контроллера (перезапуск программы микроконтроллера).

4.6. Настройка базовых значений

Как уже указывалось в пунктах «3.9 Введение в CANopen» и «3.10 Форматы вычислений и единицы измерения», система управления построена в относительных единицах, а значит перед началом работы важно правильно выбрать базовые значения, относительно которых будут нормироваться физические значения для получения относительных единиц. Настройка таких параметров производится в группе Базов. парам. Желательно (но не обязательно), чтобы базовые величины были близки к номинальным. Тем не менее, допускается отклонение даже в несколько раз. Таким образом, если номинальный ток, с которым ожидается работать, не на порядок отличается от текущего значения базового параметра, базовый параметр можно не изменять. Если же отличие существенно и есть риск переполнения значения формата IQ24 или потеря точности, рекомендуется изменить базовые параметры. Рекомендуется давать базовым коэффициентам круглые, легко запоминаемые значения, чтобы в случае необходимости пересчет из относительных единиц и обратно можно было производить в уме. Таким образом, в случае необходимости нужно скорректировать следующие параметры:

1) **Базов. парам.Номинальный ток [5113,02]**, который является базовым значением для фазных токов двигателя.
2) **Базов. парам.Номинальное напряжение ЗПТ [5113,05]**, который является базовым значением для напряжения ЗПТ (Udc).
3) **Базов. парам.Номинальная скорость [5113,04]**, который аналогично определяет масштаб для частоты вращения в об/мин.
4) **Базов. парам.Число пар полюсов [5113,01]**, который должен быт равен паспортному значению числа пар полюсов двигателя и определяет автоматически рассчитываемый базовый параметр частоты тока **Базов. парам.Номинальная частота [5113,08]**.

После изменения базовых параметров требуется сохранение параметров контроллера кнопкой «Сохранить параметры» в UniCON, перезагрузка контроллера (перезапуск программы микроконтроллера), а затем операция обновления словаря UniCON (кнопкой обновить словарь слева сверху) для считывания из микроконтроллера значений новых масштабирующих коэффициентов (базовых параметров).

Для реального оборудования базовые параметры могут быть заполнены исходя из паспортных параметров оборудования. Для модели двигателя параметры текущего двигателя могут быть найдены в группе параметров соответствующего двигателя, например, для асинхронного двигателя это параметры **Номинальная мощность [5182,0C], Номинальная скорость [5182,0D]**, **Номинальный действующий ток [5182,0E]** в группе **Модель (асинхронный)**.

4.7. Настройка модуля защит

Модуль защит позволяет отслеживать все критические ситуации и принимать меры для предотвращения аварии. Проверьте все уставки защит на соответствие ожидаемым значениям. В данном ПО предусмотрены защиты по максимальному амплитудному току в фазе, минимальному напряжению на ЗПТ, максимальному напряжению на ЗПТ и максимальной скорости вращения. Уставки защит при пусконаладочных работах должны быть как можно ближе к реальным ожидаемым значениям – лучше лишний раз убедиться в срабатывании программной защиты и отодвинуть её уровень, чем допустить выхода оборудования из строя. Рекомендуется настроить уровень защиты по напряжению звена постоянного тока в диапазоне 10% от ожидаемого значения (параметры **Защиты.Минимальное напряжение [510E,09]** и **Защиты.Максимальное напряжение [510E,08]**). Защита по максимальному току фазы (**Защиты.Максимальный ток [510E,07]**) должна быть настроена также на уровне 10% выше ожидаемого тока фазы.

Ожидаемые значения – те, которые будут на двигателе при его первом пробном запуске. Если речь идет о модели двигателя, то номинальные данные можно найти в группе с параметрами модели. Номинальное (и текущее) напряжение в модели двигателя и инвертора можно посмотреть в справочном параметре **Модель общие данные.Напряжение ЗПТ [5181,0F]**. Если речь идет о реальном оборудовании, то необходимо руководствоваться его номинальными данными. При первом запуске рекомендуется начать со значений токов привода не выше 10% от номинала и, по возможности, с пониженного напряжения питания.

При выходе параметра за диапазон уставок защит инвертор отключится, а на панели UniCON справа будет индицироваться текст сработавшей защиты на панели «Авария».

4.8. Настройка модуля АЦП

Если модуль обработки данных, который принимает и обрабатывает данные с АЦП микроконтроллера (далее модуль АЦП), настроен неверно, то большинство защит инвертора, не говоря уже об отображении величин, работать не будут. Данную настройку необходимо выполнять как для работы с математическими моделями, так и для работы с реальным оборудованием. Для трехфазного шестиключевого инвертора обычно имеется три главных измеряемых аналоговых величины: два тока фаз и напряжение звена постоянного тока (ЗПТ, оно же Udc).

В данном модуле АЦП каждая измеряемая величина представлена, как минимум, тремя объектами (смотрите группу параметров АЦП в UniCON):

1) Коэффициент усиления. Для каждой величины он задается в соответствующих единицах измерения. Его можно либо найти подбором, либо рассчитать: он равен тому, что должен измерять датчик, когда АЦП измеряет максимальное значение (на контроллер приходит максимальное измеряемое напряжение). Другими словами, если вход АЦП 0 – 3В, на него подается сигнал с датчика напряжения на звене постоянного тока и «0» на входе контроллера соответствует 0 В на ЗПТ, 3 вольта соответствует 100В на ЗПТ, то коэффициент равен 100.
2) Смещение. Если измеряемая величина равна нулю, а реально измеряется не нулевое значение (аддитивная ошибка), можно установить смещение нулевого уровня сигнала. Это смещение задается в тех единицах измерения, в которых возвращает значение сам модуль АЦП, т.е. смещение может быть в диапазоне -4095…+4095.
3) Измеряемое значение. Отображает мгновенное значение измеряемой величины. Так, если датчик измеряет фазный ток двигателя, то в процессе работы из-за быстрого изменения сигнала измеряемое значение будет «дрожать», что связано с периодичностью оцифровки

мгновенной величины («стробоскопический эффект»). При отключенном инверторе в значениях токов фаз должны быть ненулевые величины, колеблющиеся в районе нуля. Они возникают из-за погрешностей измерения и косвенно свидетельствуют о работе АЦП.

Правильно настройку коэффициентов канала модуля АЦП производить по двум точкам, чтобы корректно настроить коэффициент смещения и усиления. Первой точкой удобнее выбрать ноль измеряемого сигнала и подобрать такое смещение, чтобы измеряемое значение также было нулевым. Затем подать некоторый постоянный уровень и подобрать уже коэффициент усиления.

Настройку коэффициентов измеряемых каналов АЦП удобно начать с напряжения на звене постоянного тока (ЗПТ). Считая, что смещение уже настроено верно и равно нулю, рассчитайте или подберите такой коэффициент усиления, чтобы отображаемое измеряемое значение соответствовало реальному напряжению на ЗПТ. Для этого необходимо в программе UniCON изменять значение параметра **АЦП.Коэффициент напряжения ЗПТ [5116,0D]** и наблюдать значение **АЦП.Напряжение ЗПТ [5116,0F]**. При работе математической модели двигателя текущее напряжение ЗПТ отображается в параметре **Модель общие данные.Напряжение ЗПТ [5181,0F]** – АЦП должно измерять тоже самое. В реальном оборудовании необходимо воспользоваться вольтметром.

Калибровку приёма сигналов датчиков тока можно произвести разными способами: имитационным сигналом на измерительный вход АЦП или при работе реального двигателя. Проще всего ввести преобразователь в работу и по амперметру в фазах двигателя настроить коэффициенты усиления **АЦП.Коэффициент тока фазы А [5116,01]** и **АЦП.Коэффициент тока фазы В [5116,04]**. Ток фазы С для трехфазного двигателя настраивать не требуется, так как ток рассчитывается как сумма токов в двух других фазах со знаком минус (по закону Кирхгофа). Так как без настроенных датчиков есть риск не зафиксировать максимально-токовую аварию, или, наоборот, зафиксировать её ложно, желательно предварительно рассчитать коэффициенты усиления согласно описанию (см. выше пункт 1).

Настройка смещения может выполняться автоматически состоянии останова. Для этого нужно включить модуль автосмещения токов АЦП – вписать «1» в параметр **Настройки СУ.АЦП авто смещ.**. Если автосмещение активно, то в остановленном состоянии привода смещение АЦП для токов фаз будет подстраиваться так, чтобы отображался нулевой ток. Во включенном состоянии смещения пересчитываться не будут. Если автоматический расчет смещения отключен, то на выключенном оборудовании необходимо настроить коэффициенты смещения датчиков тока, чтобы токи фаз по показаниям АЦП были около нуля (параметры **АЦП.Смещение тока фазы А [5116,02]** и **АЦП.Смещение тока фазы В [5116,05]**).

Если запуск реального оборудования с ненастроенными датчиками может быть опасен (привод на мощность выше 1кВт и напряжение ЗПТ выше 50В), можно подать имитацию сигнала датчиков тока, рассчитать, какому току это соответствует и подобрать коэффициенты усиления, чтобы измеренное АЦП значение этому соответствовало.

Если же идет работа с низковольтным отладочным комплектом или математической моделью, то можно запустить привод и настроить датчики токов по реальным фазным токам, измеряемым внешним прибором: амперметром, токовыми клещами (в модели значения токов двигателя доступны в параметрах **Модель общие данные.Ток фазы А [5181,0C]** и далее). Для этого нужно произвести пуск двигателя (см. последовательность пуска в п.4.9) и наблюдать в программе UniCON ток в фазах двигателя. Если значения в программе UniCON и на амперметре отличаются, подстроить коэффициенты. Если используется амперметр на постоянный ток, то можно задать нулевую частоту вращения и настроить датчики тока на постоянном токе. Не следует производить настройку на очень малых токах: постарайтесь создать ток не меньше ¼ номинала, иначе настройка может оказаться не очень точной. После окончания настройки сохраните все значения в энергонезависимую память, нажав кнопку «сохранить параметры».

Если идет работа с математической моделью, то можно проверить корректность самостоятельной настройки: в параметре **Модель (асинхронный).Коэффициент АЦП тока фаз [5182,0D]** (и для других двигателей по аналогии) показан «правильный» коэффициент усиления для датчиков тока.

4.9. Запуск привода

У системы управления существует несколько различных структур управления в зависимости от типа электродвигателя и желаемых характеристик. Например, классическая структура с пропорциональным изменением напряжения и частоты (U/f) для асинхронного и синхронного двигателя, частотно-токовая система управления, векторная система управления и т.п. Желаемая структура управления задается её номером в параметре **Настройки СУ.режим работы**. Режиму работы «останов», т.е. отсутствию выбранной структуры соответствует число 0. Режиму работы U/f соответствует номер 3. Перед запуском двигателя необходимо убедиться, что в данном параметре записано это число.

Режим работы U/f обеспечивает традиционное скалярное управление АД/СД с регулированием напряжения по заранее заданной кривой *U/f*. Структура скалярной СУ представлена на Рис. 4.4.

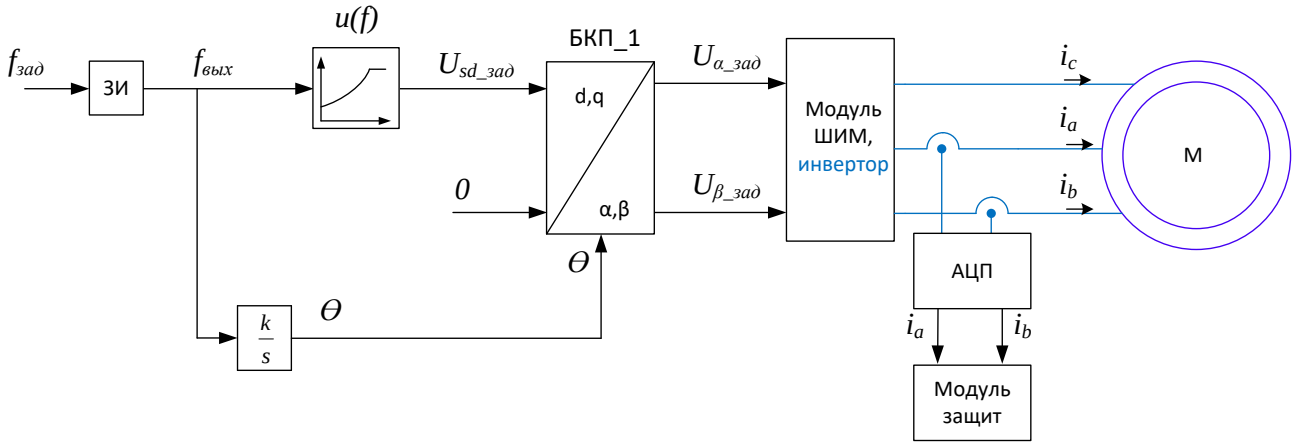


Рис. 4.4 Структура системы скалярного управления

Кривая *U/f* задана двумя опорными точками через следующие параметры группы **КриваяU-f**: **Нулевая опорная частота**, **Нулевое опорное напряжение**, **Первая опорная частота**, **Первое опорное напряжение**. Нулевую точку можно для первого пуска задать нулями, а первую задать значениями, равными номинальным для текущего двигателя. Напряжение задается как фазное, амплитудное. Разгон производится через модуль задатчика интенсивности (темп разгона задается в **ЗИ.Т разгона до номинала [5000,01]**).

Перед первым пуском еще раз проверьте защиту максимального тока фаз в модуле защит (**Защиты.Максимальный ток [510E,07]**) установите её около того значения тока, который вы ожидаете увидеть, т.е. на границе срабатывания. Это позволит проверить срабатывание защит, убедиться, что токи соответствуют ожидаемым значениям и произвести более раннее отключение, если что-то пойдет не по плану.

Для первого запуска можно задать низкое значение частоты вращения: 5-10 об/мин в параметре **Задания.Скорость [5112,01]** или, что более удобно, на панели управления справа.

Также при работе с реальным приводом необходимо сконфигурировать модуль ШИМ: выбрать частоту ШИМ (**ШИМ.Частота ШИМ [5105,0E]**), установить величину мертвого времени (**ШИМ.Мертвое время (мкс) [5105,08]**) согласно документации на аппаратное обеспечение.

Убедившись, что АЦП работоспособно (значение напряжения ЗПТ в модуле АЦП адекватно, значения токов фаз колеблются вблизи нуля), задана верная структура управления и нет проблем в аппаратной части, можно дать команду пуска: для этого нажмите на кнопку Пуск на панели управления в правой части окна UniCON. Привод должен перейти в работу и на панели оперативного управления справа сверху статус должен смениться с «ГОТОВ» на «РАБОТА».

При работе с реальным оборудованием JTAG эмулятор допустимо оставлять подключенным с запущенной сессией отладки, однако не рекомендуется останавливать микроконтроллер во время работы ШИМ.

Для наблюдения токов фаз удобно смотреть их изменение во времени, т.е. осциллограмму или «график». Чтобы добавить интересующий параметр в цифровой осциллограф программы UniCON, нужно щелкнуть по параметру правой кнопкой, выбрать пункт «Добавить в осциллограф» и любой из четырех каналов.

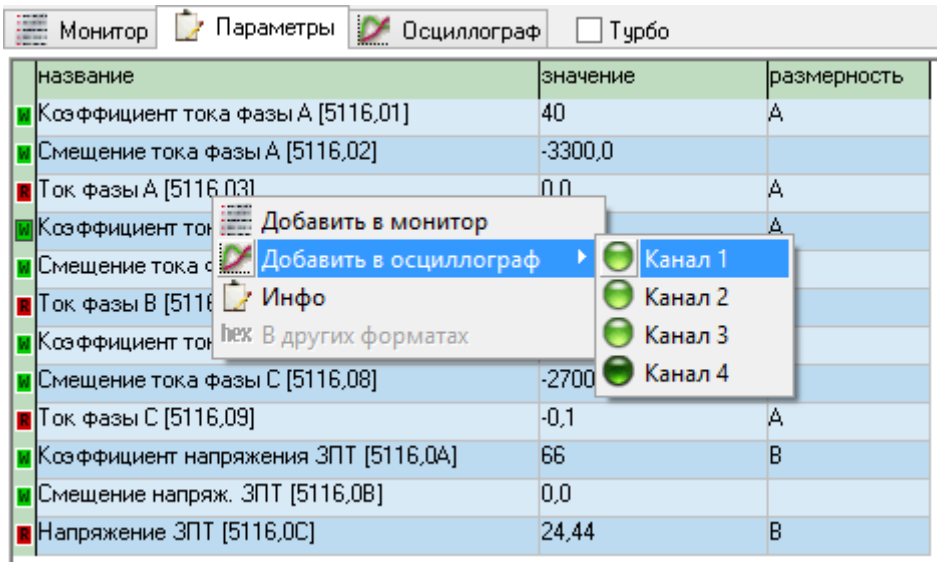


Рис. 4.5 Добавление параметра в осциллограф

Примерные осциллограммы токов фаз (**АЦП.Ток фазы A, B, C**) и заданного фазного напряжения (**ШИМ.UaЗад**) показаны на Рис. 4.6. Для более частого обновления параметров можно воспользоваться кнопкой «турбо» рядом с вкладной осциллографа, а можно перейти на вкладку «выборка» внутри окна осциллографа (подробнее о работе с осциллографом читайте в руководстве к программе UniCON, которое открывается через меню Помощь).

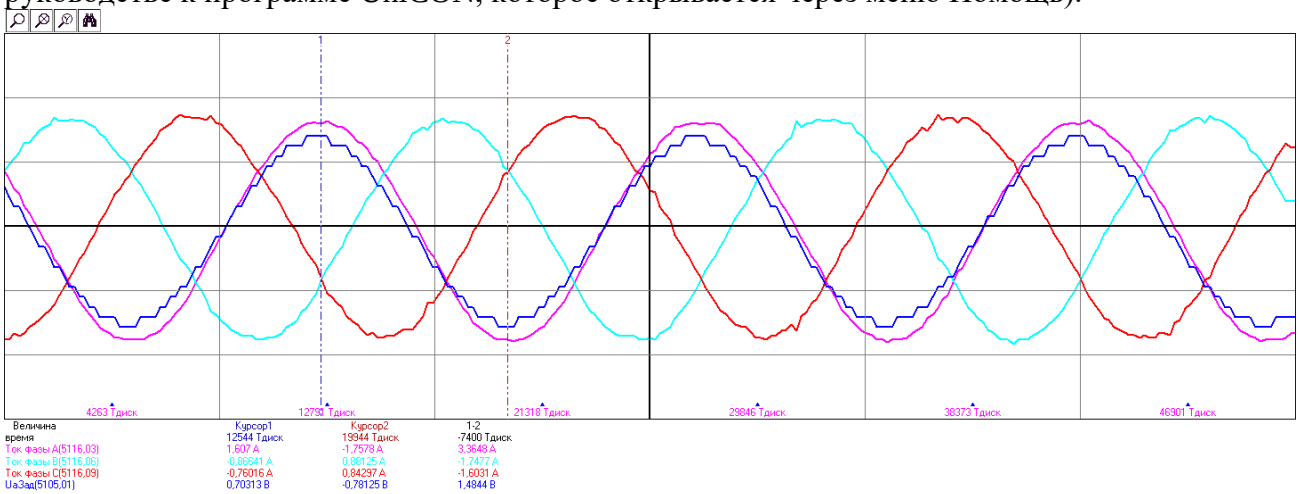


Рис. 4.6 Осциллограмма тестового запуска

Также можно визуально убедиться, что двигатель крутится. Для математической модели можно открыть справочные статусные параметры двигателя и посмотреть частоту вращения в **Модель общие данные.Скорость [5181,06]** или добавить угловое положение ротора **Модель общие данные.Механический угол [5181,08]** в осциллограф.

Если при запуске привода сработала защита, что можно видеть по статусу системы управления «АВАРИЯ», то необходимо определить какая: это написано ниже на панели «Авария». Если сработала защита по превышению фазного тока, то можно попробовать уменьшить заданное напряжение в параметре **Кривая U-f.Первое опорное напряжение [511A,06]** и повторить запуск. Если сработала защита по понижению напряжения ЗПТ, то это говорит о возможно недостаточном по мощности источнике питания (хотя и возможно КЗ). Против этого также может помочь снижение заданного напряжения в кривой U/f, и, как следствие тока и мощности. Если сработала защита повышения напряжения ЗПТ, это странно для запуска на данной частоте вращения и может говорить о помехах в измеряемом сигнале либо недостаточной ёмкости звена постоянного тока инвертора. Если сработала аппаратная авария, то это говорит либо о коротком замыкании на выходе инвертора, либо о неправильной работе драйверов ключей инвертора. Срабатывание других защит на данном этапе маловероятно.

Если система управления в состоянии «РАБОТА», но токи фаз по-прежнему равны нулю, а двигатель не крутится, это может говорить о недостаточном приложенном напряжении.

Попробуйте постепенно поднять напряжение нулевой точки, **Кривая U-f.Нулевое опорное напряжение [511A,04]**. При значении не выше 10% от максимального напряжения токи точно должны начать протекать, а двигатель должен начать крутиться. Если этого не произошло, нужно разбираться в аппаратной части или попробовать пройти шаги сначала, сбросив настройки привода на заводские кнопкой «Загрузить по умолчанию» справа сверху.

Если двигатель вращается, а осциллограммы токов похожи на приведенные, то этап первичной пусконаладки завершен. Далее можно подать команду останова (кнопка Стоп на панели управления в правой части окна UniCON), а можно поднять задание частоты вращения и попытаться раскрутить двигатель быстрее.

Для продолжения практической работы с приводом необходимо перейти к главе 6, а для продолжения знакомства с программным обеспечением к главе 5.

5. Описание модулей ПО

5.1. SM_Ctrl: структуры системы управления

В зависимости от технологических требований управление двигателем может осуществляться по-разному. Система управления принимает сигналы обратных связей (в большинстве случаев), рассчитывает и выдает управляющие воздействия на ключи инвертора, от которого питается двигатель. Набор обратных связей, состав и тип регуляторов зависят от того, какую величину и как требуется регулировать.

В данном ПО реализовано несколько видов структур управления для различных типов двигателей, каждая из которых реализована в виде отдельного состояния дискретного автомата в файле SM_Ctrl.c. Желаемое состояние дискретного автомата, в которое он переходит после запуска привода, задается в переменной (**Настройки СУ.режим работы**), что в ПО соответствует переменной `sm_ctrl.run_mode`. Список доступных режимов работы (они же структуры системы управления, они же состояния дискретного автомата SM_Ctrl) можно посмотреть в файле build.h в виде списка констант, начинающихся с префикса **«CTRL_»**. Текущее состояние дискретного автомата доступно в переменной sm_ctrl.state или в параметре **ДА sm_ctrl.sm_state [510Bh.01h].** При отключенном приводе дискретный автомат находится в состоянии «Останов», определяемом константой CTRL_STOP, где управление инвертором отключено. При приеме команды на запуск (другим дискретным автоматом SM_CmdLogic) дискретный автомат SM_Ctrl принудительно переводится в состояние «Запуск», определяемом константой CTRL_RUN. В данном состоянии запуска в зависимости от желаемой структуры управления (**Настройки СУ.режим работы)** автомат переходит в следующее состояние, соответствующее этой структуре. В общем случае автомат может не сразу перейти в желаемое состояние, а пройти через ряд промежуточных или подготовительных состояний.

Реализованы следующие структуры управления приводами:

Константа в файле build.h	N	Описание
CTRL_RUN_U2F	3	Скалярная структура обеспечивает вращение вектора напряжения с заданной частотой и амплитудой, определяемой в зависимости от частоты по закону U(f). Подходит для синхронного и асинхронного двигателей.
CTRL_RUN_I2F	4	Частотно-токовая структура обеспечивает вращение вектора тока с заданной частотой и поддержание его на заданном уровне амплитуды посредством регуляторов тока. Подходит для синхронного и асинхронного двигателей.
CTRL_FLUXING	2	Структура поддержания постоянного тока удобна для запуска синхронного двигателя, а также для настройки регуляторов тока синхронного и асинхронного двигателей.
CTRL_RUN_VECTOR_SM	5	Структура векторного датчикового управления для синхронного двигателя, имеющая контуры тока и скорости.
CTRL_RUN_VECTOR_IM_ENCODER	56	Структура векторного датчикового управления для асинхронного двигателя, имеющая контуры тока и скорости. Содержит наблюдатель потока ротора в осях d,q, привязанных к ротору.
CTRL_RUN_VECTOR_SM_ENCODER_POS	10	Структура векторного датчикового управления для синхронного двигателя имеющая контуры тока, скорости,

		положения: обеспечивает отработку приводом заданного положения.
CTRL_AUTO_SENSOR_TUNING	29	Структура для автоматической настройки смещения датчиков положения на элементах Холла и энкодера синхронного двигателя, имеющая в основе частотно-токовую структуру управления.
CTRL_DC_DRIVE	35	Структура управления двигателем постоянного тока без контуров: задание фиксированного напряжения на обмотки.
CTRL_DC_DRIVE_FB	36	Структура подчиненного управления двигателем постоянного тока с контуром тока и скорости.
CTRL_RUN_SRM_AUTOCOMMUTATION	101	Структура управления вентильно-индукторным двигателем с самовозбуждением в режиме автокоммутации с токоограничением и контуром скорости.

Более подробно все структуры управления будут описаны ниже.

5.1.1. *Структура скалярного управления* (режим работы = 3)

Режим работы U/f обеспечивает традиционное скалярное управление АД/СД с регулированием напряжения по заранее заданной кривой *U/f*. Настройка кривой (подробнее модуль рассмотрен в п. 5.3) ограничена двумя опорными точками через следующие параметры группы **Кривая U-f**: **Нулевая опорная частота**, **Нулевое опорное напряжение**, **Первая опорная частота**, **Первое опорное напряжение**. Разгон производится от задатчика интенсивности ЗИ (подробнее рассмотрен в п. 5.4). Структура скалярной СУ представлена на Рис. 5.1.

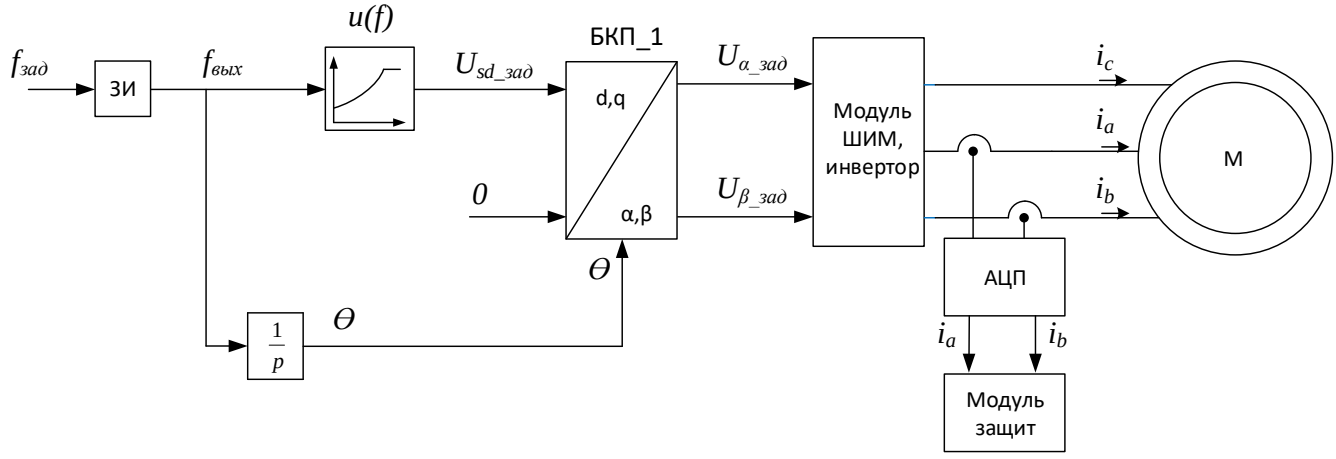


Рис. 5.1 Структура системы скалярного управления

Заданная частота вращения поступает на задатчик интенсивности (ЗИ), который формирует плавное изменение выходной частоты. Выход ЗИ поступает как на блок кривой U/f, который табличным методом по текущей частоте вращения вычисляет необходимую амплитуду вектора напряжения, так и на интегратор угла положений, который формирует заданный угол вектора напряжения. Далее вектор напряжения в полярной системе координат (амплитуда и угол) преобразуются в прямоугольную систему координат в виде двух заданий напряжения $U_{α_зад}$ и $U_{β_зад}$, которые поступают на модуль ШИМ (см. п. 5.7). Модуль ШИМ управляет шестиключевым инвертором напряжения таким образом, чтобы реализовать заданный вектор напряжения. Какая-

либо обратная связь в структуре отсутствует: датчики токов фаз используются только для аварийного отключения при превышении уставки (см. модуль защит п. 5.12).

Запуск данной структуры на практике подробнее рассмотрен в главе 4.

5.1.2. Структура частотно-токового управления (режим работы = 4)

Служебный режим, в котором происходит вращение вектора тока статора (поля двигателя) с фиксированной амплитудой и заданной частотой. Система является разомкнутой по положению ротора, т.е. угловое положение вектора тока не зависит от реального положения ротора, а скорость вращения определяется задатчиком интенсивности. Структура управления по сути формирует из инвертора напряжения инвертор тока программным способом. Исторически частотно-токовой системой управления обычно называют другую, более сложную структуру управления на основе инвертора тока, предназначенную исключительно для асинхронного двигателя и стремящуюся приблизиться по своим показателям к векторной структуре управления. Однако в данном документе под частотно-токовым управлением будет иметься ввиду прямое управление частотой и амплитудой вектора тока двигателя по заданию пользователя без каких-либо других законов регулирования.

Данная структура «инвертора тока» подходит как для синхронной, так и асинхронной машины. Ротор машины при этом должен увлекаться полем статора и вращаться за ним: у синхронной машины должен вращаться синхронно с полем статора, а у асинхронной «проскальзывать», т.е. вращаться чуть медленнее в двигательном режиме. Для синхронной машины при неверной настройке амплитуды тока и темпа задатчика интенсивности возможны колебательные процессы, вплоть до «выпадения» машины из синхронизма. Этот режим используется преимущественно при наладке привода, настройке регуляторов тока, проверке работы привода без обратной связи по положению и т.п.

В эксплуатации такой режим используется редко, так как двигатель с ним работает нестабильно, режим уступает по возможностям режиму векторного управления. В некоторых случаях похожий тип управления применяется для синхронных серводвигателей и называется микрошаговым управлением. Также пуск двигателя в таком режиме (как синхронной, так и асинхронной машины) часто применяется как начальный этап запуска бездатчиковой векторной системы управления, когда положение магнитного потока ротора двигателя по ЭДС не может быть вычислено на низкой скорости.

На Рис. 5.2 приведена структура системы управления в данном режиме.

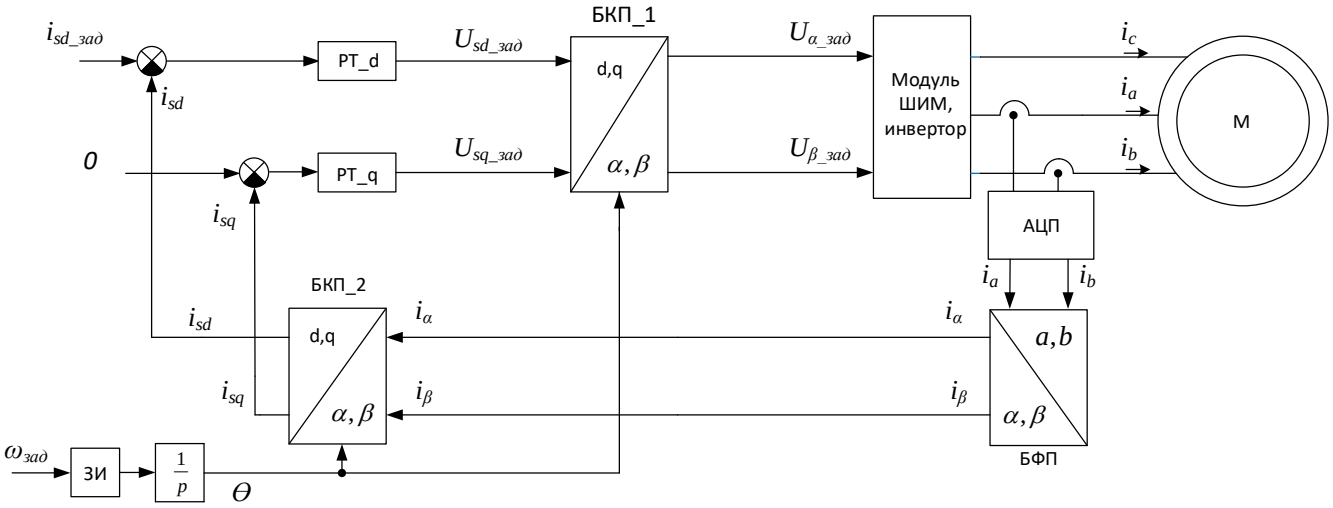


Рис. 5.2 Структура системы частотно-токового управления

Задание частоты вращения вектора тока двигателя **Задания.Скорость** поступает на задатчик интенсивности ЗИ (п.5.4), который обеспечивает формирование линейной тахограммы разгона двигателя. С его выхода сигнал скорости поступает на вход интегрирующего элемента, на выходе которого формируется задание углового положения вектора тока θ.

Измеренные значения токов двух фаз двигателя i^*_a, i^*_b поступают на вход блока фазных преобразований БФП (п.5.5). Его задача – преобразовать токи из трехфазной системы координат в двухфазную декартову систему координат *α, β*. Ток фазы *c* вычисляется по первому закону Кирхгофа (обмотки трехфазного двигателя соединены в звезду).

Токи фаз в координатах *α, β* (i^*_α, i^*_β) поступают на блок координатных преобразований БКП_2 (п.5.6), обеспечивающий преобразование токов из неподвижной системы координат к вращающейся системе координат *d,q*. Положение оси *d* будет определяться текущим значением угла *θ*. Токи i^*_q, i^*_d поступают на ПИ-регуляторы токов, выходы которых после обратного координатного преобразования в блоке БКП_1 (п.5.6, поворот в обратную сторону на угол *θ*) заводятся на инвертор напряжения (п. 5.7).

Задание на амплитуду тока (**Задания.Ток удержания**) подается на один из регуляторов, в данном случае на регулятор тока (п. 5.2) по оси *d*. Таким образом, регуляторы поддерживают постоянный ток согласно заданию $i^*_{sd_зад}$, но так как система координат *d, q*, в которой работают регуляторы, вращается, то вращается и результирующий вектор тока в осях *a, b, c*, связанных с фазами двигателя. Работа регуляторов тока во вращающихся осях позволяет исключить статическую ошибку воспроизведения амплитуды трехфазного синусоидального тока, которая присутствовала бы, если бы регуляторы токов работали в осях *α, β* и на их вход подавалось бы синусоидальное задание токов.

Необходимо заметить, что ортогональные оси d,q в данном случае никак не зависят от фактического положения ротора, как это обеспечивается при векторном управлении, а лишь поворачиваются с заданной скоростью независимо от состояния машины и положения её ротора.

Результатом работы такой системы должен стать синусоидальный ток в фазах двигателя, практически не зависящий от различных нелинейностей, влияния "мертвого" времени инвертора, колебаний механической части и напряжения в звене постоянного тока.

Запуск данной структуры на практике подробнее рассмотрен в п.п. 6.1.3.

5.1.3. *Режим удержания* (режим работы = 2)

Этот режим соответствует режиму фиксации двигателя под током и желателен перед переходом в режим частотно-токового управления (режим работы 4, вращение вектора тока с фиксированной амплитудой и нарастающей частотой). Дли синхронной машины он необходим для того, чтобы ротор двигателя предварительно повернулся и зафиксировался в определенном механическом состоянии: дождаться затухания механических колебаний. При этом последующее начало вращения вектора тока в режиме частотно-токового управления выполняется с заданной фазы и сопровождается меньшими колебаниями ротора. Для асинхронной машины в этом состоянии можно дождаться нарастания потока возбуждения перед началом вращения.

Также данная структура удобна для настройки регуляторов тока путем наблюдения осциллограммы реакции на скачок задания.

Структурная схема системы управления в данном режиме аналогична схеме частотно-токового управления (рассмотренной в 5.1.2) за одним исключением: угол положения вектора тока Θ неизменен и задается параметром **Задания.theta_elec [5112h.04h].**

Запуск данной структуры на практике подробнее рассмотрен в п.п. 6.1.2.

5.1.4. Структура векторного датчикового управления синхронным двигателем (режим работы = 5)

Основной недостаток скалярного управления заключается в малом быстродействии, как при регулировании скорости, так и при регулировании момента. Это объясняется тем, что для перехода в некоторый установившийся режим требуется завершение процессов распределения моментообразующего тока и потокообразующего тока, которое в системах скалярного управления происходит само по себе через некоторое время. Именно это время переходных процессов и определяет ту плохую динамику, которая присуща электроприводу со скалярным управлением. Однако если не пускать переходные процессы в двигателе на «самотек», можно избежать

перераспределения токов и связанных с этим переходных процессов и контролировать состояние электрической машины в каждый момент времени, увеличив быстродействие системы регулирования. Так как требуется регулирование двух ортогональных токов, один из которых является потокообразующим, а другой моментообразующим, то система управления была названа векторной.

Под системами векторного управления принято подразумевать системы, работающие при постоянстве потокосцепления ротора, хотя система управления может быть построена и для постоянства потокосцепления статора или общего потока машины.

Таким образом, векторное управление позволяет управлять непосредственно моментом двигателя, наиболее рационально использовать возможности привода, контролировать все переходные процессы и не «пускать их на самотек». Для синхронных приводов с цифровой системой управления такой режим работы является наиболее применяемым на практике.

Если записать уравнение момента синхронного двигателя с постоянными магнитами (СДПМ) в осях d, q, получим следующее выражение:

$$M = \psi_{sd} i_{sq} - \psi_{sq} i_{sd} = \Psi_{PM} i_{sq} + L_s i_{sd} i_{sq} - L_s i_{sq} i_{sd} \quad (1)$$

Приравняв $i_{sd} = 0$, получим:

$$M = \Psi_{PM} i_{sq} \quad (2)$$

где Ψ_{PM} — составляющая потокосцепления статора от постоянного магнита на роторе,

M – момент двигателя,

i_{sq} – ток статора по оси q, перпендикулярный направлению потока постоянных магнитов,

i_{sd} – ток статора по оси d, сонаправленный потоку постоянных магнитов,

L_s – индуктивность статора.

Получается, наилучшие энергетические показатели будут достигаться при таком управлении, когда ток $i_{sd} = 0$, тогда в двигателе будет лишь моментобразующий ток i_{sq} и омические потери будут минимальными. В этом режиме работы СДПМ подобен двигателю постоянного тока, так как амплитуда тока по оси q напрямую задаёт момент двигателя (аналог якорной обмотки коллекторного двигателя постоянного тока).

Для реализации векторного управления необходимо определять угловое положение вектора потокосцепления ротора двигателя, которое связано с положением ротора и определяется по датчику положения ротора (ДПР). Ортогональные подвижные оси d,q направляются так, что ось d совпадает с направлением вектора потокосцепления ротора.

На Рис. 5.3 показана структура системы датчикового векторного управления синхронным двигателем с постоянными магнитами (СДПМ). Задание частоты **Задания.Скорость** ($\omega^*_{зад}$ на Рис. 5.3) поступает на задатчик интенсивности ЗИ (п. 5.4), который обеспечивает линейный разгон. С его выхода задание частоты поступает на сумматор ПИД регулятора скорости (п. 5.2), в который также заводится обратная связь по скорости с датчика положения ротора (п.5.9 или п. 5.10). Выходом регулятора скорости является задание на ток по оси q. Это значение пропорционально электромагнитному моменту машины. Оно поступает на сумматор ПИ-регулятора тока по оси q, выходом которого является задание напряжения по оси q. Аналогично функционирует регулятор тока по ортогональной оси d, задание на входе которого нулевое.

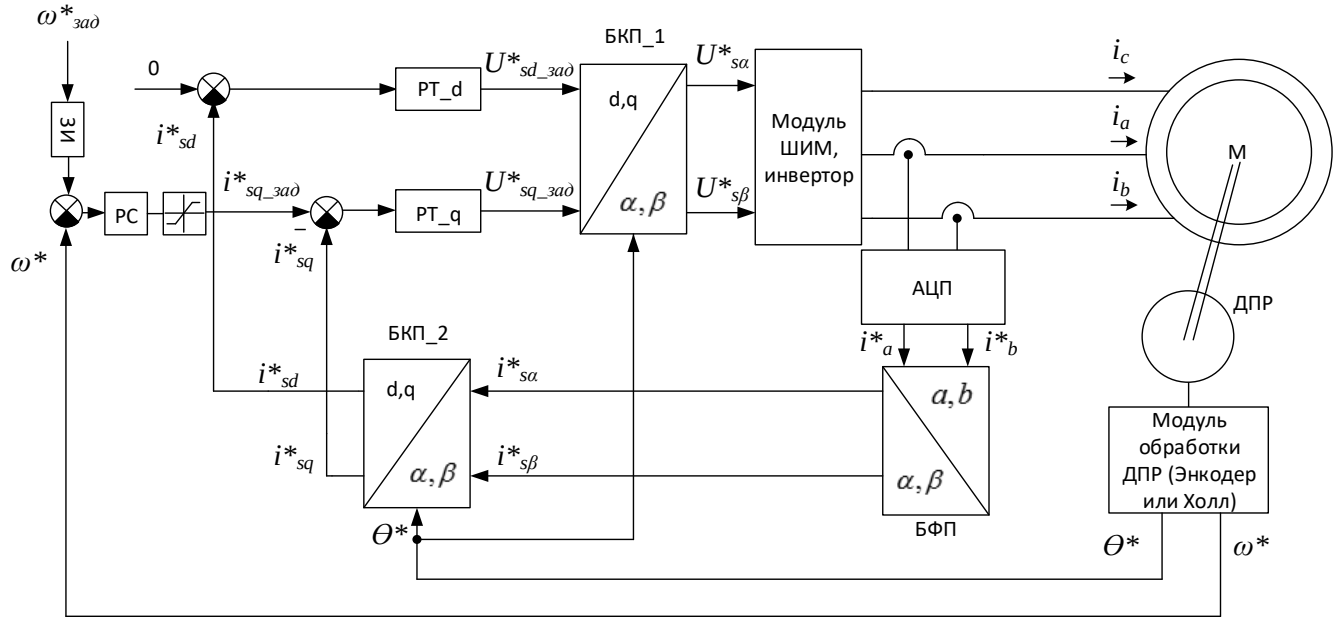


Рис. 5.3 Структура системы векторного управления

Напряжения с регуляторов токов поступают на блок обратного координатного преобразования (п. 5.6, поворот системы координат в обратную сторону на угол θ) БКП_1, а затем заводятся на инвертор напряжения (модуль ШИМ, п. 5.7). Измеренные значения токов фаз двигателя i^*_a, i^*_b поступают на вход блока фазных преобразований БФП (п. 5.5). В этом блоке они преобразуются из трехфазной системы координат в двухфазную декартовую систему координат α, β. Ток фазы c вычисляется по первому закону Кирхгофа (обмотки двигателя соединены в звезду). Токи i^*_α, i^*_β поступают на блок координатных преобразований БКП_2 (п. 5.6), поворачивающих систему координат на угол θ. На выходе получаются фактические значения токов в системе координат d,q, ориентированной по потокосцеплению ротора. Эти значения используются в качестве сигналов обратных связей на входах регуляторов токов по осям d,q.

Данная структура может использовать разные типы датчиков положения. Выбор типа датчика положения осуществляется в переменной **Базов. парам. Тип датчика положения [5113h.09h]**.

Также следует отметить, что выход регулятора скорости (задание на ток/момент) имеет ограничения: **Рег. скорости.Максимум выхода [5101h.04h]** и **Рег. скорости.Минимум выхода [5101h.05h]**. Установка минимума и максимума происходит автоматически в зависимости от задания тока для векторного управления в параметре **Задания.Макс. ток ВУ**. При этом по умолчанию минимум установлен нулем, чтобы исключить отрицательный момент двигателя, а максимум устанавливается равным значению этого параметра. Существует возможность включения режима рекуперации установкой флага **Настройки СУ.Рекуперация**. В этом случае минимум регулятора скорости получит значение параметра **Задания.Макс. ток ВУ** со знаком минус. Разрешение рекуперации может в некоторых режимах привести к повышению напряжения в звене постоянного тока, при отсутствии рассеивания энергии торможения в тормозном резисторе сработает защита по превышению напряжения звена постоянного тока (которая должна быть верно настроена – иначе возможен выход оборудования из строя!).

Запуск данной структуры на практике подробнее рассмотрен в п.п. 6.1.5**Ошибка! Источник ссылки не найден.**.

Примечание 1. Если читатель плохо знаком с принципами векторного управления электродвигателями, то рекомендуется к прочтению вводная статья «Векторное управление электродвигателем «на пальцах», доступная по ссылке https://geektimes.ru/company/npf_vektor/blog/269486/ . Также рекомендуется книга «Системы управления электроприводов. Учебник для вузов» А.С. Анучин, 2015 год.

Примечание 2. Если необходимо управления синхронной машиной с обмоткой возбуждения, то структура управления принципиально ничем не отличается от уже приведенной, за исключением нового контура возбуждения. В простейшем случае можно подать на обмотку

возбуждения просто постоянное номинальное напряжение возбуждения, после чего электрическая машина будет вести себя с точки зрения управления также, как и машина с постоянными магнитами. В текущем ПО предусмотрен такой тип управления возбуждением: для этого необходимо разрешить управление ОВ установкой единицы в параметр **Настройки СУ. Возбудитель [5114h.06h],** а также задать желаемое задание напряжения возбуждения в параметре **Задания.Заданное напряжение ОВ [5112h.05h]**. Более сложные структуры совместного управления током статора и током ОВ выходят за рамки данного документа.

5.1.5. Структура векторного датчикового управления асинхронным двигателем (режим работы = 55)

Такой тип управления наиболее распространен в современных асинхронных электроприводах с цифровым управлением (хотя простейшее скалярное управление для асинхронных приводов также занимает высокие позиции популярности по причине простоты настройки).

Векторное управление асинхронным двигателем схоже с векторным управлением синхронным двигателем, поэтому рекомендуется предварительно ознакомиться с описанием векторной системы управления синхронным двигателем (п. 5.1.4.).

Пусть ось x в декартовой системе координат *x, y* сонаправлена с потокосцеплением ротора, тогда для асинхронного двигателя можно записать следующие уравнения равновесия

$$\left.\begin{aligned} u_{sx} &= i_{sx}R_s + \frac{\mathrm{d}\psi_{sx}}{\mathrm{d}t} - \omega_к\psi_{sy};\\ u_{sy} &= i_{sy}R_s + \frac{\mathrm{d}\psi_{sy}}{\mathrm{d}t} + \omega_к\psi_{sx};\\ 0 &= i_{rx}R_r + \frac{\mathrm{d}\psi_{rx}}{\mathrm{d}t} - (\omega_к - \omega)\psi_{ry};\\ 0 &= i_{ry}R_r + \frac{\mathrm{d}\psi_{ry}}{\mathrm{d}t} + (\omega_к - \omega)\psi_{rx}, \end{aligned}\right\} \quad (3)$$

и потоков

$$\left.\begin{aligned} \psi_{sx} &= L_s i_{sx} + L_m i_{rx};\\ \psi_{sy} &= L_s i_{sy} + L_m i_{ry};\\ \psi_{rx} &= L_m i_{sx} + L_r i_{rx};\\ \psi_{ry} &= L_m i_{sy} + L_r i_{ry}. \end{aligned}\right\} \quad (4)$$

где:
u_{sx}, u_{sy} – напряжение статора по осям *x*, у,
i_{sx}, i_{sy} – ток статора по осям *x*, у,
i_{rx}, i_{ry} – ток ротора по осям *x*, у,
Ψ_{rx}, Ψ_{ry} – потокосцепление ротора по осям *x*, у,
Ψ_{sx}, Ψ_{sy} – потокосцепление статора по осям *x*, у,
R_r, R_s – сопротивление обмоток ротора (приведенное) и статора,
L_r, L_s – индуктивности обмоток ротора (приведенная) и статора,
L_m – взаимная индуктивность,
ω – частота вращения ротора,
$\omega_к$ – частота вращения осей x,y (частота вращения потокосцепления ротора).

Если принять, что система управления будет поддерживать потокосцепление ротора постоянным (или изменять его медленно), а ориентация осей x, y будет всегда соответствовать положению вектора потокосцепления ротора, то $\frac{\mathrm{d}\psi_{rx}}{\mathrm{d}t}=0$, $\frac{\mathrm{d}\psi_{ry}}{\mathrm{d}t}=0$ и $\psi_{ry}=0$, и уравнения (3) для роторной цепи можно упростить:

$$\left.\begin{aligned} 0&=i_{rx}R_r;\\ 0&=i_{ry}R_r+\left(\omega_{\text{к}}-\omega\right)\psi_{rx}. \end{aligned}\right\} \tag{5}$$

Из первого уравнения системы следует, что $i_{rx}=0$, тогда

$$\left.\begin{aligned} \psi_{rx}&=L_m i_{sx};\\ L_m i_{sy}&=-L_r i_{ry}. \end{aligned}\right\} \tag{6}$$

Теперь запишем уравнение момента асинхронного двигателя через статорные токи:

$$M=-\psi_{rx}i_{ry}=-\psi_{rx}\left(-\frac{L_m}{L_r}i_{sy}\right)=\frac{L_m}{L_r}\psi_{rx}i_{sy}=\frac{L_m}{L_r}L_m i_{sx}i_{sy}=\frac{L_m^2}{L_r}i_{sx}i_{sy}. \tag{7}$$

Полученный результат показывает, что для регулирования потокосцепления ротора надо регулировать ток статора по оси x, а для регулирования момента — ток статора по оси y. Структура системы векторного управления АД показана на Рис. 5.4.

Управление производится в осях x, y, раздельно регулируется ток статора по оси x — потокообразующий ток — с помощью регулятора (п. 5.2) тока x (РТx), и ток статора по оси y — моментообразующий ток — с помощью регулятора (п. 5.2) тока y (РТy). Полученные регуляторами составляющие вектора напряжения в осях x, y с помощью блока координатных преобразований (п. 5.6) переводятся в оси α,β и реализуются инвертором (п. 5.7). В фазах двигателя производится измерение токов и их преобразование в оси α,β с помощью блока фазных преобразований (п. 5.5).

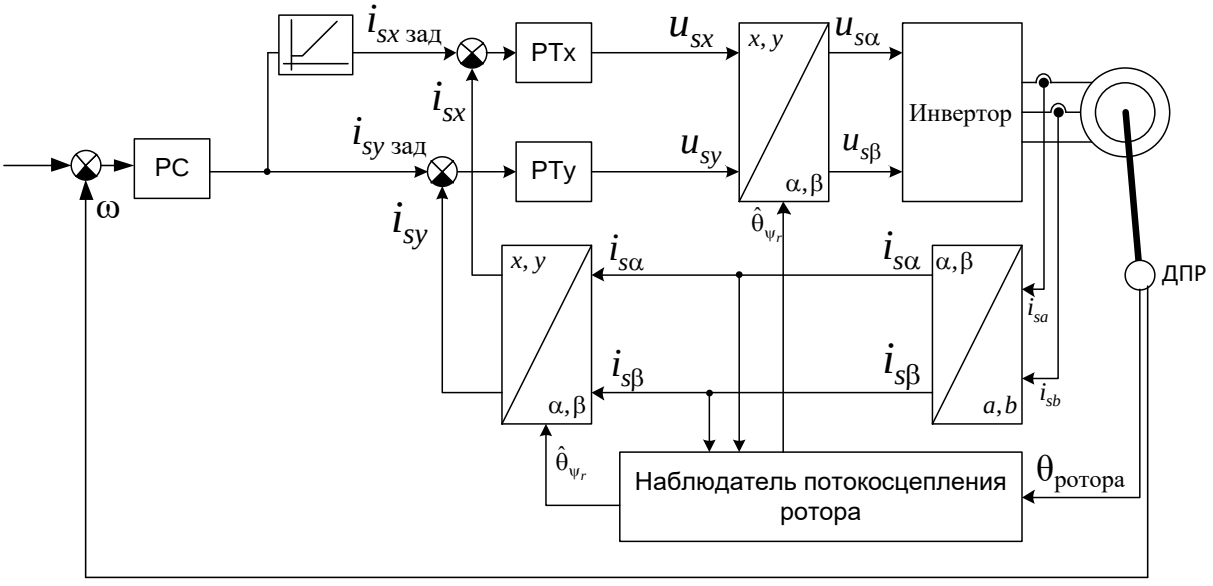


Рис. 5.4 Структура системы векторного управления АД

Для унификации реализация в программном коде использует имена регуляторов тока `pid_id` и `pid_iq` по аналогии с синхронной машиной (так как имена регуляторов одинаковы для всех двигателей), однако в данном случае необходимо помнить, что это регуляторы осей *x* и *y* соответственно.

Задание на ток возбуждения i_x формируется пропорциональным моментообразующему току i_y, с коэффициентом пропорциональности **Наблюдатель ротора АД.Соотношение тока возб. X к току Y [5B04,02]** и ограничением минимума на уровне **Наблюдатель ротора АД.Минимальный ток возбуждения [5B04,03]**. Это обеспечивает приближение к стратегии управления МТРА (Maximum Torque Per Ampere) одновременно с автоматическим ослаблением поля. Здесь необходимо отметить, что в случае регулирования потока допущение $\frac{\mathrm{d}\psi_{rx}}{\mathrm{d}t}=0$, принятое выше, в общем случае перестает быть верным и данный член необходимо учитывать, но так как индуктивность L_m двигателя большая и поток изменяется фактически медленно, для упрощения в первом приближении данным членом можно пренебречь.

Важной частью структуры является блок оценки потокосцепления ротора, который служит для вычисления амплитуды и углового положения вектора потокосцепления ротора – относительно него ориентируется вектор тока статора. Наблюдатель потокосцепления ротора реализован в файле v_rotor_observer.c в виде инерционного звена в осях, привязанных к ротору (d,q). Наблюдатель использует положение вала ротора с датчика положения (инкрементального энкодера).

Для получения наблюдателя возьмем уравнения равновесия роторной цепи АД и уравнения потокосцеплений в осях d,q:

$$\left.\begin{aligned} 0 &= i_{rd}R_r + \frac{\mathrm{d}\psi_{rd}}{\mathrm{d}t};\\ 0 &= i_{rq}R_r + \frac{\mathrm{d}\psi_{rq}}{\mathrm{d}t};\\ \psi_{rd} &= L_m i_{sd} + L_r i_{rd};\\ \psi_{rq} &= L_m i_{sq} + L_r i_{rq}. \end{aligned}\right\} \qquad (8)$$

Выражаем роторные токи через статорные и потокосцепления ротора:

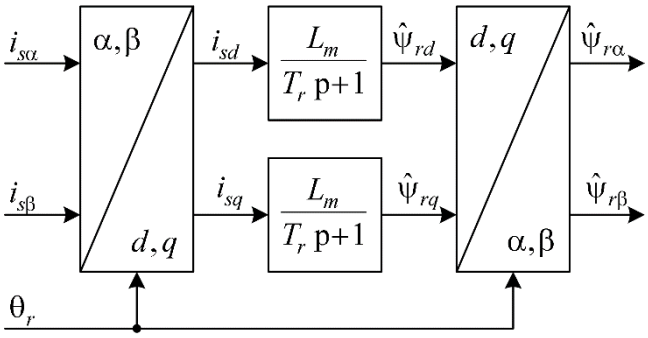
$$\left.\begin{aligned} i_{rd} &= \frac{\psi_{rd} - L_m i_{sd}}{L_r};\\ i_{rq} &= \frac{\psi_{rq} - L_m i_{sq}}{L_r}, \end{aligned}\right\} \qquad (9)$$

и, подставив их в уравнения равновесия, получим формулы:

$$\left.\begin{aligned} L_m i_{sd} &= \psi_{rd} + T_r \frac{\mathrm{d}\psi_{rd}}{\mathrm{d}t};\\ L_m i_{sq} &= \psi_{rq} + T_r \frac{\mathrm{d}\psi_{rq}}{\mathrm{d}t}, \end{aligned}\right\} \qquad (10)$$

где:
i_{rd}, i_{rq} – ток ротора по осям d, q,
Ψ_{rd}, Ψ_{rq} – потокосцепление ротора по осям d, q,
R_r – сопротивление обмоток ротора (приведенное),
L_r – индуктивность обмоток ротора (приведенная),
L_m – взаимная индуктивность,
T_r – постоянная времени ротора.

По данным уравнениям можно изобразить структурную схему наблюдателя. Кроме полученной структуры в наблюдатель необходимо включить координатное преобразование из токов статора в осях α,β в токи осей d,q.



Структура наблюдателя потокосцепления ротора в роторных координатных осях

Стоит отметить, что для работы векторной структуры управления без регулятора потока наличие L_m в числителе инерционного звена наблюдателя не требуется, так как по выходным потокам берется функция арктангенса для вычисления углового положения вектора потока, а абсолютное значение потока не важно. Поэтому для упрощения настройки параметров L_m в числителе можно принять равное единице (исключить).

Отдельно стоит отметить работу регулятора скорости (п. 5.2). Регулятор скорости должен управлять моментом двигателя, либо величиной, ему пропорциональной, что для синхронного двигателя выполнялось регулированием тока по оси q. В асинхронном двигателе согласно уравнению (7) момент пропорционален произведению токов статора *x* и *y*. Тогда, если в структуре управления организовано пропорциональное одновременное изменение токов статора i_{sx}, i_{sy}, то регулятор скорости, выдавая задание на ток i_y, по сути управляет величиной, пропорциональной квадрату момента, что некорректно. Однако приведенные выше формулы справедливы для линейной зоны работы электрической машины, без учета насыщения магнитопровода. В реальной же асинхронном двигателе магнитный поток растет не строго пропорционально току, а по закону экспоненты. Учитывая эффект насыщения, реальный момент двигателя растет в данной структуре не квадратично заданию i_y (и как следствие i_x) а по более сложной кривой – сначала, при малых токах, квадратично, а с увеличением токов рост момента становится ближе к линейному. Вследствие этого в первом приближении допустимо регулятором скорости регулировать непосредственно задание обоих токов сразу, так как при больших токах эта зависимость стремится к линейной, однако более правильно между выходом регулятора скорости и заданием на токи установить блок нелинейности, который можно настроить по точкам для конкретной электрической машины опытным путем с использованием датчика момента.

Для работы данной структуры необходима настройка инкрементального энкодера: проверка, что частота вращения и знак определяются верно (проверку проще всего осуществить в скалярном режиме управления). Также необходимо настроить постоянную времени ротора для наблюдателя потока ротора (параметр **Наблюдатель ротора АД.Пост. времени ротора [5В04,01])**.

Запуск данной структуры на практике подробнее рассмотрен в п.п. **Ошибка! Источник ссылки не найден.**.

5.1.6. Структура векторного датчикового управления для синхронного двигателя с дополнительным внешним контуром положения (режим работы = 10)

Структура позиционирования (Рис. 5.5) при помощи дополнительного внешнего контура положения на основе структуры векторного управления, описанной выше в п.п. 5.1.4, реализует поддержание приводом заданного пользователем положения ротора $\theta^*_{зад}$. Регулятор положения (РП) имеет тип ПИД (п. 5.2), хотя дифференциальная часть может быть опущена. Из практических соображений в качестве регулятора скорости (РС) рекомендуется П-регулятор (Ки и Кд необходимо обнулить). Допустима и другая конфигурация: ПИ-регулятор скорости и П регулятор положения (в этом случае обнуляются Ки и Кд регулятора положения).

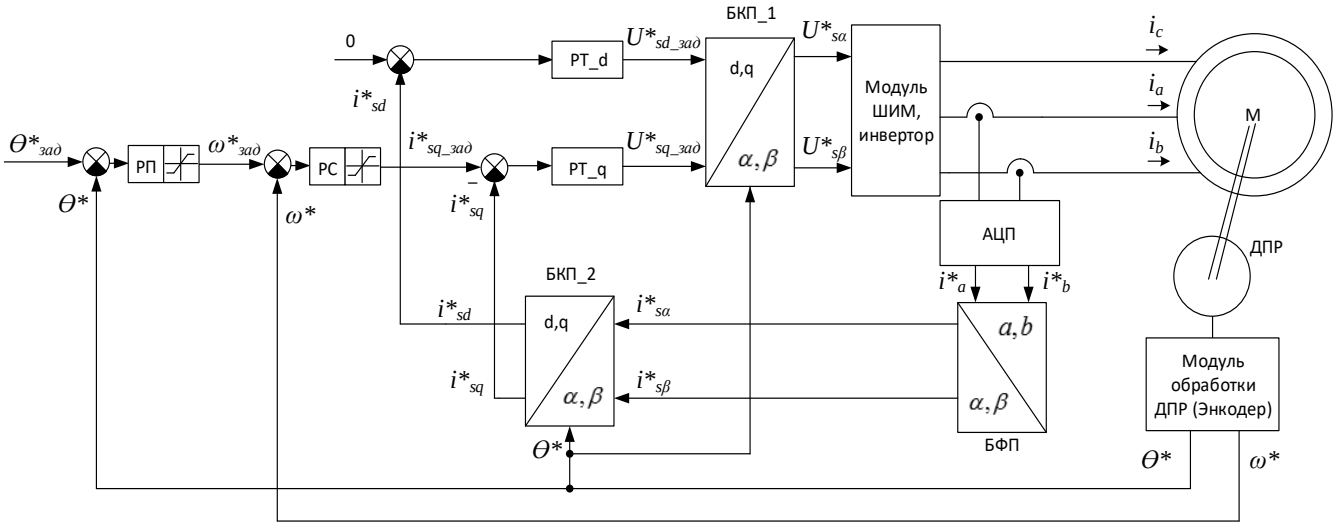


Рис. 5.5 Структура для поддержания заданного положения на основе векторного управления: трехконтурная система подчиненного регулирования

Структура работает при использовании ДПР типа «инкрементальный энкодер» (п. 5.10), так как точности ДПР на элементах Холла недостаточно для данного режима. Пользователь задает желаемое положение вала ротора в электрических градусах в параметре **Задания.theta_elec**. Допустимо задавать положение не превышающее 50 оборотов: в обратной связи регулятора положения используется переменная углового положения, отслеживающая число полных оборотов. Параметр **Задания.Скорость** в данном режиме ограничивает максимальную частоту вращения (выход регулятора положения). Если регулятор скорости пропорциональный, эту переменную необходимо задавать из соображений, чтобы мог быть достигнут максимально разрешенный ток на статоре при максимальном задании скорости (на практике обычно выше, чем желаемое ограничение скорости для сервопривода). Параметр **Задания.Макс. ток ВУ** в данном режиме ограничивает максимальный моментообразующий ток статора (выход регулятора скорости). Рекуперация по умолчанию разрешена, так как без нее невозможна корректная работа сервопривода. Однако при типичных частотах работы серводвигателей реальное повышение напряжения на инверторе обычно маловероятно. В качестве обратной связи по контуру положения используется электрическое угловое положение, учитывающее совершенное валом двигателя число оборотов.

Подробнее пусконаладка данного режима работы рассмотрена в 6.1.6.

5.1.7. Режим автонастройки смещений ДПР (режим работы = 29)

Режим работы по структуре полностью аналогичен структуре частотно-токового управление, 5.1.2, однако в данный режим добавлена логика автоматической подстройки смещений угловых положений датчиков положения ротора (параметры **ДПР-Холл.СмещУгол** и **ДПР-Энкодер. СмещУгол**). Смещения подстраиваются таким образом, чтобы максимуму тока фазы А соответствовало нулевое показание углового положения обоих датчиков положения. Этот режим корректно настраивает датчики положения ротора при условии отсутствия нагрузки на валу, относительно невысокой частоте вращения и правильному чередованию фаз (при задании положительной частоты вращения датчики положения показывают также положительную частоту вращения). Этот режим необходим перед первым пуском векторной структуры управления и снимает необходимость ручной настройки датчиков. Для корректной настройки двигатель должен сделать в таком режиме несколько полных оборотов.

Подробнее о запуске данного режима работы написано в п.п. 6.1.46.1.6.

5.1.8. Структура управления двигателем постоянного тока заданием напряжения на обмотках (режим работы = 35)

В данном состоянии дискретного автомата по сути отсутствует какая-либо структура: заданные напряжения якоря (U) и возбуждения ($U_в$) двигателя постоянного тока (ДПТ) напрямую прикладываются к двигателю, как это показано на рисунке.

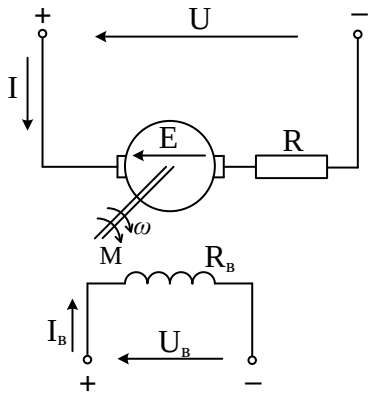


Рис. 5.6 Управление двигателем постоянного тока путем прямого задания напряжения на обмотках

Это самый простой способ управления двигателем. Для задания напряжения якоря используется параметр **ДПТ Сист. Упр.Заданное напряжение якоря [5185h.02h]**, а для напряжения обмотки возбуждения **ДПТ Сист. Упр.Заданное напряжение ОВ [5185h.03h]**. Также в данной структуре для снижения пускового тока можно ввести в цепь якоря резистор, значением которого можно управлять через параметр **Модель (ДПТ).Добавочное сопротивление [5184h.0Eh].**

Подробнее пусконаладка данного режима работы рассмотрена в **Ошибка! Источник ссылки не найден.**.

5.1.9. Структура подчиненного управления двигателем постоянного тока с контуром тока и скорости (режим работы = 36)

Невозможность качественно влиять на динамические и статические показатели привода в разомкнутой системе, а также стремление улучшить основные показатели регулирования координат электропривода (время переходного процесса, перерегулирование, колебательность) приводит к необходимости применения замкнутых систем управления с обратными связями по регулируемым координатам.

Реализованная замкнутая система управления построена на принципе последовательной коррекции контуров тока и скорости.

Сам метод последовательной коррекции основан на том, что передаточные функции двух последовательных звеньев перемножаются и, если перед объектом регулирования поставить звено со специально подобранной передаточной функцией, то можно получить некий результат их перемножения, который будет обладать необходимыми нам статическими и динамическими характеристиками.

Двухконтурная система подчиненного регулирования с внутренним контуром тока и внешним контуром скорости показана на Рис. 5.7. Термин «подчиненное регулирование» показывает, что внутренний контур подчинен внешнему.

Следует отметить, что контуров в общем случае может быть больше двух. Так, к имеющейся системе регулирования можно добавить контур положения, и система станет трехконтурной. Для систем подчиненного регулирования можно сформулировать основные свойственные им особенности:

- быстродействие каждого следующего внешнего контура в два раза медленнее предыдущего внутреннего;

- контура следует настраивать по очереди: сначала внутренний, потом последовательно все внешние;
- внешний контур воздействует на внутренний и выход его регулятора должен быть ограничен на приемлемом для внутреннего контура уровне.

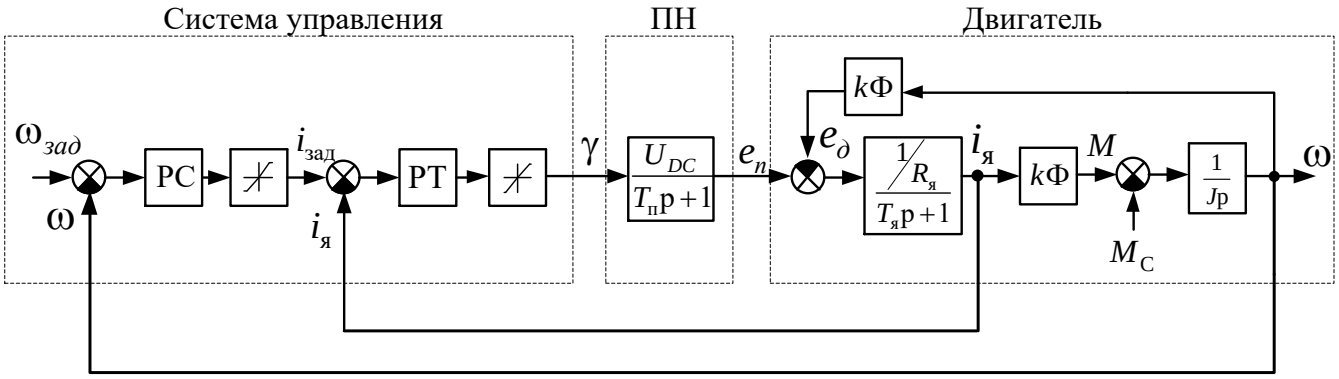


Рис. 5.7. Структура системы подчиненного регулирования ДПТ. РС – регулятор скорости, ТР – регулятор тока

Структурную схемы системы подчиненного регулирования (рис. 5.7) условно можно разбить на три части:

1. Структура, описывающая двигатель,
2. Структура, описывающая преобразователь напряжения (ПН),
3. Структура, описывающая систему управления.

На вход системы управления подается задание на частоту вращения. Разница между заданием ($\omega_{зад}$) и реальной частотой вращения двигателя (ω) поступает на вход регулятора скорости (РС). Регулятор скорости в зависимости от своей настройки и величины входного сигнала, формирует выходной сигнал, который является заданием для контура тока. Выходной сигнал регулятора скорости предварительно должен быть ограничен, чтобы задание на величину тока не превышало установленной пользователем уставки, исходя из номинальных параметров двигателя и преобразователя. Разница между заданной регулятором скорости величиной тока ($i_{зад}$) и реальным током якоря двигателя ($i_я$) поступает на вход регулятора тока (РТ). Регулятор тока аналогичным образом, в зависимости от своей настройки и величины входного сигнала, формирует выходной сигнал, который является заданием на напряжение преобразователя (или на скважность γ, пропорциональную этому напряжению γ). Выходной сигнал регулятора тока предварительно должен быть ограничен, чтобы задание скважности управления силовыми ключами находилась в границах 0-100%. В зависимости от величины задания скважности, преобразователь формирует выходное напряжение (ЭДС преобразователя - e_n), которое является напряжением питания обмотки якоря двигателя.

Если необходимо управлять синхронной машиной с обмоткой возбуждения, то структура управления принципиально ничем не отличается от уже приведенной, за исключением нового контура возбуждения. В простейшем случае можно подать на обмотку возбуждения просто постоянное номинальное напряжение возбуждения, после чего электрическая машина будет вести себя с точки зрения управления также, как и машина с постоянными магнитами. Более сложные структуры совместного управления током статора и током ОВ выходят за рамки данного документа.

В программе UniCON параметры управления данной структурой собраны в группе **ДПТ Сист. Упр.** Задать желаемую частоту вращения двигателя и напряжение ОВ можно в параметрах **Задание скорости (Подч. Рег.) [5185.01]** и **Заданное напряжение ОВ [5185.03]**.

В группе параметров **Рег. тока якоря (ДПТ)** необходимо установить требуемые настройки регулятора тока якоря, а в группе **Рег. скорости** – требуемые настройки регулятора скорости.

Подробнее пусконаладка данного режима работы рассмотрена в **Ошибка! Источник ссылки не найден.**.

5.1.10. Структура управления вентильно-индукторным двигателем с самовозбуждением в режиме автокоммутации с токоограничением и контуром скорости. (режим работы = 101)

Для вентильно-индукторного двигателя с самовозбуждением доступна всего одна структура управления, реализующая так называемую автокоммутацию: переключение работающих обмоток двигателя в зависимости от положения ротора двигателя. Структура системы управления показана на Рис. 5.8.

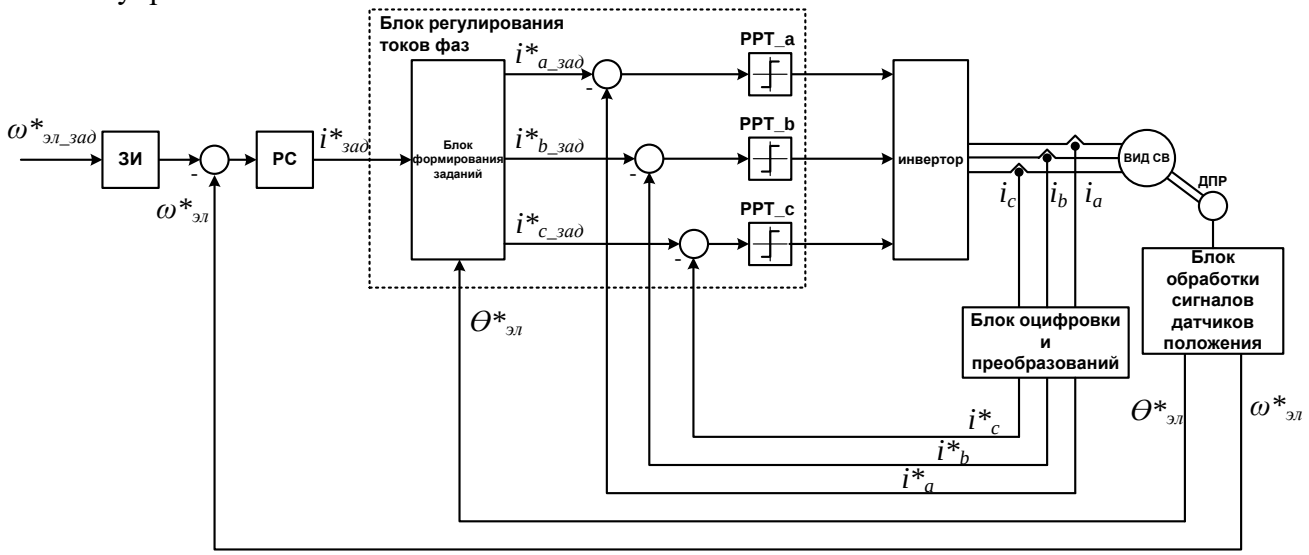


Рис. 5.8. Структура управления вентильно-индукторным двигателем с самовозбуждением в режиме автокоммутации с токоограничением и контуром скорости

Задание скорости $\omega^*_{эл_зад}$ через задатчик интенсивности ЗИ поступает на ПИ-регулятор скорости РС. В качестве обратной связи в РС используется скорость $\omega^*_{эл}$, рассчитанная по показаниям датчика положения ротора ДПР. Выход РС служит заданием тока $i^*_{зад}$ для фаз двигателя. Блок формирования заданий распределяет задание тока по фазам в зависимости от углового положения $\Theta^*_{эл}$, полученного с датчика положения. Здесь используется нейтральная коммутация: задание тока $i^*_{зад}$ прикладывается к фазе двигателя на интервале углового положения, на котором данная фаза формирует положительный момент; на остальной части оборота к фазе прикладывается нулевое задание тока. Задание тока для каждой фазы $i^*_{a_зад}$, $i^*_{b_зад}$, $i^*_{c_зад}$ поступает на соответствующий релейный регулятор тока РРТ. В качестве обратных связей для РРТ служат сигналы измеренных токов с соответствующих датчиков тока i^*_a, i^*_b, i^*_c. Выходы релейных регуляторов тока служат заданием напряжения (или напрямую скважности ШИМ) для соответствующих фаз инвертора.

Релейный регулятор тока реализован простейшим образом: каждый вызов данного состояния дискретного автомата регулятор прикладывает к двигателю максимальное напряжение, если текущий ток меньше задания, и минимальное напряжение, если ток больше задания.

Данная структура реализована в самом простом виде в качестве обучающего примера. В реальном применении структура обычно выполняется более сложно: регуляторы тока делаются с возможностью прикладывания трех уровней напряжения (-U, 0, +U) и имеют токовый коридор, а блок формирования заданий кроме углового положения также учитывает текущую частоту вращения и уровень задания тока, формируя при этом разные углы коммутации.

Подробнее пусконаладка данного режима работы рассмотрена в **Ошибка! Источник ссылки не найден.**.

5.2. ПИД регулятор

Упрощенная схема ПИД-регулятора показанна на Рис. 5.9.

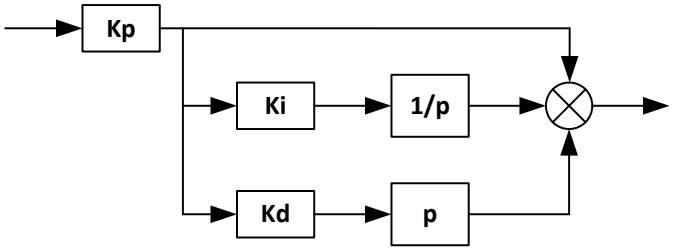


Рис. 5.9 Структура ПИД-регулятора

ПИД регулятор реализован в файле V_pid_reg3.c в виде модуля `TPidReg3`. В проекте модуль ПИД регулятора используется в качестве регуляторов тока, скорости (частоты вращения).

Наиболее простой и часто применяемый способ настройки таких регуляторов – подбор коэффициентов. К примеру, чтобы настроить коэффициенты ПИ-регулятора, можно сначала обнулить Ki, Kd и задать маленький Kp. Повышать Kp до тех пор, пока переходной процесс реакции на скачок задания не станет колебательным. Несколько снизить Kp до устранения колебательности (например, в два раза). Аналогично повышать Ki до достижения приемлемого быстродействия регулятора или до появления колебательности. Попеременной настройкой коэффициентов добиться требуемого быстродействия и формы переходного процесса (технический оптимум, симметричный оптимум и т.п.). Удобнее всего настраивать регулятор наблюдая в осциллограмме реакцию на скачок задания, хотя иногда можно настроить регулятор и «на слух», определяя, когда начались колебания и снижая коэффициент.

Регуляторы токов по осям d и q настраиваются одновременно, с одинаковыми коэффициентами. Сначала настраиваются внутренние контура системы управления, а затем внешние.

Более подробное описание структуры данного ПИД регулятора можно изучить в документе «pid_reg3.pdf» от Texas Instruments, который можно легко найти в сети Интернет.

5.3. Кривая U/f

Кривая U/f – зависимость напряжения, прикладываемого к двигателю, от частоты. Данный блок служит для расчета величины выходного напряжения в зависимости от выходной частоты (т.н. закон управления двигателем). Расчет осуществляется методом линейной интерполяции между двумя опорными точками:

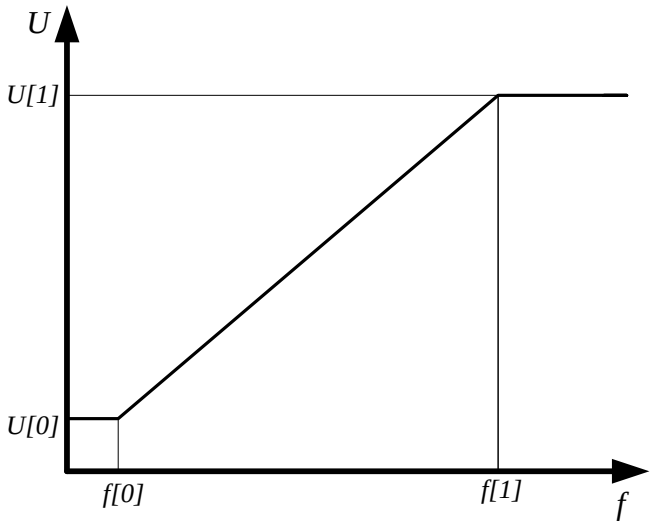


Рис. 5.10 Вид кривой U/f

Настройки модуля содержатся в группе **Кривая U-f**. Напряжение и частота каждой точки могут быть заданы пользователем.

Кривая U/f симметрична для положительной и отрицательной скорости.

Модуль применяется в структуре скалярного управления. Реализован в файле vhzprof.c в виде модуля TVhzProf.

5.4. Задатчик интенсивности (ЗИ)

Данный блок обеспечивает ограничение темпов разгона/торможения (ускорение) двигателя.

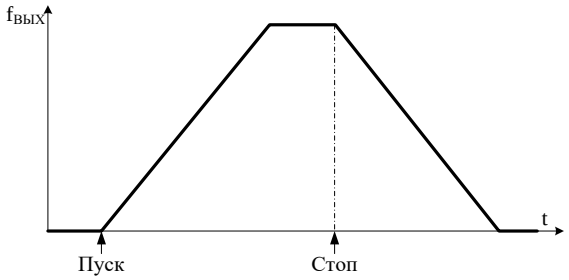


Рис. 5.11 Пуск и торможение по задатчику интенсивности

Настройки темпа разгона/торможения задается параметром **ЗИ.Т разгона до номинала**.

Темп ускорения и замедления это время, за которое частота с нуля достигает номинальной. Т.е., если номинальная частота соответствует 100Гц, темп разгона «5 секунд» значит, что с 0 Гц до 100Гц частота будет нарастать 5 секунд. Соответственно, например, с 20 Гц до 40 Гц разгон будет идти 1 секунду. Торможение, соответственно, наоборот. Номинальная частота рассчитывается исходя из параметров **Базов. параметры. Номинальная скорость** и **Базов. параметры. Число пар полюсов** и отображается в параметре **Базов. параметры. Номинальная частота**.

Задатчик интенсивности применяется при разгоне и торможении двигателя в большинстве структур управления. Реализован в файле v_rmp_ctrl.c в виде модуля TRMPCtrl.

5.5. Модуль фазных преобразований

Фазные преобразования переводят трехфазную систему величин A,B,C со сдвигом на 120° в двухфазную систему α,β со сдвигом 90°. В зарубежной литературе называется преобразованием Кларк (Clarke).

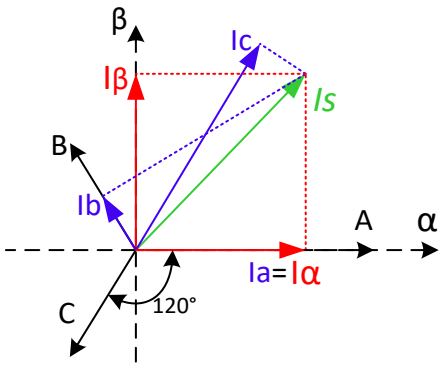


Рис. 5.12 Принцип фазных преобразований

Преобразование осуществляется с сохранением амплитуды тока фазы, т.е. а $I_A = I_\alpha$.

В ПО используется обозначение, принятое за рубежом. Входы модуля clarke: as – фаза A, bs – фаза B. Выходы: ds – фаза α, qs – фаза β. Формулы, по которым осуществляется преобразование в данном модуле, следующие:

$$ds = as;$$

$$qs = \frac{as + 2 \cdot bs}{\sqrt{3}}$$

В ПО данный модуль используется для преобразования токов статора из трехфазной системы координат в двухфазную. Реализован в файле clarke.c в виде модуля TClarke.

5.6. Модуль координатных преобразований

Координатные преобразования поворачивают двухфазную систему со сдвигом 90° на заданный угол. В зарубежной литературе называется преобразованием Парка (Park). В основном используется для преобразования стационарных осей α,β к вращающимся d,q и обратно (обратно часто используется инверсное преобразование, ipark).

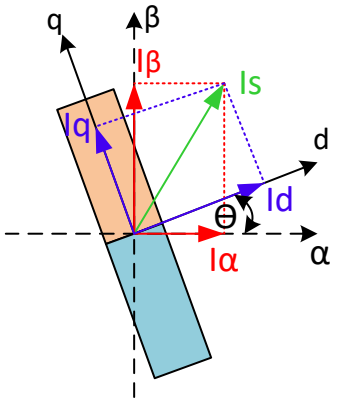


Рис. 5.13 Принцип координатных преобразований

В ПО обозначения используются следующие. Входы модуля park: ds – обычно фаза α, qs – обычно фаза β, ang – угол поворота осей. Выходы: de – обычно фаза d, qe – обычно фаза q. Для модуля ipark входные и выходные оси названы наоборот (входы – de и qe, а выходы ds и qs).

Формулы, используемые в ПО в модулях park и ipark представлены ниже.

$$de = ds \cdot \cos(ang) + qs \cdot \sin(ang);$$

$$qe = qs \cdot \cos(ang) - ds \cdot \sin(ang).$$

$$ds = de \cdot \cos(ang) - qe \cdot \sin(ang);$$

$$qs = qe \cdot \cos(ang) + de \cdot \sin(ang).$$

Данный модуль используется практически во всех структурах системы управления для обеспечения вращения координат. Реализован в файле park.c в виде модуля TPark (прямое преобразование координат) и в файле ipark.c в виде модуля TIPark (обратное преобразование координат).

5.7. Модуль ШИМ

Модуль ШИМ (широтно-импульсной модуляции) предназначен для управления ключами инвертора и отработки заданного вектора напряжения. Здесь будет описана организация самого распространённого модуля ШИМ для шестиключевого инвертора напряжения и предназначенного для трехфазных синхронных и асинхронных двигателей. Модули ШИМ для ДПТ и ВИД в данном пункте рассмотрены не будут. Модуль ШИМ реализован в файле V_PWM_Module.c в виде модуля TPWM_Module.

Задание на вход модуля поступает в виде заданий мгновенных напряжений $U_{\alpha.зад}$ и $U_{\beta.зад}$ и в ортогональной системе координат с осями альфа, бета (в двухфазной системе координат). Если модуль ШИМ включен, то на следующем периоде ШИМ заданное напряжение будет отработано. Модуль отрабатывает постоянное напряжение, никакой эволюции вектора во времени он не осуществляет: «синусоидальность» выходного напряжения должен обеспечивать модуль более высокого уровня путем соответствующего изменения напряжений задания $U_{\alpha.зад}$ и $U_{\beta.зад}$.

Модуль ШИМ в данном проекте рассчитывается на фиксированной частоте 10кГц, на этой частоте рассчитываются и обновляются уставки сравнения таймеров ШИМ микроконтроллера. Однако частота ШИМ может быть задана любой. При этом отработка ШИМ и изменение его задания происходит асинхронно. Такой подход имеет ряд достоинств и недостатков, которые были рассмотрены в п. 3.5.

Модуль позволяет «на ходу» изменять частоту ШИМ, мёртвое время, тип алгоритма ШИМ.

На Рис. 5.14 показана общая структура модуля ШИМ.

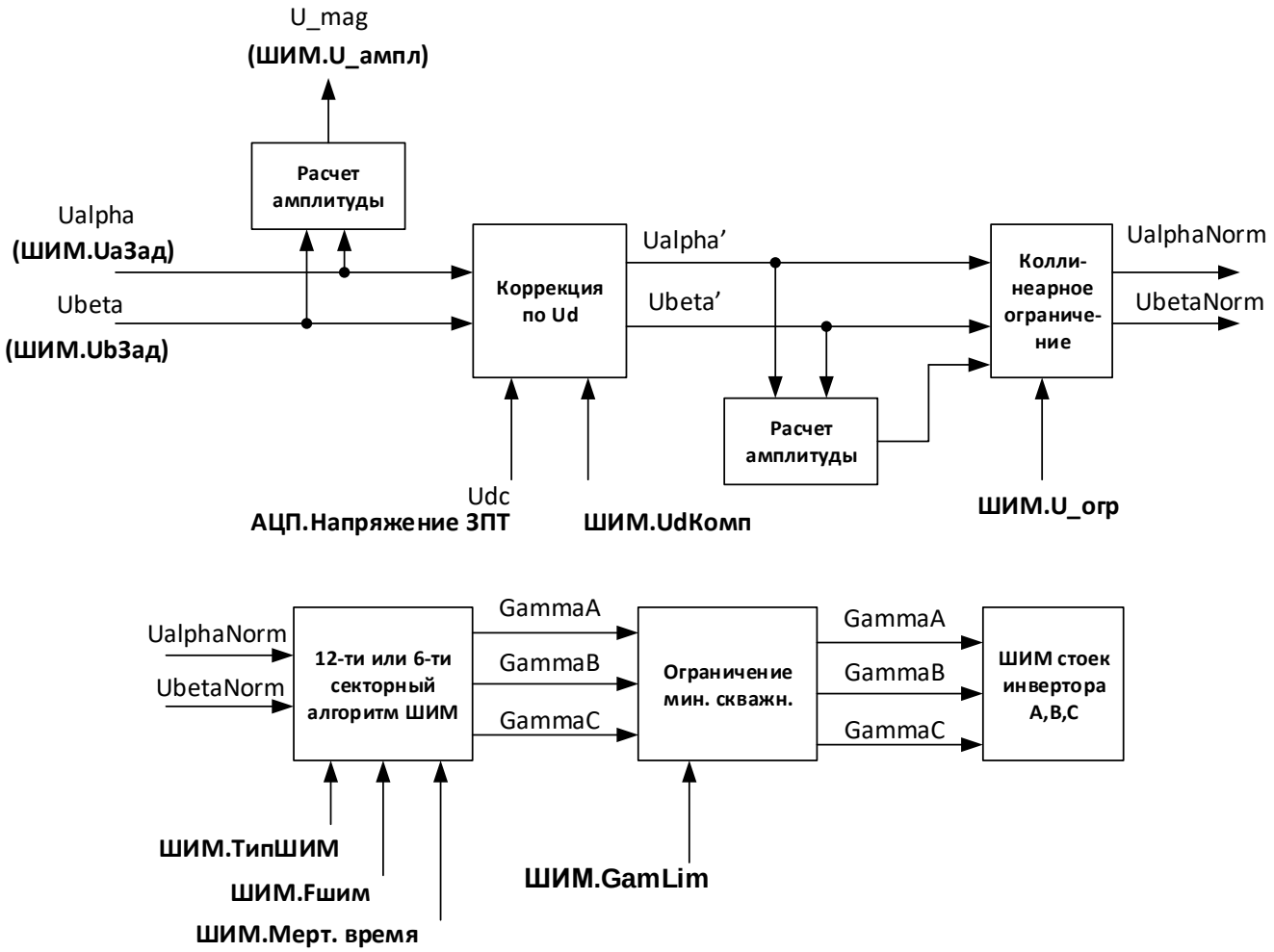


Рис. 5.14 Схема алгоритма ШИМ

Заданные напряжения по перпендикулярным осям альфа (Ualpha) и бета (Ubeta) поступают на вход модуля из более высокоуровневой структуры управления двигателем (из SM_Ctrl). Их мгновенные значения доступны пользователю для чтения в переменных **ШИМ.UaЗад** и **ШИМ.UbЗад**. Далее следует блок расчета их амплитуды, результат которого помещается в **ШИМ.U_ампл**. После этого напряжения Ualpha и Ubeta поступают на блок коррекции выходного напряжения в зависимости от напряжения на звене постоянного тока. Так, если напряжение на звене постоянного тока понижено, то требуется пропорционально этому повысить задание на выходное напряжение. На блок поступает флаг разрешения работы компенсации (параметр **ШИМ.Компенсация Ud**) и напряжение Ud с соответствующего датчика. Кроме того, для более точной подстройки компенсации может использоваться коэффициент **ШИМ.Коэф. компенсации Ud**, использующийся в формуле компенсации:

Ualpha' = Ualpha * 1/[1+(Uизмер - Uном)* Квых/ Uном],

Ubeta' = Ubeta* 1/[1+(Uизмер - Uном)* Квых/ Uном],

где

Квых – коэффициент компенсации **ШИМ. Коэф. компенсации Ud**;

Ualpha, Ubeta – напряжения по осям альфа, бета до блока компенсации пульсаций;

Ualpha', Ubeta' – напряжения по осям альфа, бета после блока компенсации пульсаций;

Uном – номинальное напряжение на звене постоянного тока;

Uизмер – измеренное напряжение на звене постоянного тока.

Следует заметить, что работа блока компенсации никак не сказывается на отображаемой амплитуде выходного напряжения **ШИМ.U_ампл**, так как реальное напряжение на выходе инвертора при изменении напряжения Ud и работе блока компенсации также меняться не должно. Кроме того, существуют ограничения на возможности компенсации связанные с максимальным

напряжением, реализуемым инвертором. Так, если привод работает на максимальной частоте с максимальным напряжением, и в это время напряжение на звене постоянного тока снижается, то у модуля ШИМ нет резерва, чтобы повысить задание на амплитуду выходного напряжения (за исключением работы «по граням шестигранника», что описано ниже). Правильно коррекция может работать только или при повышении напряжения на звене постоянного тока, или при запасе в амплитуде задания выходного напряжения.

Далее следует блок расчета амплитуды, но уже скорректированных Ualpha и Ubeta. Их амплитуда поступает на блок ограничения выходного напряжения, который обеспечивает коллинеарное ограничение заданного вектора напряжения. Для более детального понимания работы блока коллинеарного ограничения вектора напряжения обратимся к рисунку ниже (справедливо только для векторной ШИМ).

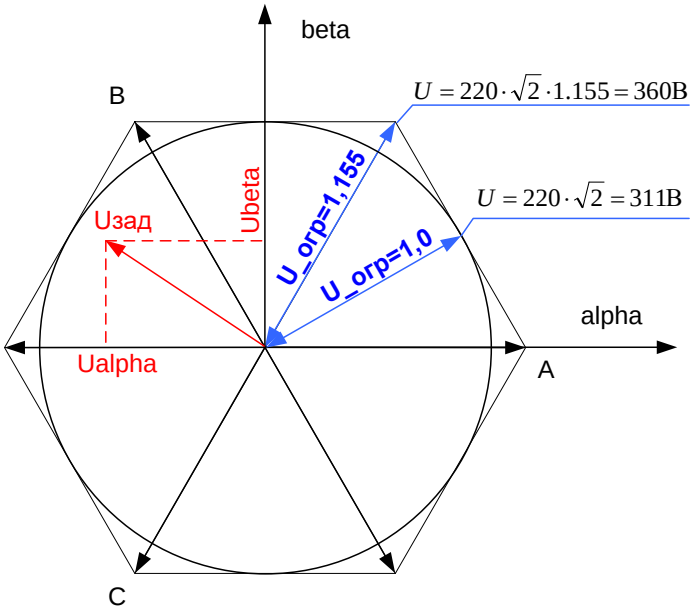


Рис. 5.15 Графическое представление работы векторной ШИМ

На нем изображены шесть базовых векторов инвертора напряжения, расходящихся из центра координат. Это максимальные фазные амплитудные напряжения, которые могут быть приложены к двигателю при тех или иных комбинациях ключей инвертора. Например, базовый вектор, направленный в сторону оси фазы А, может быть получен открытием верхнего ключа стойки А инвертора и открытием нижних ключей стоек В и С. Все напряжения, которые могут быть получены в результате использования широтно-импульсной модуляции, лежат внутри образованного базовыми векторами шестиугольника и не могут превосходить его. На данном рисунке отмечены сразу две системы координат: образованная перпендикулярными осями alpha и beta, в которых происходит работа модуля векторной ШИМ и остальной системы управления, а также соответствующие им трехфазные оси A,B,C. Если система управления задает вектор напряжения небольшой амплитуды, например Uзад, как показано на рисунке, то блок ограничения не используется. Если же Uзад превысит шестиугольник (что может быть, если, например, пользователь задаст в модуле кривой U/f слишком большое напряжение), то требуется ограничение вектора. Используется так называемое коллинеарное ограничение, при котором угловое положение вектора (и, соответственно, направление) сохраняются, а амплитуда уменьшается до необходимой величины. Если ограничивать вектор напряжения вписыванием в шестиугольник, то при таком ограничении выходное фазное напряжение будет не синусоидальным (похожим на трапецию), поэтому чаще всего используется ограничение заданного вектора окружностью, вписанной в шестиугольник. Окружность показана на рисунке. Параметр **ШИМ.U_огр** управляет амплитудой окружности, которой ограничивается вектор напряжения. Когда этот параметр равен 1.0, то вектор будет ограничен максимально-допустимым напряжением, при котором еще не происходит искажения синусоидальности (справедливо для векторных типов ШИМ). Если задавать этот параметр больше 1.0, то начнет автоматически работать ограничение вектора шестиугольником там, где окружность превысила его. Так, если

задать **ШИМ.U_огр=1,155**, то окружность будет описана вокруг шестиугольника, в результате чего ограничение будет происходить по граням шестиугольника. Это является предельным выходным напряжением для инвертора. Например, при напряжении на звене постоянного тока 540 В , т.е. когда фазное действующее напряжение на входе 220 В, параметру **ШИМ.U_огр=1,0** соответствует фазное действующее напряжение на двигателе 220В (или 311 В амплитудное фазное) без учета падений напряжения на ключах и мертвого времени. Иногда требуется любыми способами повысить выходное напряжение, даже ценой искажения синусоидальности. Для этого необходимо задать **ШИМ.U_огр=1,155** (и соответствующие напряжения в модуле кривой U/f). В этом случае выходное напряжение в нашем примере повысится до 360 В амплитудного фазного напряжения (при 540 В на ЗПТ). Кроме повышения напряжения такое ограничение дает также уменьшение потерь в ключах инвертора (при работе на максимальном напряжении). Это связано с конкретным алгоритмом реализации векторной ШИМ. В этом случае количество коммутаций на периоде ШИМ сокращается в два раза (в векторной ШИМ).

Также следует отметить, что в некоторых типах инверторов напряжения применяется так называемое «бутстрепное» питание драйверов ключей. Это значит, что для питания драйвера верхних ключей инвертора используется конденсатор, который заряжается в то время, когда открыт нижний ключ инвертора. Поэтому в схеме ШИМ для таких инверторов необходимо, чтобы на какой-то промежуток времени нижние ключи инвертора обязательно открывались на периоде ШИМ. Иначе ёмкость разрядится и верхние ключи перестанут управляться. Необходимо использовать ограничение максимальной скважности инвертора в модуле ШИМ, чтобы скважность по каждой стойке никогда не превышала, например, 90%. Для этого используется параметр **ШИМ. U_lim**. Если скважность приблизится к 100%, то драйвера верхних ключей останутся без питания, что в большинстве случаев приводит к срабатыванию аппаратной аварии драйверов ключей инвертора. Поэтому перед работой с инвертором следует выяснить, как у него организовано питание драйверов ключей и соответствующим образом настроить параметр **ШИМ.U_lim**. Например, в отладочном комплекте VectorCARD с инвертором используется именно «бутстрепное» питание драйверов ключей.

При этом параметр **ШИМ. U_lim** по-разному работает для векторной и скалярной ШИМ. Для векторной ШИМ его можно задать равным 90%. Для синусоидальной ШИМ предельным работоспособным значением является ограничение 70% в силу более низкого выходного напряжения в данном типе модуляции.

Указанные выше подготовительные нормировочные операции выполняются в функции PWM_Module_NormInput.

Далее, скорректированные и ограниченные напряжения, которые для отличия названы UalphaNorm и UbetaNorm, поступают непосредственно на блок, реализующий алгоритм переключения ключей в соответствии с алгоритмом выбранного типа ШИМ (синусоидальный или векторный). Блок управляется параметрами:

- **ШИМ.Тип ШИМ**: выбирает синусоидальный или 6-ти секторный векторный алгоритм ШИМ;
- **ШИМ.Частота ШИМ**: задает частоту ШИМ в кГц;
- **ШИМ.Мертвое время (мкс)**: задает величину мертвого времени в мкс.

Результатом работы блока являются скважности для каждой из стоек инвертора GammaA, GammaB, GammaC. Под скважностью понимается отношение времени открытого состояния верхнего ключа стойки к периоду ШИМ.

Алгоритмов управления ключами два: синусоидальная и векторная 6-ти секторная модуляция.

Скважности синусоидальной ШИМ в относительных единицах рассчитываются по формулам:

$$\left.\begin{aligned} U_{A.зад.} &= U_{\alpha.зад.}; \\ U_{B.зад.} &= \left(U_{\beta.зад.} \cdot \frac{\sqrt{3}}{2} - U_{\alpha.зад.} \cdot \frac{1}{2} \right); \\ U_{C.зад.} &= \left(-U_{\beta.зад.} \cdot \frac{\sqrt{3}}{2} - U_{\alpha.зад.} \cdot \frac{1}{2} \right). \end{aligned} \right\}$$

$$\left.\begin{aligned} \gamma_A &= \frac{1}{2} - U_{A.зад.}; \\ \gamma_B &= \frac{1}{2} - U_{B.зад.}; \\ \gamma_C &= \frac{1}{2} - U_{C.зад.}. \end{aligned} \right\}$$

В формулах принимается, что напряжения $U_{\alpha.зад}$ и $U_{\beta.зад}$ заданы в относительных единицах. Первая система уравнений пересчитывает задание из двух фаз в три фазы, получая заданные напряжения для фаз A,B,C. Вторая система уравнений рассчитывает значения скважностей на инверторе. Формулы интуитивно понятны: скважность отклоняется от значения 50% пропорционально напряжению, требуемого для конкретной фазы.

Для шестисекторной векторной ШИМ скважности рассчитываются по более сложному методу, предложенному к.т.н. Изосимовым Д.Б. (ИПУ РАН). Метод подробно рассмотрен в диссертационной работе Чуева Павла Вячеславовича «Разработка систем векторного управления асинхронными приводами на базе специализированных сигнальных микроконтроллеров», которая доступна, например, по электронному адресу http://motorcontrol.ru/wp-content/uploads/2015/11/Chuev_vector_control.pdf , глава 3.

Синусоидальная ШИМ, в отличие от векторной, управляет всеми ключами инвертора на периоде ШИМ (больше потери), имеет на 15% более низкое доступное выходное напряжение по сравнению с векторной ШИМ, но имеет меньшую степень подверженности влиянию мертвого времени инвертора. Известен метод ШИМ, сочетающий преимущества обоих методов и рассмотренный в статье Анучин А.С. «ШИРОТНО-ИМПУЛЬСНАЯ МОДУЛЯЦИЯ МЕТОДОМ РЕАЛИЗАЦИИ МГНОВЕННЫХ ФАЗНЫХ ПОТЕНЦИАЛОВ ДЛЯ ТРЕХФАЗНЫХ ИНВЕРТОРОВ НАПРЯЖЕНИЯ», которая доступна по электронному адресу http://motorcontrol.ru/wp-content/uploads/2015/11/PW1.pdf . Однако в данном ПО такой метод ШИМ не реализован.

После расчета скважностей на три стойки ключей инвертора в относительных единицах, они умножаются на период несущего таймера ШИМ и загружаются в регистры сравнения модулей ШИМ.

Далее рассчитанные скважности стоек инвертора поступают на блок ограничения минимальной скважности, реализованный в микроконтроллере аппаратно. Прикладывать к ключам управляющий сигнал, сравнимый с временем их коммутации в ряде случаев нежелательно (для мощных инверторов), нужно искусственно ограничить минимальные скважности. Так, если скважность любой из стоек оказываются меньше заданного в параметре **ШИМ.Ограничение мин. скважн.** времени в мкс, происходит обнуление этой скважности. Если **ШИМ.Ограничение мин. скважн.** равен нулю, блок ограничения минимальной скважности не используется. После всех описанных преобразований скважности GammaA, GammaB, GammaC попадают непосредственно на выход контроллера в виде ШИМ-сигнала и управляют 6ю ключами инвертора.

5.8. Модуль АЦП

Модуль реализован в файле V_adc.c в виде модуля с названием TAdcDrv. Модуль измеряет токи фаз двигателя (в общем случае токи фазы A, B, C, D, но в зависимости от типа аппаратной части могут отсутствовать те или иные датчики) и напряжение звена постоянного тока инвертора. Параметры отображаются в группе АЦП. О настройке коэффициентов датчиков подробно написано в п.п. 4.8.

Для совместимости программного кода с разными типами инверторов применяется лишь одно измерение тока на одном периоде ШИМ. В некоторых типах инверторов может применяться шунтовые датчики токов фаз, в которых измерить корректно ток в фазе можно только в тот момент, когда включен нижний ключ инвертора. Измерение тока производится строго в тот момент, когда во время работы ШИМ открываются нижние ключи инвертора, т.е. в момент приложения нулевого вектора напряжения. Это происходит, когда таймеры микроконтроллера, формирующие ШИМ, достигают нуля при счете вверх-вниз. В этот момент по событию запускается измерение токов АЦП – см. Рис. 5.16.

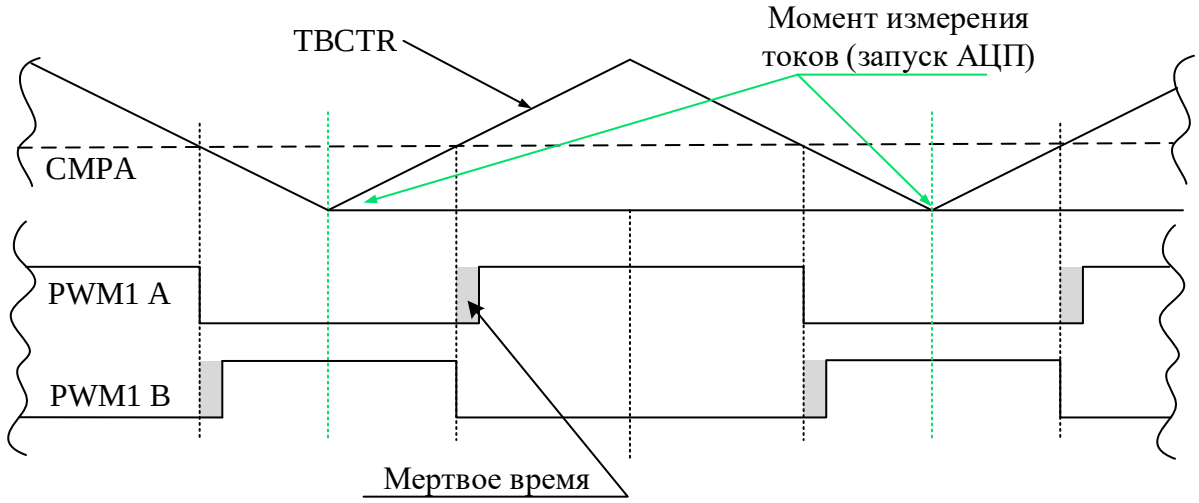


Рис. 5.16 Синхронизация ШИМ и измерения токов АЦП

Для улучшения точности измерения (уменьшения шума АЦП) применяется усреднение измерений, полученных на разных периодах ШИМ. Так, основной цикл расчета контуров тока выполняется на частоте 10кГц. При частоте ШИМ 20кГц образуется два измерения токов, которые помещаются в аппаратное FIFO модуля АЦП микроконтроллера, а затем из них вычисляется среднее значение, которое уже используется в системе управления. При большей частоте ШИМ большее количество точек будет применяться для усреднения.

При необходимости в зависимости от особенностей своей аппаратной конфигурации пользователь может реорганизовать работу модуля АЦП. Текущая конфигурация является наиболее совместимой со всеми возможными типами инверторов и датчиков тока.

Отдельно стоит упомянуть о преобразовании изменяемого АЦП сигнала в целочисленный формат переменных системы управления IQ8.24. Такой пересчет осуществляется с учетом текущих базовых единиц, выбранных для измеряемой величины (см. п.3.10), а также с учетом настроек конкретного канала: коэффициента усиления и смещения. Чтобы упростить вычисления, выполняющиеся на частоте 10кГц вызова основной функции обработки АЦП (AdcDrv_PWM_calc), в фоновой функции расчёта модуля (AdcDrv_slow_calc) подготавливаются промежуточные коэффициенты, которые включают в себя все остальные с учетом необходимого преобразования форматов. В основной функции расчета модуля используются эти промежуточные коэффициенты и минимальный набор математических операций для обеспечения высокого быстродействия.

5.9. Модуль ДПР на дискретных элементах Холла

ДПР состоит из трех дискретных элементов (чаще всего это элементы Холла, реагирующие на магнитное поле, но бывают датчики и на другом физическом принципе), которые меняют свое

выходное электрическое состояние в зависимости от положения ротора двигателя. На Рис. 6.8 показаны временные диаграммы работы датчика. Из дискретных сигналов, поступающих на вход контроллера, программным драйвером формируется дискретное значение углового положения с шагом в 60 электрических градусов (доступно для наблюдения в параметре **ДПР-Холл.ЭлУгол6**). На основе этого углового положения и текущей частоты вращения формируется экстраполированное значение углового положения, непрерывное на электрическом периоде двигателя (доступно для наблюдения в параметре **ДПР-Холл.ЭлУгол**). Именно это значение используется далее для векторной системы управления. Также к параметру **ДПР-Холл.ЭлУгол** прибавляется задаваемое пользователем значение смещения датчика положения ротора, используемое для программной юстировки (калибровки) положения датчика (параметр **ДПР-Холл.СмещУгол**). Помимо углового положения модуль вычисляет частоту вращения двигателя (скорость), засекая время между метками датчика. Значение скорости используется для регулятора скорости двигателя в векторной систему управления и доступно для наблюдения в параметре **ДПР-Холл.Скорость**.

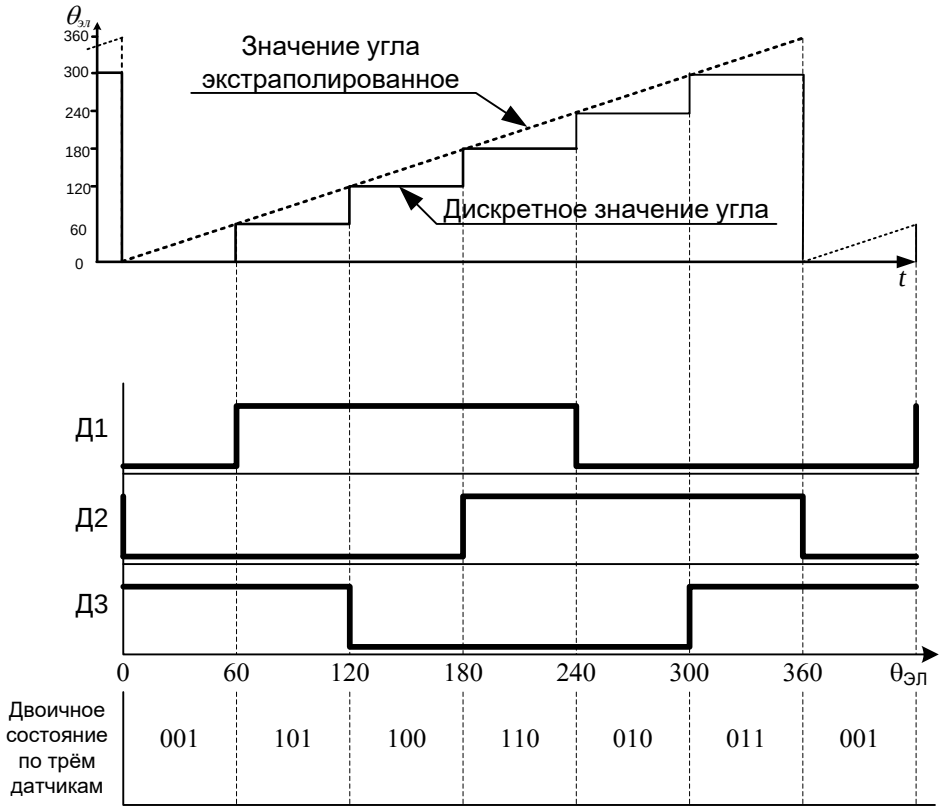


Рис. 5.17 Временные диаграммы работы датчика положения ротора

Интерполяцию можно отключить параметром **ДПР-Холл.Инт-полятор вкл**. Она также отключается автоматически при скорости ниже уставки **ДПР-Холл.Уставка скорости**. Скорость обнуляется, если состояние датчика не изменялось более **ДПР-Холл.Уставка обнуления скорости** в миллисекундах. Смещение угла настраивается параметром **ДПР-Холл.СмещУгол**.

В параметре **ДПР-Холл.Дискретные входы** можно наблюдать текущее состояние дискретных входов микроконтроллера, используемых для приёма сигнала датчика. В параметре **ДПР-Холл.Счётчик прерываний по приему** видно срабатывание прерываний по изменению фронтов сигнала датчика.

Модуль реализован в файле V_DPR_eCAP.c в виде модуля с названием TDPReCAP, кроме того имеются обработчики прерываний CAP в файле main.c.

5.10. Модуль инкрементального энкодера

Модуль обработки инкрементального энкодера реализован в файле V_QEP.c в виде модуля с названием TpsspeedEqep. Инкрементальный энкодер, в отличие от датчика на элементах Холла, не выдаёт абсолютного положения ротора, а лишь показывает, на какой угол и в каком направлении поворачивается двигатель. В энкодере может присутствовать, а может отсутствовать индексная (реперная) метка – одна метка на один оборот вала двигателя, которая обычно служит

для определения абсолютного положения вала ротора, когда вал провернется до этой метки в процессе работы. Количество импульсов на оборот в инкрементальном энкодере может быть разным и в зависимости от задачи варьируется от 64 до 100000 меток на оборот или даже более широких пределах. Следует отметить, что существует некоторая путаница между количеством периодов датчика и его количеством импульсов. При количестве периодов датчика, например, 1000, суммарное количество импульсов по обоим каналам на обороте составит 4000. В параметр **ДПР-Энкодер.Кол-во периодов [5152,01]** вводится именно число до учетверения.

В модуле обработки механический угол поворота вала ротора преобразуется в электрический (электрический оборот равен механическому, разделенному на число пар полюсов)

Угол положения ротора (электрический и механический) отображается в параметрах **ДПР-Энкодер.Угол. электр.** и **ДПР-Энкодер.Угол мех.**. Частота вращения, рассчитываемая по энкодеру, отображается в **ДПР-Энкодер.Скорость**.

Для работы с синхронным двигателем в векторной системе управления важно знать абсолютное значение углового положения ротора, чтобы обеспечить правильную ориентацию вектора тока статора относительно него. Однако после включения системы управления это положение в общем случае по инкрементальному датчику положения неизвестно, так как инкрементальный энкодер показывает только относительное смещение ротора. Абсолютное положение может быть известно только при достижении индексной метки, но чтобы её достичь, надо начать вращение, а векторную систему нельзя включить, пока нет известного положения ротора. Чтобы выйти из замкнутого круга применяют либо вспомогательный датчик (на элементах Холла), либо запускают систему управления в разомкнутом по положению виде, дожидаются срабатывания индексной метки и потом переходят в векторную систему управления. Существуют и другие методы. Чтобы не усложнять данное ПО реализацией частных случаев работы с энкодером, пользователю предлагается после каждого перезапуска самостоятельно проводить настройку инкрементального энкодера: задавать в параметре начальное смещение угла, чтобы при условно нулевом угле поворота ротора датчик также показывал нулевой электрический угол. Это смещение задается параметром **ДПР-Энкодер.СмещУгол**. Это смещение необходимо настраивать каждый раз заново при включении системы управления, так как система управления «не знает», в каком в данный момент положении находится вал двигателя после включения и что считать нулевым положением. Процедура настройки смещения может быть ручной или автоматической и подробно описана в п.п. 6.1.4. В дальнейшем пользователь может самостоятельно реализовать необходимый ему способ автоматической юстировки энкодера, например, по сигналу с датчика на элементах Холла, по реперной (индексной) метке (если она имеется в используемом датчике), по концевым выключателям и т.п. в зависимости от задачи. Следует напомнить, что такая настройка нужна только для синхронной машины: асинхронный двигатель или ДПТ знания абсолютного положения ротора для работы двигателя не требуют.

В то время как определение углового положения по аппаратному подсчету меток не вызывает вопросов, следует пояснить способ определения частоты вращения (скорости). В модуле реализовано два принципиально разных способа расчета скорости:

1) Аппаратный метод, основанный на возможностях модуля QEP засекать время между заданным количеством импульсов (или между каждым импульсом). При этом количество меток, между которыми засекается время, с ростом скорости вращения автоматически переключается: на низкой скорости вращения засекается время между каждой следующей меткой (для повышения быстродействия измерений), а с ростом скорости делитель количества меток увеличивается (для повышения точности измерений). Этот способ показывает хорошую точность измерения скорости, но имеет проблемы с помехоустойчивостью: если аппаратная часть формирует «ложные» импульсы энкодера (например, от помех работы инвертора), то данный способ захватывает ложное «малое» время между метками и приводит к ложному определению скорости вращения. Кроме того, модуль плохо ведет себя на сверхнизкой скорости вращения и при реверсе, так как засекает время только между двумя соседними импульсами, произошедшими при вращении в одном направлении, и выдает частоту

вращения равной нулю при дрожании вала ротора на две метки, что может быть нежелательно для сервоприводов.

2) Программный метод, основанный на вычислении скорости вращения по производной изменения углового положения (которое меньше подвержено влиянию помех). Метод в какой-то мере лишен двух недостатков аппаратного способа измерения скорости, но имеет существенно более низкую точность определения скорости (которой, впрочем, достаточно для большинства практических задач).

По умолчанию активирован программный способ измерения скорости, как более робастный. Но имеется возможность сменить алгоритм обнулив параметр **ДПР-Энкодер.Рассчитывать скорость программно [5152h.03h]**. В активированном программном способе измерения скорости существенной является настройка фильтрации производной углового положения, которая задается в параметре **ДПР-Энкодер.К быстродействия фильтра скорости [5152h.0Bh]**. Чем больше этот параметр, тем слабее значение производной углового положения «зафильтровано», выше быстродействие по времени, но ниже точность измерения скорости.

5.11. Модуль цифрового осциллографа (даталоггер)

Реализован в файле V_data_log.c в виде модуля TDataLog. Осуществляет цифровое осциллографирование (запись в четыре массива) значений из четырех выбранных пользователем переменных. Модуль работает совместно с режимом «выборка» осциллографа ПО UniCON.

Пользователь может выбрать в словаре объектов до четырех любых переменных, которые он хочет осциллографировать, и добавляет их в осциллограф UniCON. Затем нужно перейти на вкладку осциллографа «выборка». В этот момент UniCON записывает адреса (индекс, подындекс) желаемых для осциллографирования объектов словаря CANopen в параметры **dlog.ind_subind1-4** группы **dlog**. Далее функция dlog_background_analizer в ПО микронтроллера по известным адресам словаря CANopen вычисляет адреса этих переменных в оперативной памяти микроконтроллера, настраивая на них указатели dlog_iptr1-4. Также та функция обрабатывает поле команд control, через которое UniCON управляет осциллографом (команда на запуск, опрос готовности данных и т.п.).

В функции data_log_update осциллограф каждый такт вызова модуля заполняет четыре 16-ти разрядных массива по 256 точек значениями переменных, которые выбрал пользователь. Функция вызывается с частотой вычисления системы управления (10кГц) и записывает в массив значения очередной точки. Запись можно делать не каждый такт расчета системы управления, а с прореживанием, которое задается через UniCON на вкладке «выборка» в поле ввода «делитель»: например, каждый пятый такт расчета или каждый сотый (но не более 255 – с таким прореживанием данных рационально перейти на осциллографирование в реальном времени, на вкладку «непрерывный» режим работы осциллографа).

Запуск осциллограммы может производиться как в от команды UniCON, так и самостоятельно по триггеру в ПО микроконтроллера, чтобы можно было получить осциллограмму, соответствующую какому-либо событию. Запуск осциллографа внутри ПО осуществляется функцией dlog.set_mode(1, &dlog); Эта функция запускает запись осциллограммы с того момента, когда она вызвана. Эта функция размещена во многих участках кода программы микроконтроллера – при запуске системы управления в том или ином режиме, при переходе в останов и так далее. Чтобы выбрать, когда конкретно нужно в данном опыте запустить осциллограф, сделана битовая переменная dlog.StartBits, в которой пользователь через UniCON устанавливает тот или иной бит, соответствующий тому или иному событию в системе управления. Так, например, запуск осциллографа по событию включения векторной датчиковой системы управления синхронным двигателем выполняется следующими строками:

```
if (dlog.StartBits & (1 << 4))
      dlog.set_mode(1, &dlog);
```

Что означает «запустить осциллограф, если пользователь установил 4й бит начиная с нулевого». Пользователь должен установить в параметрах **dlog.StartBitL** и **dlog.StartBitH** (нижняя и верхняя части переменной StartBits) желаемые биты, соответствующие наступлению желаемого события в

программе. Какие биты каким событиям соответствуют можно узнать, выполнив поиск строки dlog.set_mode в проекте ПО.

UniCON по нажатию кнопки «обновить» считывает текущий статус осциллографа. Если есть уже записанная осциллограмма, UniCON предложит отобразить её. Если текущей осциллограммы не записано, в момент нажатия кнопки «обновить» запишется и отобразится новая осциллограмма.

Чтобы иметь возможность видеть предысторию, а именно осциллограмму до и во время наступления определенного события, применяется следующая схема. После окончания загрузки осциллограммы из микроконтроллера UniCON запускает осциллограф в режим постоянной кольцевой записи выбранных данных. Далее пользователь создает условия эксперимента, чтобы запись осциллограммы внутри МК произошла снова по заданному им событию. Когда внутри ПО микроконтроллера вызывается функция записи осциллограммы dlog.set_mode, то дальше ещё записывается еще некоторое количество точек, а затем осциллограф останавливается. Параметром **dlog.Количество точек предыстории [5150h.0Ah]** в процентах задается, сколько точек осциллограммы кольцевого буфера в процентах должно остаться. Так, если задано значение 70%, то пользователь увидит осциллограмму, где 179 точек будут показывать график до наступления события (вызова функции dlog.set_mode), а 77 точек график после события. Такая схема позволяет увидеть данные в системе управления, например, как до наступления аварийной ситуации, так и после.

Загрузка записанной осциллограммы в UniCON производится через передачу массива данных драйвера CANopen (функция блочной передачи), или, если эта функция по какой-то причине не работает, поэлементным вычитыванием данных через служебную переменную **dlog.next_value_var**.

Особо стоит отметить, что осциллограф 16ти разрядный. При добавлении в осциллограф 32х разрядной переменной в осциллографе записывается её старшая часть. Для переменных в формате IQ8.24, на которых основана вся целочисленная математика данного проекта, это приемлемо. После усечения переменной получается формат 8.8, что достаточно для наблюдения большинства процессов. Таким образом следует помнить, что в режиме работы «выборка» осциллографа переменные будут иметь соответствующее усечение формата разрядности (в отличие от «непрерывного» режима, где переменные запрашиваются в реальном времени, без усечения формата, но медленно). Если добавить в осциллограф какую-либо 32х разрядную переменную в формате Uint32, то в режиме «выборка» не будет видно изменения его младших 16ти разрядов, что обычно неинформативно. Необходимо для отображения такой переменной копировать её значение внутри программы МК в 16ти разрядную переменную, например, в отладочные переменные Debug3,4 и смотреть отдельно её младшую и, при необходимости, старшую часть отдельно.

Дополнительную информацию о работе функции осциллографирования можно прочитать в руководстве пользователя на программу UniCON.

5.12. Модуль защит

Модуль защит является одним из наиболее важных элементов системы управления, так как обеспечивает быстрое распознавание некоторого числа аварийных событий. Это, в свою очередь, позволяет системе управления среагировать и предпринять необходимые меры по прекращению развития аварийной ситуации и безопасному отключению оборудования. Модуль обеспечивает не только защиту электрооборудования инвертора (защиту силовых ключей и т.п.), но и защиту непосредственно двигателя (защиту от коротких замыканий и т.п.). Фиксирование и отработка защит реализуется как программными (например, максимальный ток) средствами контроллера, так и аппаратными средствами драйвера (защита ключей инвертора от сверхтока и перегрева обрабатывается на аппаратном уровне, что позволяет диагностировать аварию и отключать оборудование с минимально возможными задержками).

Характеристики модуля:

- частота дискретизации (для программно-реализуемых защит) – 10 кГц;
- возможность фиксации до 32 аварийных событий;

- возможность оперативной настройки списка разрешенных и запрещенных аварий;

Список идентифицируемых аварий представлен в таблице.

№	Тип аварии	Описание	Параметр порога срабатывания в группе «Защиты»	Бит маскирования аварии
1.	**Понижение Udc**	Понижение напряжения на ЗПТ ниже уставки	**Минимальное напряжение**	mask1L.b1 1111 1110
2.	**Повышение Udc**	Повышение напряжения на ЗПТ выше уставки	**Максимальное напряжение**	mask1L.b2 1111 1101
3.	**Резерв**			mask1L.b3 1111 1011
4.	**Резерв**			mask1L.b4 1111 0111
5.	**Резерв**			mask1L.b5 1110 1111
6.	**Резерв**			mask1L.b6 1101 1111
7.	**Резерв**			mask1L.b7 1011 1111
8.	**Резерв**			mask1L.b8 0111 1111
9.	**Сбой прогр-ы 1кГц**	Программная ошибка 1кГц (превышение времени выполнения программного кода в прерывании 1кГц)	-	mask1H.b1 1111 1110
10.	**Сбой прогр-ы 10кГц**	Программная ошибка 10кГц (превышение времени выполнения программного кода в прерывании 10кГц)	-	mask1H.b2 1111 1101
11.	**Резерв**			mask1H.b3 1111 1011
12.	**Резерв**			mask1H.b4 1111 0111
13.	**АППАРАТНАЯ**	Аппаратная авария инвертора	-	mask1H.b5 1110 1111
14.	**Резерв**			Mask1H.b6 1101 1111
15.	**Резерв**			mask1H.b7 1011 1111
16.	**Резерв**			mask1H.b7 0111 1111
17.	**Резерв**			mask2L.b2 1111 1101
18.	**Резерв**			mask2L.b3 1111 1011
19.	**Резерв**			mask2L.b4 1111 0111
20.	**Резерв**			mask2L.b5 1110 1111
21.	**Макс. ток фазы А**	Превышение тока фазы А;	**Максимальный ток**	mask2L.b6 1101 1111
22.	**Макс. ток фазы В**	Превышение тока фазы В	**Максимальный ток**	mask2L.b7 1011 1111
23.	**Макс. ток фазы С**	Превышение тока фазы С	**Максимальный ток**	mask2L.b8 0111 1111
24.	**Резерв**			mask2H.b1 1111 1110
25.	**Резерв**			mask2H.b2 1111 1101
26.	**Макс. скорость**	Превышение уставки защиты по скорости	**Максимальная скорость**	mask2H.b3 1111 1011

Все уставки защит можно настроить в группе **Защиты**. При возникновении любой незамаскированной аварии инвертор отключается (время реакции не более 100мкс). Каждая авария может быть замаскирована. При её маскировании ПО контроллера не обрабатывает событие возникновения аварии и преобразователь продолжит работу (если это возможно…). Для маскирования служат параметры с **Защиты.Маска аварий 1L** по **Защиты. Маска аварий 2H**. Установленный ноль в соответствующем бите маскирует аварию. Соответствие номера бита и маскируемой аварии см. в таблице (нумерация бит с единицы), также в таблицы приведены примеры масок.

Модуль реализован в файле SM_Protect.c в виде модуля с названием TSM_Protect.

5.13. Модуль обработки команд управления SM_CmdLogic

Задача данного дискретного автомата контролировать все управляющие (команды оператора «Старт/Стоп») воздействия и, с учетом запрещающих сигналов (например, сигнала аварии от дискретного автомата защит), вырабатывать управляющее воздействие для дискретного автомата SM_Ctrl, реализующим требуемую структуру системы управления и управляющим силовым оборудованием (ключами инвертора).

Дискретный автомат SM_CmdLogic реализует всего два состояния, которые интерпретируют два состояния преобразователя – «Работа» и «Останов».

Переход из состояния «Останов» в состояние «Работа» осуществляется при **одновременном** выполнении следующих условий:

- контроллер готов к работе (все процедуры инициализации завершены);
- дискретный автомат защит SM_Protect функционирует и не находится в состоянии авария;
- пришла команда оператора «Старт» (от кнопки, или через сеть CAN).

Переход из состояния «Работа» в состояние «Останов» осуществляется при выполнении **любого** из условий:

- дискретный автомат защит SM_Protect находится в состоянии авария (возникла какая-то авария и необходимо срочно выключаться);
- дискретный автомат защит SM_ Ctrl находится в состоянии «Стоп» (инвертор выключен по какой то причине, соответственно и состояние преобразователя должно быть – «Останов»);
- пришла команда оператора «Стоп» (от кнопки, или через сеть CAN).

5.14. Библиотека функций целочисленной математики (IQmath)

Библиотека реализует быстрые целочисленные функции умножения и деления, различные тригонометрические функции в формате IQ24 (8 разрядов целой части и 24 разряда дробной части). Также реализовано несколько функцией в других форматах данных, а также функции преобразования форматов данных. Библиотека реализована в файлах V_IQmath.c и V_IQmath.h, причем в заголовочном файле присутствуют не только объявления функций, а также их реализация в виде inline функций и макроопределений. В текущей версии библиотеки реализованы следующие функции:

Название	Такты	Точность	Описание
_IQ24mpy(арг_1, арг_2)	8	31	Выполняет умножение аргументов в формате IQ24 друг на друга, возвращает результат в формате IQ24.
_IQ24div(арг_1, арг_2)	32	23	Выполняет деление аргумента_1 на аргумент_2 в формате IQ24, возвращает результат в формате IQ24.
_IQ24sqrt (аргумент)	43	23	Вычисляет квадратный корень. Входные и выходные данные в формате IQ24.

_IQ24sinPU(аргумент) _IQ24cosPU(аргумент)	30	10	Выполняет вычисление синуса/косинуса. Аргумент передается в относительных единицах IQ24, где 1.0 соответствует 360 градусов.
_IQ24sinPU_accurate (аргумент) _IQ24cosPU_accurate (аргумент)	83	14	Выполняет вычисление синуса/косинуса. Аргумент передается в относительных единицах IQ24, где 1.0 соответствует 360 градусов.
_IQ24atan2PU(арг_1, арг_2)	100	8	Функция арктангенса двух аргументов. Вычисляет угловое значение вектора в декартовой системе координат. Результат представлен в относительных единицах, где 1.0 соответствует 360 градусов.
_IQ24mag(арг_1, арг_2).	60	23	Функция вычисления амплитуды вектора, заданного аргументов 1 и 2.
_IQ24toF(аргумент_1)	2	23	Функция преобразования из формата IQ24 в формат с плавающей точкой.
_IQXX(аргумент)	2	-	Преобразование данных в формате с плавающей точкой в целочисленные форматы от IQ1 до IQ30. Реализовано в виде макроопределений формата _IQXX(число), где XX это число знаков после запятой целочисленного формата. Например, My24Var=_IQ24(0.75). Точность зависит от целевого и исходного форматов.

Количество фактически затраченных тактов может отличаться в зависимости от используемого компилятора и настроек оптимизации. Приведенные результаты получены для компилятора GCC с уровнем оптимизации – O2. Точность отображает количество достоверных значимых бит результата.

Также библиотека содержит некоторые производные из приведенных выше функций, например, с другим форматом входных или выходных данных. Функции добавляются в зависимости от нужд системы управления.

5.15. Модуль математических моделей электродвигателей (MotorModel)

Описание математических моделей доступно в отдельном документе «Описание имитационных математических моделей электродвигателей.pdf».

5.16. Модуль работы с пользовательской флеш памятью (UserMemory)

Модуль реализован в файле V_UserMemory.c и обеспечивает побайтовый кеширующий доступ к пользовательской флеш памяти, встроенной в микроконтроллеры серии 1921ВК01. Размер одной страницы памяти равен 256 байт, этому же равен размер кеша в оперативной памяти. Пока пользователь работает с одной и той же страницей памяти (читает и записывает), все изменения происходят в оперативной памяти. Если же адрес выходит за размеры текущей страницы, все изменения записываются на флеш, а в кеш прочитывается другая страница, соответствующая указанному адресу. Также все изменения с кешированными данными автоматически записываются на флеш по таймауту 0.5 секунды.

Драйвер предоставляет следующий интерфейс пользователя. Пользователь устанавливает указатель модуля «MCUStartAddr» на данные в оперативной памяти контроллера, которыми необходимо взаимодействовать. В переменной «MemStartAddr» пользователь устанавливает адрес (номер ячейки в байтах) пользовательской памяти, с которой необходимо работать. В переменной data_length указывается длина данных в байтах.

Далее пользователь вызывает метод write или read модуля для осуществления записи данных на флеш или чтения из нее соответственно. Функция «медленная» – вернет управление только после того, как данные будут действительно записаны или прочитаны (возможно, данные частично будут записаны в кеш и уже в теневой операции сброшены на флеш). Также имеется метод «flush» для принудительной записи кеша на «флеш» память. Помимо перечисленного для корректной работы модуля необходимо вызывать служебные методы «ms_calc» и «slow_calc».

5.17. Модуль часов реального времени

Модуль реализован в файле V_RTC_Clock.c в виде модуля TRTCClock. Представляет из себя интерфейс между часами реального времени микроконтроллера и ПО UniCON. Запаковывает текущее время в переменную packed_time, доступную в параметре **Текущ. знач. Часы реального времени**. В распакованном виде время доступно на панели UniCON справа сверху. Для установки времени используется функция RTC_Clock_Set_Time, которая вызывается через функцию обратного вызова драйвера CANopen (в файле CANopenUDfuncs.c) при записи в параметр **Текущ. знач. Часы реального времени**.

5.18. Модуль работы с драйвером ключей DRV8301

Модуль необходим для совместимости ПО с отладочным комплектом управления двигателем на основе VectorCARD.

Модуль реализован в файле DRV8301_SPI.c в виде модуля Tdrv8301interface. Служит для конфигурирования микросхемы драйвера ключей DRV8301 через интерфейс SPI (для отладочного комплекта на основе DRV8301). Настраивает, по скольким каналам управляются ключи, как работает реакция на аварию, коэффициент усиления внутренних операционных усилителей для шунтов и т.п.

5.19. Модуль драйвера CANopen

Описание драйвера CANopen доступно в отдельном документе «Описание драйвера CANopen.pdf».

5.20. Модуль работы с драйвером CANopen через интерфейсы UART, USB (CANtoRS)

Модуль обеспечивает возможность работы с сервисом SDO драйвера CANopen через последовательный интерфейс UART или интерфейс USB. Посылки сети CAN «запаковываются» как посылки последовательного интерфейса, снабжаются собственной контрольной суммой и отправляются по последовательному интерфейсу. Драйвер в первую очередь предназначен для работы с программой UniCON через последовательный интерфейс вместо CAN интерфейса, используя функционал драйвера CANopen. Так как используется только сервис SDO, существует ряд ограничений: загрузка осциллограмм модуля цифрового осциллографа (п. 5.11) происходит не средствами буферной передачи, а по одной точке через SDO сервис, что приводит к гораздо более медленной загрузке осциллограмм. Кроме того, в сети может быть только одно ведомое устройство (один микроконтроллер) и один мастер (программа UniCON). Соответственно, никакого межконтроллерного распределенного взаимодействия при использовании данного способа связи быть не может.

Для взаимодействия по сервису SDO драйвер CANtoRS пользуется вспомогательными функциями драйвера CANopen: функцией co_CANToExtInterface_Send, реализованной в файле CANopenUDfuncs.c, а также функцией обратного вызова Z_co_receiveSDOrequest. Драйвер CANtoRS самостоятельно эмулирует статусные посылки сервиса heartbeat CANopen, необходимые для отображения устройств в сети в программе UniCON.

Драйвер может быть сконфигурирован для работы с любым доступным модулем UART, конфигурирование выполняется в начале функции CANtoRS_init. Скорость обмена данными фиксирована и должна составлять 57600 бит/с.

5.21. Модуль драйвера MODBUS

Описание драйвера CANopen доступно в отдельном документе «Описание драйвера MODBUS.pdf».

5.22. Отображение текущих параметров привода (модуль CurPar)

Модуль CurPar, реализованный в файлах V_CurPar.c и V_CurPar.h осуществляет сбор и расчет текущих параметров привода. Значения доступны в группе **Наблюдаемые**. В данном пункте описано, как происходит вычисление этих величин.

- **Скорость** – текущая частота вращения двигателя в оборотах в минуту. Вычисляется по датчику положения ротора (в зависимости от выбранного типа ДПР). Для скалярной и частотно-токовой системы управления отображается заданная скорость после ЗИ ($f_{вых}$ на Рис. 5.1).
- **Акт. мощность –** текущая потребляемая двигателем активная мощность. Вычисляется исходя из вектора напряжения, заданного на модуль ШИМ и показаний датчиков токов фаз.
- **Ток статора –** амплитудное значение тока статора двигателя. Вычисляется при помощи координатных и фазных преобразований и равно току по оси q, т.к. по оси d ток задается нулевым.
- **ЭлУголЗад –** заданный угол положения ротора. Вычисляется при расчете системы управления.

5.23. Различные важные параметры, не относящиеся к конкретному модулю

5.23.1. Настройки системы

В группе **Настройки СУ** собраны различные настройки, не вошедшие в отдельную тематическую группу параметров.

Настройки СУ.РежРаб [5114h.02h] – выбор структуры управления двигателем, подробнее описано в п.5.1.

Настройки СУ.Рекуперация [5114h.03h] – разрешенные рекуперации. Влияет на нижнее ограничение регулятора скорости. Подробнее описано в п. 5.1.4.

Настройки СУ.АЦП авто смещ. [5114h.04h] – разрешение работы модуля автоматической подстройки смещения каналов датчиков тока АЦП.

Настройки СУ.Перезагрузка [5114h.05h] – установка единицы в этот параметр выполняет перезагрузку ПО микроконтроллера.

Настройки СУ.Возбудитель [5114h.06h] – осуществляет разрешение управления обмоткой возбуждения двигателя.

5.23.2. Переменные для отладки

В словарь объектов выведены несколько служебных отладочных переменных, в которые можно внутри ПО микроконтроллера присваивать любые данные и наблюдать их затем в ПО UniCON. Эти переменные доступны в группе параметров «Для отладки», а внутри ПО представлены как Debug1- Debug4 (доступны в словаре только на чтение), DebugW1- DebugW4 доступны и для записи, а DebugF1-DebugF4 имеют формат плавающей точки. Стоит отметить, что модуль цифрового осциллографа data_log 16ти разрядный. И в режиме работы осциллографа UniCON «Выборка», когда данные записываются во внутренний массив контроллера, в каждую точку записываются только 16 разрядов. Если переменная 32-х разрядная, то записывается содержимое её верхней части (для формата IQ8.24 приемлемо). Поэтому для просмотра 16ти разрядных переменных (таких как счетчики, состояние ножек и т.п.) переменные Debug3, Debug4 сделаны 16ти разрядными.

5.23.3. Счетчики

В группе параметров **Счетчики** содержатся параметры со счетчиков прерываний. По их инкрементированию можно судить о работоспособности прерываний и фонового цикла.

Также в группе параметров **Счетчики** присутствуют параметры, отображающие длительность расчета программного кода в каждом из прерываний в тактах микроконтроллера. T_10кГц и T_1кГц показывают время расчета в тактах прерывания 10кГц и 1кГц соответственно. В ПО микроконтроллера это время высчитывается таймером общего назначения NT_TIMER2. По показаниям данных счетчиков удобно оценивать процент загрузки ядра микроконтроллера.

6. Лабораторные работы

6.1. Синхронный двигатель

6.1.1. Скалярное управление

Перед работой с реальным оборудованием рекомендуется сначала провести виртуальную лабораторную работу с имитационной моделью синхронного электродвигателя и только затем повторять шаги на реальном оборудовании.

Для выбора имитационной модели синхронного двигателя необходимо скомпилировать проект и запрограммировать контроллер для использования имитационных моделей (тип аппаратной части выбирается в файле build.h). Далее в параметре **Модель общие данные.Тип двигателя модели [5181,01]** выбрать «Синхронный», а в параметре **Модель общие данные.Параметры двигателя [5181,02]** выбрать набор параметров электродвигателя, наиболее близкий к желаемому. Наборы параметров можно посмотреть в файле V_motorParams_SM.h. Также можно задать набор параметров равным нулю и задать свои собственные параметры двигателя в группе **Модель (синхронный)**. После завершения настройки нужно дать команду сохранения параметров кнопкой «Сохранить параметры» справа сверху программы UniCON, а затем выполнить перезагрузку микроконтроллера для применения выбранных настроек (**Настройки СУ.Перезагрузка [5114,05]).**

Запуск скалярного управления, как самого простого, вместе с настройкой защит и модуля АЦП подробно рассмотрен в главе 4.

Отличие может заключаться лишь в том, что если выбрана модель синхронного электродвигателя с обмоткой возбуждения (ненулевое значение параметра номинального тока ОВ **Модель (синхронный).Номинальный ток ОВ [5183h.0Eh]**), то необходимо также включить возбуждение. Эта обмотка управляется самым простым способом путем задания напряжения. Необходимо разрешить управление ОВ установкой единицы в параметр **Настройки СУ.Возбудитель [5114h.06h],** а также задать желаемое задание напряжения возбуждения в параметре **Задания.Заданное напряжение ОВ [5112h.05h]**. Желательно установить такое напряжение, чтобы ток возбуждения был номинальным. Текущее значение тока ОВ можно посмотреть в **АЦП.Ток фазы D [5116h.0Ch]**, или, если АЦП настроено неверно, в **Модель (синхронный).Ток ОВ [5183h.10h]**.

Следует отметить, что для синхронной машины зачастую бывает очень сложно раскрутить её в скалярном управлении на достаточно высокую скорость, что связано с колебательностью машины, а также со сложностью настройки кривой u/f. Но этого и не требуется, так как есть более подходящие структуры управления. Для проверки базовой работоспособности достаточно запустить двигатель хотя бы с частотой 1-10 об/мин.

6.1.2. Настройка регуляторов тока

Регуляторы тока необходимы для замкнутых по току структур управления, включая векторное управление. Для их настройки необходимо наблюдать на осциллограмме реакцию на скачок задания тока при остановленном двигателе. Для продолжения работы по этому пункту необходимо уже иметь корректно настроенные датчики токов и остальное, т.е. успешно запустить скалярное управление согласно главе 4.

Для этого необходимо включить замкнутую по току структуру управления: режим удержания – в **Настройки СУ.Режим работы [5114,02]** ввести значение «2». Подробнее структура описана в п.п. 5.1.3. В осциллограф UniCON добавим мгновенные токи фаз из группы **АЦП** (через контекстное меню параметра). Также может быть интересно наблюдать заданное напряжение (**ШИМ.UaЗад [5105,01]**).

Для правильного отображения процесса пуска в параметр **dlog.StartBitL** вводим значение «00000010», в **dlog.StartBitH** – «00000000», в параметре **dlog.Количество точек предыстории [5150,0A]** ставим 0%, чтобы начало осциллограммы соответствовало времени скачка задания. Параметры **dlog.StartBitL** и **dlog.StartBitH** задают триггеры срабатывания осциллографа, в данном случае по событию перехода машины состояний модуля sm_ctrl в состояние 2 (структура

постоянного тока). Подробнее о работе модуля осциллографа читайте в руководстве пользователя к программе UniCON, а также в главе 5.11.

Настраивать регуляторы токов для реальной эксплуатации привода желательно с уровнем тока, близким к номинальному. Однако для начала можно задать и меньшей уровень тока. Задание тока настраивается в параметре **Задания.Ток удержания [5112,03].**

Нажимаем кнопку «Пуск» на панели управления в правой части окна UniCON, заходим во вкладку «Осциллограф» и нажимаем кнопку «Обновить» (Рис. 6.1).

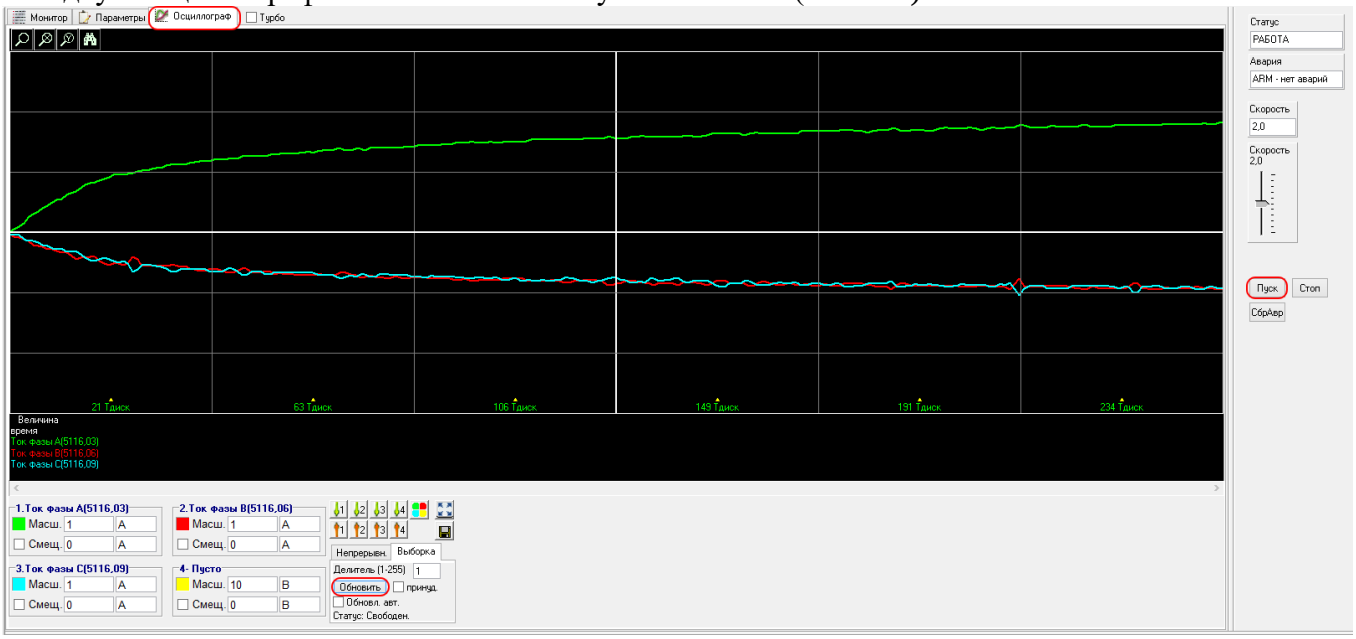


Рис. 6.1 Окно осциллографа

Если при нажатии кнопки «Обновить» появляется окно Рис. 6.2, значит по одному из активных триггеров произошел захват осциллограммы и её можно загрузить. Однако если это первый запуск осциллографа после изменения параметров осциллографирования (набора каналов, делителя), то необходимо отказаться от загрузки осциллограммы, так как там «мусор».

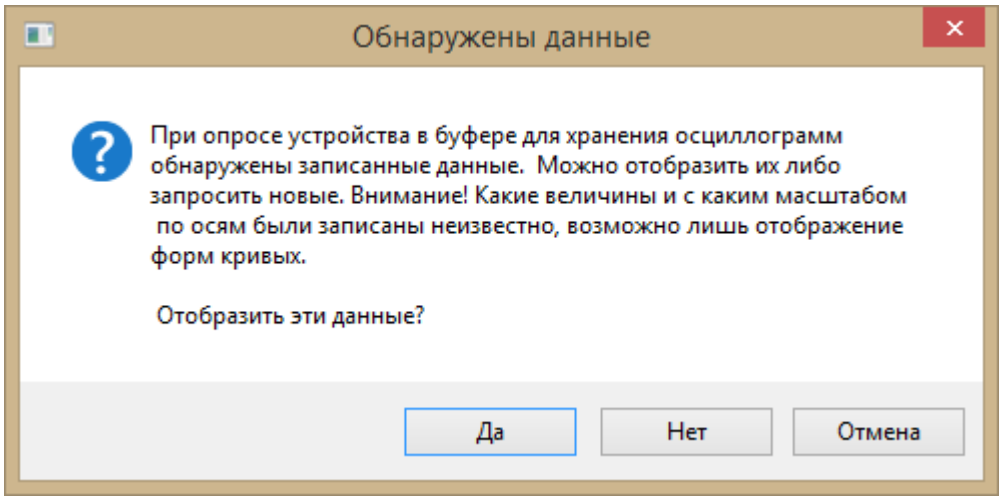


Рис. 6.2 Диалоговое окно загрузки захваченных осциллограмм

Для отображения осциллограммы запуска в появившемся окне нужно щелкнуть по кнопке «Да». Переходной процесс при ненастроенных регуляторах может выглядеть подобно Рис. 6.3.

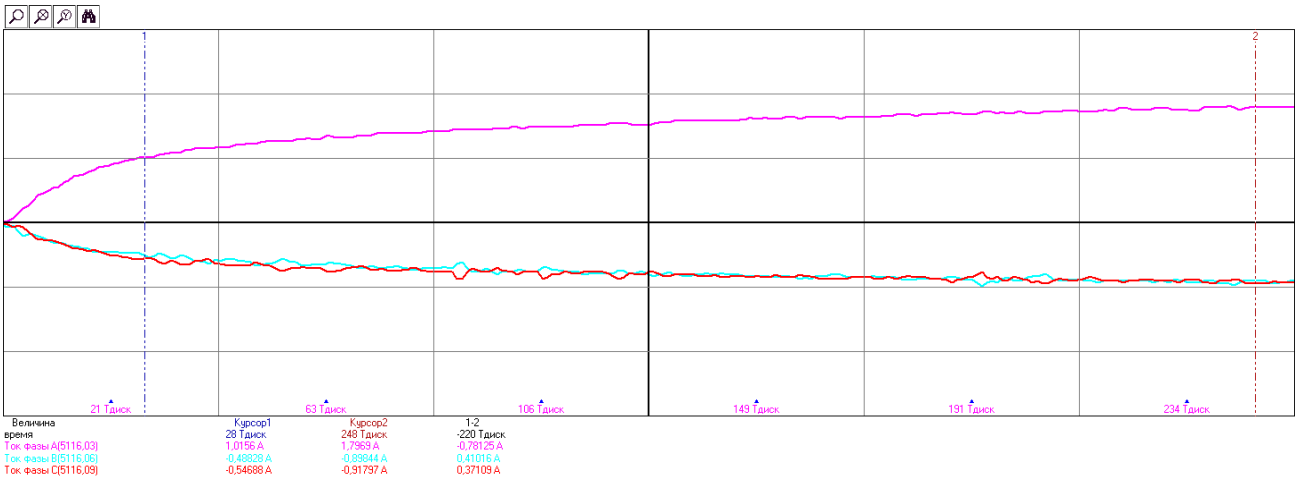


Рис. 6.3 Переходной процесс пуска при ненастроенных регуляторах

Чтобы видеть процесс с максимальным разрешением по времени, рекомендуется установить делитель на вкладке «выборка» равным 1. Для его применения нужно нажать кнопку «Обновить».

Если сработала какая-либо защита, то необходимо исследовать проблему. В этом должна также помочь осциллограмма переходного процесса. Наиболее вероятно срабатывание защиты по превышению фазного тока. Если это так, необходимо убедиться, что задание тока меньше уставки защиты, а также попробовать дополнительно уменьшить значение пропорционального коэффициента регуляторов. При запуске системы управления на новом аппаратном обеспечении зачастую могут быть перепутаны знаки или порядок датчиков токов фаз, а также выходы управления ШИМ инвертора. В этом случае переходной процесс на осциллограмме будет выглядеть экспоненциально расходящимся: необходимо вернуться к скалярному режиму работы с сверить показания измерения токов фаз с реальными данными внешнего прибора, а также с заданным напряжением на модуле ШИМ (на низкой частоте вращения напряжению **ШИМ.UaЗад [5105,01]** должен соответствовать **АЦП.Ток фазы A [5116,03]**).

Процесс подбора коэффициентов регуляторов тока рассмотрен на рисунках ниже. Сначала интегральный коэффициент обнуляется (параметры **Рег. тока Id.Интегральный коэфф. Ki [5100,06]** и **Рег. тока Iq.Интегральный коэфф. Ki [5102,06]**), а пропорциональный (**параметры Рег. тока Id.Пропорц. коэфф. Kp [5100,03]** и **Рег. тока Iq.Пропорц. коэфф. Kp [5102,03]**) постепенно повышается до появления колебательности переходного процесса. С каждым изменением коэффициенты нужно подавать команду останов и пускать привод снова, чтобы загрузить осциллограмму и оценить вид переходного процесса. Коэффициенты для обоих регуляторов тока (Iq, Id) должны быть равны и изменяются одновременно. На Рис. 6.4 показан переходной процесс регулятора тока на скачок задания с перерегулированием. Как видно, в контуре тока есть статическая ошибка (задание равно 2А, а фактическое значение меньше – около 1А). Устранить её поможет интегральная часть регулятора. После получения похожего переходного процесса с несколькими колебаниями пропорциональный коэффициент нужно уменьшить и перейти к повышению интегрального коэффициента.

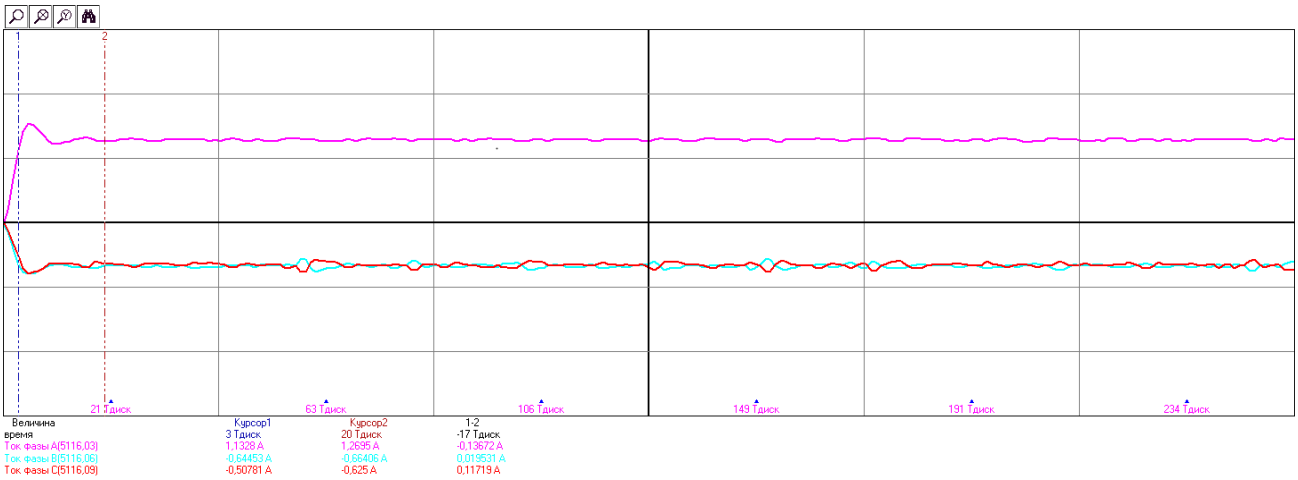


Рис. 6.4 Переходной процесс регулятора тока на скачок задания с перерегулированием

При настройке желательно следить, чтобы заданное на инвертор напряжение не достигало максимального ограничения, что можно видеть по выставлению флага **ШИМ.ULimitation [5105,0D]**, который удобно также добавить на осциллограмму. Если во время переходного процесса флаг выставляется, необходимо снизить задание тока.

По окончанию грубой настройки необходимо произвести финальный подбор обоих коэффициентов регуляторов, чтобы получить переходной процесс, похожий на настройку на технический оптимум. Желаемый переходной процесс показан на Рис. 6.5.

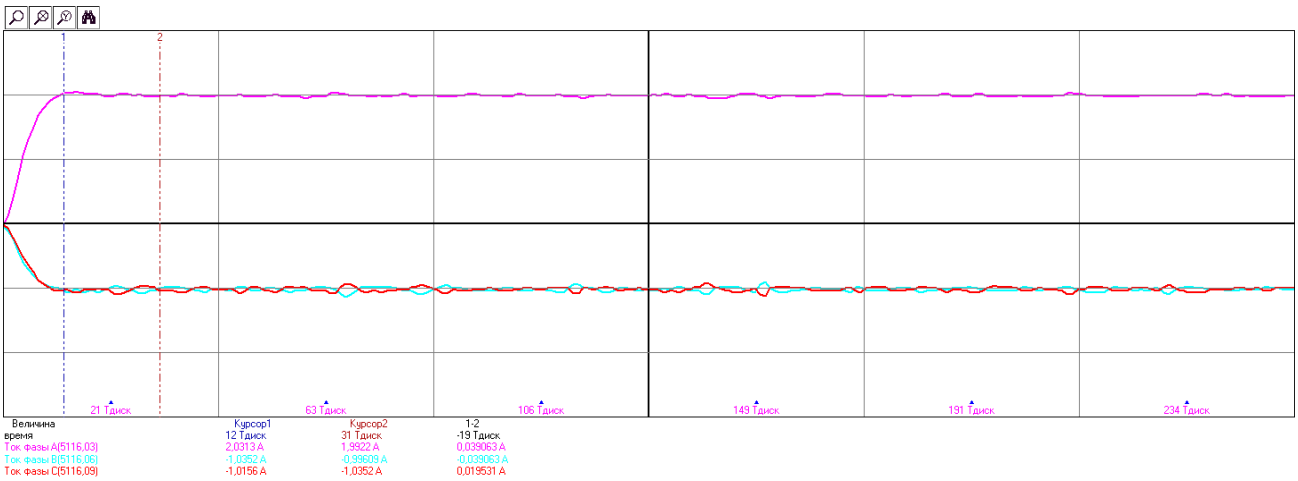


Рис. 6.5. Настройка регуляторов тока на технический оптимум

При получении аналогичной осциллограммы регуляторы тока можно считать настроенными. Можно сохранить текущие настройки в энергонезависимую память, нажав кнопку «Сохранить параметры» в UniCON. Также в данном режиме можно дополнительно откалибровать показания датчиков тока, так как постоянный ток легче измерить и сравнить с заданием, чем переменный.

6.1.3. Частотно-токовое управление

Частотно-токовое управление является наиболее простым способом привести во вращение синхронную машину. Подробнее структура данного способа управления рассмотрена в п. 5.1.2. Для запуска структуры управления необходимо провести успешный запуск скалярной системы управления и настроить регуляторы токов (п.п. 6.1.1 и п.п. 6.1.2). Только после этого можно следовать далее.

Для выбора частотно-токовой структуры управления необходимо установить **Настройки СУ.Режим работы [5114,02]** = «4». Установить желаемое задание амплитуды тока **Задания.Макс. ток ВУ [5112,02]**, желаемую частоту вращения **Задания.Скорость [5112,01]**. Начать

рекомендуется с небольшого тока, порядка 10% номинала и небольшой частоты вращения, порядка 1-10 об/мин. В осциллограф рекомендуется добавить три тока фаз и напряжение ЗПТ из группы АЦП.

Подать команду Пуск на панели в UniCON. Двигатель должен вращаться со скоростью (частотой вращения), заданной в **Задания.Скорость [5112,01]**, форма токов – синусоидальная (Рис. 6.6).

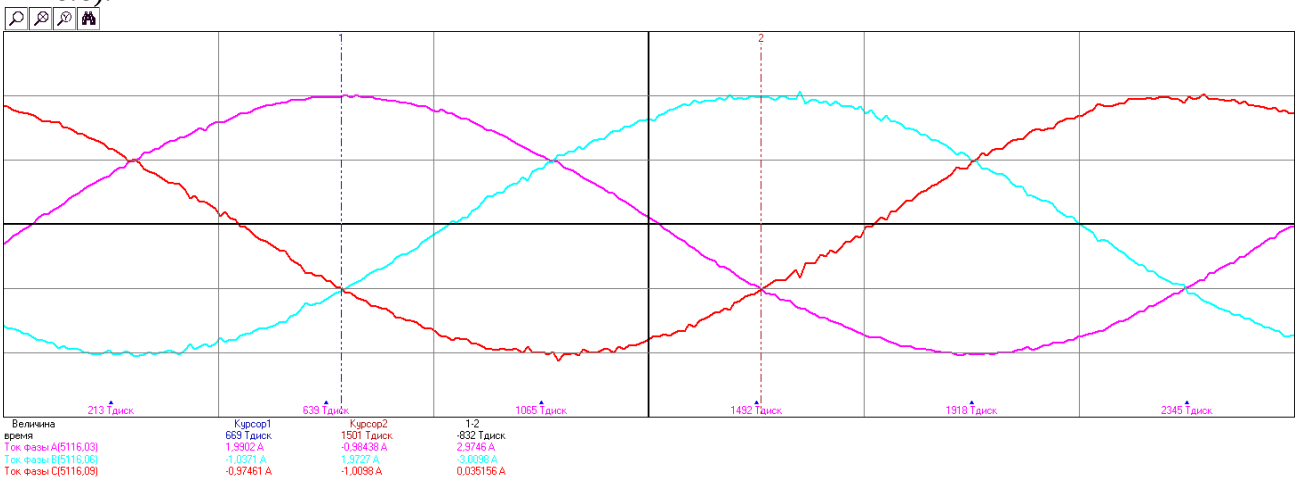


Рис. 6.6. Форма токов фаз при работе частотно-токовой структуры управления

При низкой частоте вращения осциллограмму токов можно наблюдать как в непрерывном режиме осциллографа (рекомендуется активировать опцию «Турбо» рядом с вкладкой осциллографа для более быстрого обновления графиков), так и в режиме «выборка» с большим делителем. С повышением частоты вращения для наблюдения формы токов фаз необходимо пользоваться только режимом «выборка», так как непрерывный режим реального времени не будет успевать обновлять данные с нужной дискретностью по времени.

Вал двигатель должен вращаться. Для математической модели можно открыть справочные статусные параметры двигателя и посмотреть частоту вращения в **Модель общие данные.Скорость [5181,06]** или добавить угловое положение ротора **Модель общие данные.Механический угол [5181,08]** в осциллограф. Если двигатель не крутится или крутится нестабильно, хотя токи синусоидальны и равны заданию, можно попробовать увеличить задание амплитуды тока, а также снизить задание частоты вращения.

Если двигатель крутится стабильно, то для повышения частоты вращения рекомендуется предварительно настроить темп разгона (задатчик интенсивности) в параметре ЗИ.Т разгона до номинала [5000,01]. Подробнее о ЗИ можно прочитать в п.п. 5.4.. Чем медленнее разгон, тем меньше вероятность выпадения двигателя из синхронизма во время разгона. Далее можно повысить задание частоты вращения в переменной **Задания.Скорость [5112,01]**. При этом есть риск «вываливания двигателя из синхронизма» – ротор может не успеть следовать за полем статора, магнитная пружина «разорвется», после чего ротор двигателя остановится и будет качаться в одном положении. Если это произошло, нужно увеличить задание на ток статора, снизить задание частоты вращения или замедлить темп разгона.

С ростом частоты вращения необходимо следить, что напряжение на инверторе имеет запас: флаг ограничения выходного напряжения ШИМ.ULimitation [5105,0D] не выставляется в единицу.

Если при разгоне привода срабатывает какая-либо защита, необходимо изучить её причины. Наиболее вероятно, что могут сработать защиты по превышению тока статора или повышения напряжения ЗПТ. Проще всего исследовать проблему путем наблюдения осциллограммы события аварии. Для этого необходимо добавить интересующие переменные в осциллограф (например, токи фаз и напряжение ЗПТ), поставить параметр **dlog.Количество точек предыстории [5150,0A]** = 70%, чтобы видеть предысторию развития аварии, сконфигурировать флаг срабатывания триггера на событие останова привода (**dlog.StartBitL [5150,07]= 00000000**, **dlog.StartBitH [5150,08]= 00000001**), открыть режим «выборка», настроить желаемый делитель и

нажать кнопку «обновить» с отказом от загрузки уже записанной осциллограммы (если такой запрос будет). Теперь осциллограф готов к захлопыванию осциллограммы по аварии: необходимо воспроизвести аварийное отключение, после чего нажать кнопку «обновить» и согласиться с загрузкой записанной по событию осциллограммы. Например, на Рис. 6.7 показана форма токов фаз при срабатывании максимально-токовой защиты при разгоне привода в частотно-токовом управлении. Причина такой аварии – раскачка машины с данным типом управления при работе на высоких частотах вращения (малая демпфируемость колебаний).

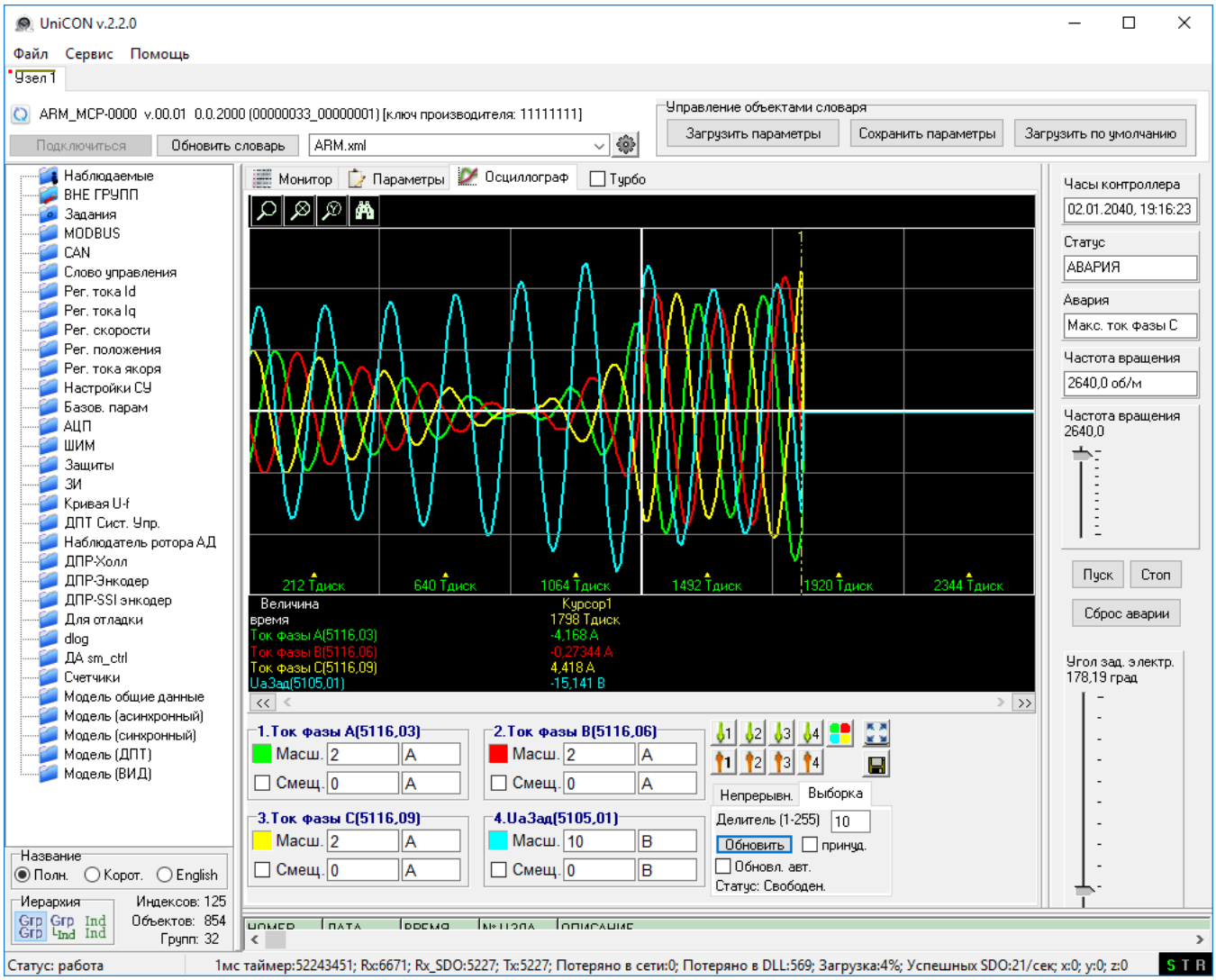


Рис. 6.7. Форма токов фаз при срабатывании максимально-токовой защиты при разгоне привода в частотно-токовом управлении

На данном этапе лабораторная работа с частотно-токовым управлением закончена, читатель может дальше самостоятельно проводить исследования работы привода в этом режиме управления.

6.1.4. Проверка и настройка ДПР

Перед тем как запустить векторную СУ, нужно проверить работу датчиков положения ротора. Необходимо выбрать используемый тип датчика положения в параметре **Базов. парам.Тип датчика положения [5113,09]** (инкрементальный энкодер или на датчиках Холла). В случае инкрементального энкодера необходимо настроить число меток в параметре **ДПР-Энкодер.Кол-во периодов [5152,01]**, а также задать наличие индексной метки в параметре **ДПР-Энкодер.Наличие репера [5152,02]**. Параметры реального датчика положения можно в его документации. Следует отметить, что существует некоторая путаница между количеством периодов датчика и его количеством импульсов. При количестве периодов датчика, например,

1000, суммарное количество импульсов по обоим каналам на обороте составит 4000. В параметр **ДПР-Энкодер.Кол-во периодов [5152,01]** вводится именно число до учетверения. После выбора и настройки датчика необходимо подать команду сохранения параметров и перезагрузить микроконтроллер, сбросив питание или вписав «1» в параметр **Настройки СУ.Перезагрузка [5114,05]**.

Далее необходимо следует запустить двигатель в частотно-токовой системе управления (режим работы 4, см. п.п. 6.1.2), задать небольшую скорость, к примеру, 2 об/мин и наблюдать осциллограммы сигналов токов и сигналов датчиков положения: **ДПР-Холл.ЭлУгол [5155,01]** (для датчика на элементах Холла) или **ДПР-Энкодер.Угол. электр. [5152,08]** (угловое положение с квадратурного энкодера в электрических градусах).

Корректный сигнал с ДПР для датчика на элементах Холла представлен на Рис. 6.8, а для инкрементального энкодера – на Рис. 6.9.

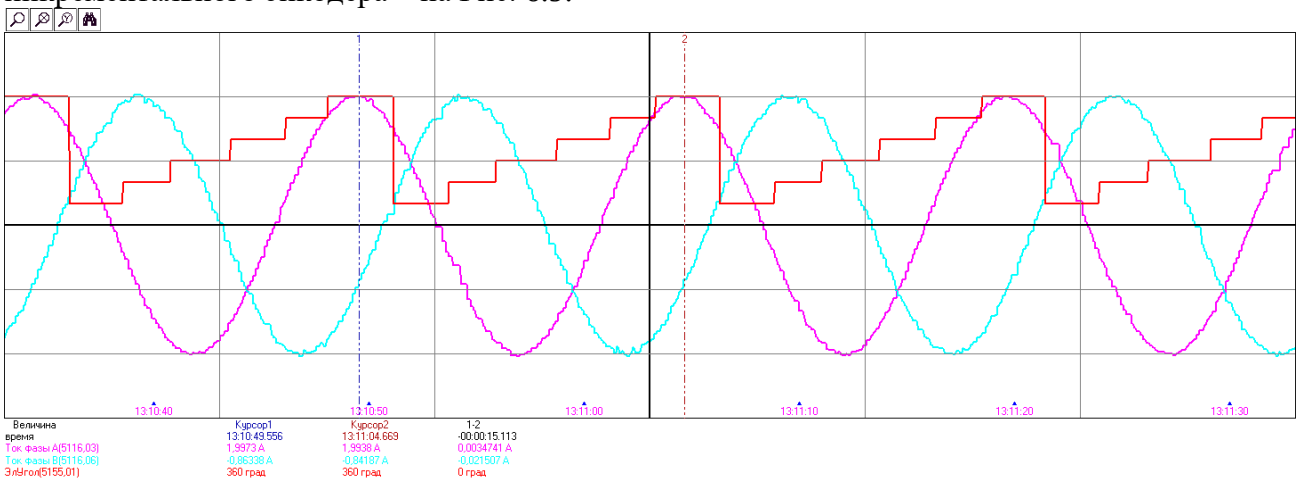


Рис. 6.8 Осциллограмма сигнала ДПР на элементах Холла

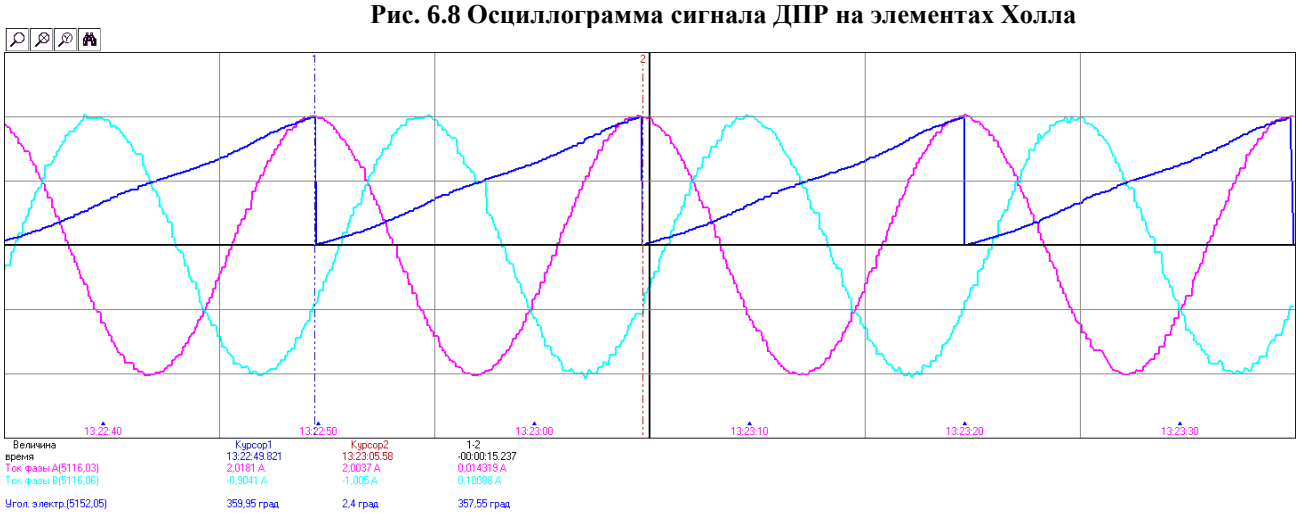


Рис. 6.9 Осциллограмма сигнала инкрементального энкодера

Сигнал с ДПР любого типа должен иметь равномерно возрастающий характер, его период должен совпадать с периодом тока в фазах, а обнуление углового сигнала ДПР должно происходить в районе максимума тока в фазе А. В этом случае датчик считается настроенным.

Если сигнал ДПР отсутствует или «совсем не похож» на представленные или отсутствует, проверьте правильность подключения проводов ДПР к разъемам платы контроллера. Полезным может быть наблюдение параметра, показывающего двоичные значения входов контроллера, на которые заведены каналы датчика. Они отображены в параметрах **ДПР-Энкодер.GPIOsValue [5152h.0Dh]** и **ДПР-Холл.Дискретные входы [5155h.09h]**.

Если характер сигнала ДПР спадающий, а не нарастающий, поменяйте местами любые две фазы питания двигателя или датчика положения. Если период сигнала ДПР отличается от периода тока, проверьте **параметр Базов. парам.Число пар полюсов**: он настроен неправильно (не соответствует двигателю). Если период ДПР больше, чем тока, число пар полюсов следует увеличить, в противном случае – уменьшить. После изменения числа пар полюсов нужно

сохранить параметры (кнопка «Сохранить параметры») и перезагрузить микроконтроллер. В случае инкрементального энкодера необходимо также проверить корректность настроенного числа меток. Если обнуление сигнала ДПР не совпадает по времени с максимумом тока в фазе А (так и должно быть в общем случае), настройте смещение датчика положения параметрами **ДПР-Холл.СмещУгол** или **ДПР-Энкодер. СмещУгол** так, чтобы эти два события происходили одновременно. Для инкрементального энкодера такую настройку смещения необходимо делать каждый раз при включении в работу, так как процедура «причаливания» и сигнал индексной метки в данном ПО не используются соответствующим образом. Для ДПР на элементах Холла настройку можно провести один раз для данного двигателя и сохранить параметры в энергонезависимую пользовательскую память устройства.

Настройка датчиков положения «вручную» полезна как обучающий пример. Чтобы экономить время при каждом запуске, можно запускать автоматическую настройку. Она реализована в виде отдельного режима работы. Для запуска автоматической настройки смещений обоих ДПР необходимо задать **Настройки СУ.Режим работы [5114,02]** = 29 и запустить привод без нагрузки на частоте вращения 2-5 об/мин. Структура системы управления в режиме 29 аналогична частотно-токовой структуре управления (режим 4), однако в этом состоянии добавлена логика автоматической подстройки смещений обоих типов датчиков относительно текущего углового положения вектора тока. Необходимо запустить этот режим работы и подождать около минуты, смещения датчиков настроятся автоматически. Для визуализации процесса автонастройки можно добавить параметры смещений датчиков в осциллограф (параметры **ДПР-Холл.СмещУгол** и **ДПР-Энкодер. СмещУгол**).

На данном этапе настройка датчиков положения закончена.

6.1.5. Векторное двухконтурное управление

Для запуска двигателя в векторной системе управления необходимо предварительно настроить регуляторы токов (п.п. 6.1.2) и датчик положения ротора (п.п. 6.1.4). Далее для выбора векторной двухконтурной структуры управления (п.п. 5.1.4) необходимо установить **Настройки СУ.режим работы** = 5.

Векторная СУ с плохо настроенным регулятором скорости или грубым датчиком положения (на элементах Холла или инкрементальным энкодером с малым числом меток) обычно неустойчиво работает на низких скоростях (менее 100-500 об/мин в зависимости от двигателя), поэтому задавать следует бóльшие значения задания скорости в параметре **Задания.Скорость [5112,01]**. Также необходимо установить желаемое токоограничение амплитуды тока в параметре **Задания.Макс. ток ВУ [5112,02]**.

Настройки регулятора скорости в группе **Рег. Скорости** рекомендуется для первого пуска задать мягкими: Кп порядка 0.05, Ки 0.01. В осциллограф удобно добавить параметры из группы **Наблюдаемые**: скорость, амплитудный ток, мощность и оставить работать в непрерывном режиме. Далее можно произвести пуск векторной системы управления.

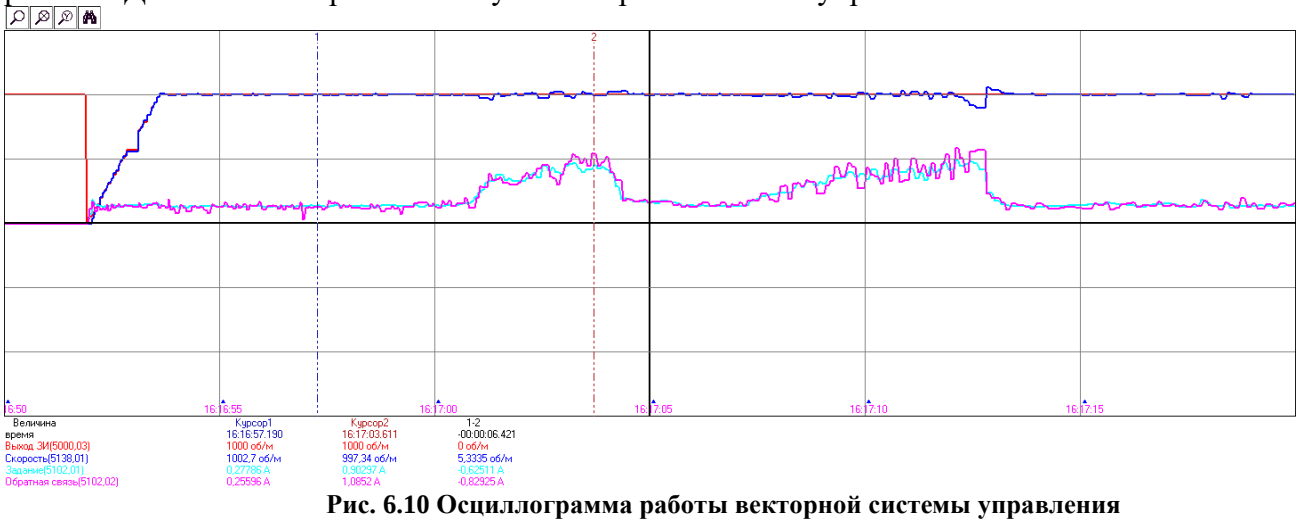


Рис. 6.10 Осциллограмма работы векторной системы управления

На Рис. 6.10 показана работа векторной системы управления. Выход задатчика интенсивности является заданием скорости. Задание и обратная связь (голубой и розовый графики) – задание регулятора тока по оси q и фактическое значение тока по оси q. Показан пуск и приложение нагрузки к двигателю. Как видно из осциллограммы, при повышении момента нагрузки ток по оси q возрастает, чтобы поддержать задание скорости (частоты вращения).

Регулятор скорости векторной СУ, как и регуляторы тока, рекомендуется использовать пропорционально-интегральный. Его настройки так же можно изменять для получения требуемой динамики и точности регулирования скорости. Настройку можно проводить по методике, аналогичной настройке регуляторов тока, только наблюдать в осциллограмме уже частоту вращения и задание на ток. Для такой методики необходимо предварительно отключить задатчик интенсивности, обнулив **ЗИ.Т разгона до номинала [5000h.01h]**. При настройке следует следить, чтобы задание на ток не достигало токоограничения. Если оно достигает, следует либо увеличить токоограничение, либо снизить задание скорости.

Также при использовании инкрементального датчика положения и программного способа расчета скорости на поведение привода сильно влияет параметр фильтрации сигнала по скорости **ДПР-Энкодер.К быстродействия фильтра скорости [5152h.0Bh]**.

6.1.6. Позиционное управление

Для запуска структуры, замкнутой по положению (описана в п.п. 6.1.6) необходимо использовать инкрементальный датчик положения ротора. Необходимо пройти все шаги пусконаладочных работ до п.п. 6.1.5 включительно и качественно настроить регулятор скорости в этом пункте. Далее необходимо задать **Настройки СУ.режим работы** = 10, чтобы выбрать замкнутую по положению структуру управления, а затем обнулить параметр **ДПР-Энкодер.Число оборотов**: если он окажется не нулевой, то привод после запуска начнет «отматывать» обороты назад, стремясь к нулевому заданию положения (и нулевому счетчику оборотов). Параметры регулятора положения по умолчанию настроены на достаточно жесткое поддержание положения. Для первого запуска рекомендуется сделать регулятор более мягким, а интегральную составляющую отключить полностью. Первый раз рекомендуется запускать привод с небольшим разрешенным током (**Задания.Макс. ток ВУ**), а также ограничить частоту вращения **Задания.Скорость [5112,01]** на низком уровне порядка 50 об/мин. В параметре **Задания.theta_elec** пользователем задается желаемое положение в электрических градусах, к которому привод должен следовать значением параметра **ДПР-Энкодер.Угол электрический не огранич. [5152h.17h]**. Можно запустить привод и попробовать задать разные положения. На осциллографе в непрерывном режиме интересно смотреть задание углового положения, текущее угловое положение, текущий ток статора и скорость из группы **Наблюдаемые**.

Качественная настройка контура положения достаточно сложная процедура и выходит за рамки данного руководства. Пользователю предлагается изучить данный вопрос самостоятельно, экспериментируя с разными параметрами регулятора скорости и положения. Рекомендуется, как минимум, испытать две схемы с регуляторами: ПИ-регулятор скорости и П регулятор положения, а также П-регулятор скорости и ПИ-регулятор положения (в этом случае значение **Задания.Скорость [5112,01]** должно быть большим, чтобы пропорциональный регулятор скорости мог реализовать полное задание на ток). Регуляторы тока всегда должны быть ПИ.

После завершения настройки можно сохранить все параметры привода в энергонезависимую память. Это делается в программе UniCON путем нажатия на кнопку «Сохранить параметры», расположенной справа сверху. При этом значения всех параметров CANopen, доступных на запись, сохранятся в пользовательской флеш памяти микроконтроллера и будут в дальнейшем восстанавливаться при перезагрузке микроконтроллера в процедуре инициализации драйвера CANopen. Также можно сохранить все настройки привода в файл на компьютер, воспользовавшись меню Сервис-Перенос параметров (подробнее см. руководство пользователя на UniCON).

6.2. Асинхронный двигатель

6.2.1. Скалярное управление

Перед работой с реальным оборудованием рекомендуется сначала провести виртуальную лабораторную работу с имитационной моделью асинхронного электродвигателя и только затем повторять шаги на реальном оборудовании.

Для выбора имитационной модели асинхронного двигателя необходимо скомпилировать проект и запрограммировать контроллер для использования имитационных моделей (тип аппаратной части выбирается в файле build.h). Далее в параметре **Модель общие данные.Тип двигателя модели [5181,01]** выбрать «Асинхронный», а в параметре **Модель общие данные.Параметры двигателя [5181,02]** выбрать набор параметров электродвигателя, наиболее близкий к желаемому. Наборы параметров можно посмотреть в файле V_motorParams_IM.h. Также можно задать набор параметров равным нулю и задать свои собственные параметры двигателя в группе **Модель (асинхронный)**. После завершения настройки нужно дать команду сохранения параметров кнопкой «Сохранить параметры» справа сверху программы UniCON, а затем выполнить перезагрузку микроконтроллера для применения выбранных настроек (**Настройки СУ.Перезагрузка [5114,05]).**

Запуск скалярного управления, как самого простого, вместе с настройкой защит и модуля АЦП подробно рассмотрен в главе 4.

6.2.2. Настройка регуляторов тока

Настройка регуляторов тока асинхронной машины ничем не отличается от настройки регуляторов синхронной, поэтому необходимо обратиться к методике п.п. 6.1.2.

6.2.3. Частотно-токовое управление

Настройки частотно-токовой системы управления асинхронной машины похожа на аналогичную для синхронной машины, поэтому необходимо обратиться к методике п.п. 6.1.3. Принципиальное отличие заключается в том, что асинхронный двигатель не может «вывалиться из синхронизма» по принципу работы, а поэтому работает более устойчиво и имеет меньше проблем с запуском. Тем не менее, некоторая колебательность асинхронного двигателя в данной структуре управления также может присутствовать.

6.2.1. Проверка и настройка ДПР

Перед тем как запустить векторную СУ, нужно проверить работу датчиков положения ротора. Для векторной системы АД в данном ПО предусмотрен только один датчик положения – инкрементальный. Необходимо выбрать данный тип датчика положения в параметре **Базов. парам.Тип датчика положения [5113,09]** и настроить число меток в параметре **ДПР-Энкодер.Кол-во периодов [5152,01]**, а также задать наличие индексной метки в параметре **ДПР-Энкодер.Наличие репера [5152,02]**. Параметры реального датчика положения можно в его документации. Следует отметить, что существует некоторая путаница между количеством периодов датчика и его количеством импульсов. При количестве периодов датчика, например, 1000, суммарное количество импульсов по обоим каналам на обороте составит 4000. В параметр **ДПР-Энкодер.Кол-во периодов [5152,01]** вводится именно число до учетверения. После выбора и настройки датчика необходимо подать команду сохранения параметров и перезагрузить микроконтроллер, сбросив питание или вписав «1» в параметр **Настройки СУ.Перезагрузка [5114,05]**.

Далее необходимо следует запустить двигатель на холостом ходе в частотно-токовой системе управления (режим работы 4, см. п.п. 6.2.3), задать небольшую скорость, к примеру, 2-10 об/мин и наблюдать осциллограммы сигналов токов и, убедившись что двигатель крутится, нужно наблюдать сигнал датчика положения **ДПР-Энкодер.Угол. электр. [5152,08]** (угловое положение с квадратурного энкодера в электрических градусах).

Нужно убедиться, что при вращении двигателя угловое положение с датчика увеличивается вверх, а параметр **ДПР-Энкодер.Скорость [5152,07]** показывает положительное значение

скорости. Если это не так и датчик работает «в обратную сторону», то нужно либо перевернуть две фазы двигателя, либо два канала квадратурного сигнала датчика.

Далее необходимо разогнать двигатель на боле высокую скорость (не меньше четверти номинала) и задать амплитуду тока также не меньше четверти номинала. Убедиться, что значение скорости в параметре **ДПР-Энкодер.Скорость [5152,07]** чуть ниже (на 1-5%) чем задание скорости. Если разница в задании скорости и текущей скорости по датчику больше, то необходимо найти проблему: наиболее вероятно, что неверно задано количество пар полюсов двигателя, количество меток датчика, или СУ не была перезагружена после сделанных изменений настроек.

6.2.2. Векторное двухконтурное управление

Для запуска двигателя в векторной системе управления необходимо предварительно настроить регуляторы токов (п.п. 6.2.2) и датчик положения ротора (п.п. 6.2.16.1.4).

Кроме того, необходимо настроить постоянную времени ротора для наблюдателя потокосцепления ротора. Это можно сделать экспериментально многими способами, первый будет предложен в этом пункте, другой (уточняющий) ниже.

Для определения постоянной времени необходимо снять переходной процесс нарастания тока фазы при подаче на двигатель постоянного напряжения. Для этого необходимо установить **Настройки СУ.Режим работы [5114,02]=3** (скалярное управление п.п. 5.1.1), установить частоту вращения 0, запустить привод, отрегулировать нулевую точку кривой u/f (параметр **Кривая U-f.Нулевое опорное напряжение [511A,04]**) чтобы результирующий ток составлял порядка четверти номинала (в зависимости от двигателя и возможностей источника питания). При помощи режима Выборка осциллографа необходимо снять осциллограмму **АЦП.Ток фазы A [5116,03]** и **АЦП.Ток фазы B [5116,06]** при скачке задания напряжения. Для этого нужно установить **dlog.Количество точек предыстории [5150,0A]**=0, так как необходимо видеть процесс с самого начала, а параметр **dlog.StartBitL [5150,07]** установить равным 00000100, чтобы возвести триггер запуска записи осциллограммы по переходу в работу структуры скалярного управления. После этого открыть режим Выборка, дать команду обновления осциллограммы, а затем подать команду Пуск (или предварительно Стоп, если привод уже был в работе). Далее после нажатия кнопки «Обновить» программа UniCON запросит необходимость загрузки уже записанной осциллограммы, необходимо согласиться и нажать кнопку «Да». Примерный желаемый вид переходного процесса показан на рисунке Рис. 6.11.

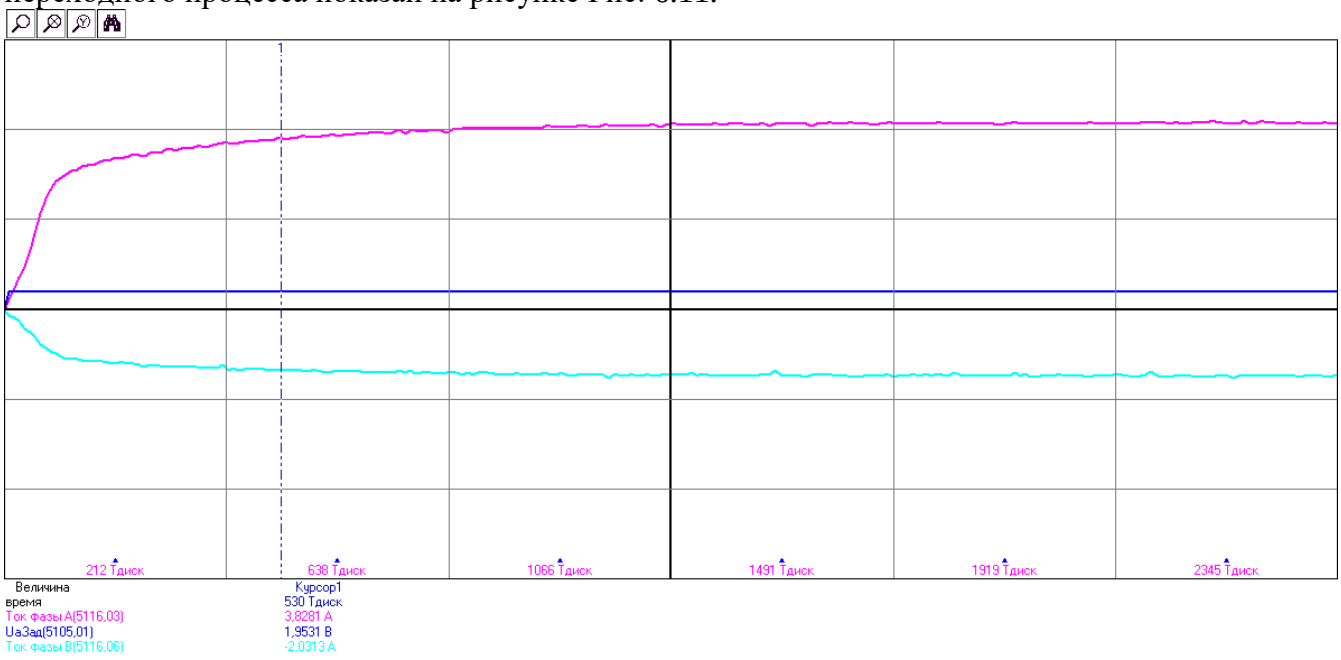


Рис. 6.11 Осциллограмма реакции токов АД на скачок постоянного напряжения

Необходимо отрегулировать делитель режима Выборка так, чтобы на осциллограмму влезал полный процесс нарастания тока до установившегося значения. Для этого нужно менять значение делителя и повторять процесс пуска-останова привода, получая новую осциллограмму. Для двигателей большой мощности постоянная времени может оказаться настолько большая, что возможно пользоваться непрерывным режимом осциллографа вместо режима выборки.

Далее необходимо найти точку, где ток достигает 95% установившегося значения, поставить курсор. С учетом того, что модуль даталоггера (dlog) в данном ПО вызывается с частотой 10кГц, а постоянная времени экспоненты составляет примерно треть от времени достижения ей 95%, нужно рассчитать постоянную времени из числа тактов (Тдиск) на курсоре. Для примера на рисунке постоянная времени составляет 530/10000/3=0.017с. Для больших двигателей постоянная времени больше, для малых – меньше. Далее необходимо внести рассчитанное значение в параметр наблюдателя потока ротора **Наблюдатель ротора АД.Пост. времени ротора [5B04,01]**.

Примечание: строго говоря, полученный переходной процесс в АД состоит из суммы двух экспонент, одной с большой постоянной времени, и другой с малой. В данном опыте интересует постоянная времени большей экспоненты.

Предварительная настройка на этом закончена. Далее для выбора векторной двухконтурной структуры управления АД (описана в п.п. 5.1.5) необходимо установить **Настройки СУ.режим работы** = 55.

Векторная СУ с плохо настроенным регулятором скорости или грубым датчиком положения (инкрементальным энкодером с малым числом меток) обычно неустойчиво работает на низких скоростях (менее 100-500 об/мин в зависимости от двигателя), поэтому задавать следует бóльшие значения задания скорости в параметре **Задания.Скорость [5112,01]**. Также необходимо установить желаемое токоограничение амплитуды тока в параметре **Задания.Макс. ток ВУ [5112,02]**.

Настройки регулятора скорости в группе **Рег. Скорости** рекомендуется для первого пуска задать мягкими: Кп порядка 0.05, Ки 0.01. В осциллограф удобно добавить параметры из группы **Наблюдаемые**: скорость, амплитудный ток, мощность и оставить работать в непрерывном режиме. Далее можно произвести пуск векторной системы управления.

Если двигатель не начал крутится, в зависимости от ситуации, произвести коррекцию параметров регулятора скорости, уровня токоограничения, уставок защит. Желательно не достигать выходным напряжением инвертора допустимого максимума (может произойти на высокой частоте вращения и большом токе возбуждения i_x), это может привести к «разваливанию» системы управления, так как в ПО нет специальных средств ограничения регуляторов тока в насыщении.

Необходимо настроить регулятор скорости – наиболее простой вариант настройки: дойти увеличением коэффициента Kp (**Рег. скорости.Пропорц. коэфф. Kp [5101,03]**) до колебательного режима, уменьшить в два раза и оставить. Аналогично повторить с коэффициентом Ki.

Далее необходимо подобрать оптимальное соотношение тока статора и тока возбуждения (параметр **Наблюдатель ротора АД.Соотношение тока возб. X к току Y [5B04,02]**). Его можно рассчитать теоретически, зная номинальный ток двигателя, ток его холостого хода и поделив их друг на друга, а также можно получить экспериментально. Необходимо наблюдать амплитуду тока статора (параметр **Наблюдаемые.Ток статора [5138,03]**) и изменять соотношение, пока не будет достигнут минимум тока статора. Опыт необходимо проводить на фиксированной частоте вращения и нагрузке. Чем ближе будет рабочая точка к номинальной, тем точнее будет произведен подбор соотношения.

Далее можно оценить другим экспериментом постоянную времени ротора для верификации первого метода. Эксперимент проводится на холостом ходе. Суть эксперимента в изменении тока возбуждения скачком и наблюдении повышения выходного напряжения инвертора в ответ на скачок (напряжение повышается вместе с ростом потока ротора). Для этого надо наблюдать параметры **Рег. тока Id.Обратная связь [5100,02]**, **ШИМ.U_ампл [5105,10]**, **Рег. тока Iq.Вых.Интегр. [5102,0C]**, **ДПР-Энкодер.Скорость [5152,07]**. Необходимо изменять задание на

ток возбуждения путем изменения либо соотношения между токами (**Наблюдатель ротора АД.Соотношение тока возб. X к току Y [5B04,02]**), либо изменением минимального тока возбуждения (**Наблюдатель ротора АД.Минимальный ток возбуждения [5B04,03]**). При большой постоянной времени (порядка половины секунды) наблюдать осциллограмму можно в непрерывном режиме осциллографа UniCON с включенной опцией Турбо (рядом с вкладкой осциллографа). При малой постоянной времени ротора необходимо запрограммировать запуск осциллографа по изменению параметра (дополнить код программы отслеживанием изменения значения параметра и однократным вызовом функции dlog.set_mode(1, &dlog)). Наблюдать изменение потока ротора можно как по параметру выходного напряжения, так и по изменению выхода интегральной части регулятора оси *y* (который называется **Рег. тока Iq**). При эксперименте необходимо следить за неизменностью частоты вращения ротора. Примерный вид получившейся осциллограммы показан на рисунках Рис. 6.12 и Рис. 6.13.

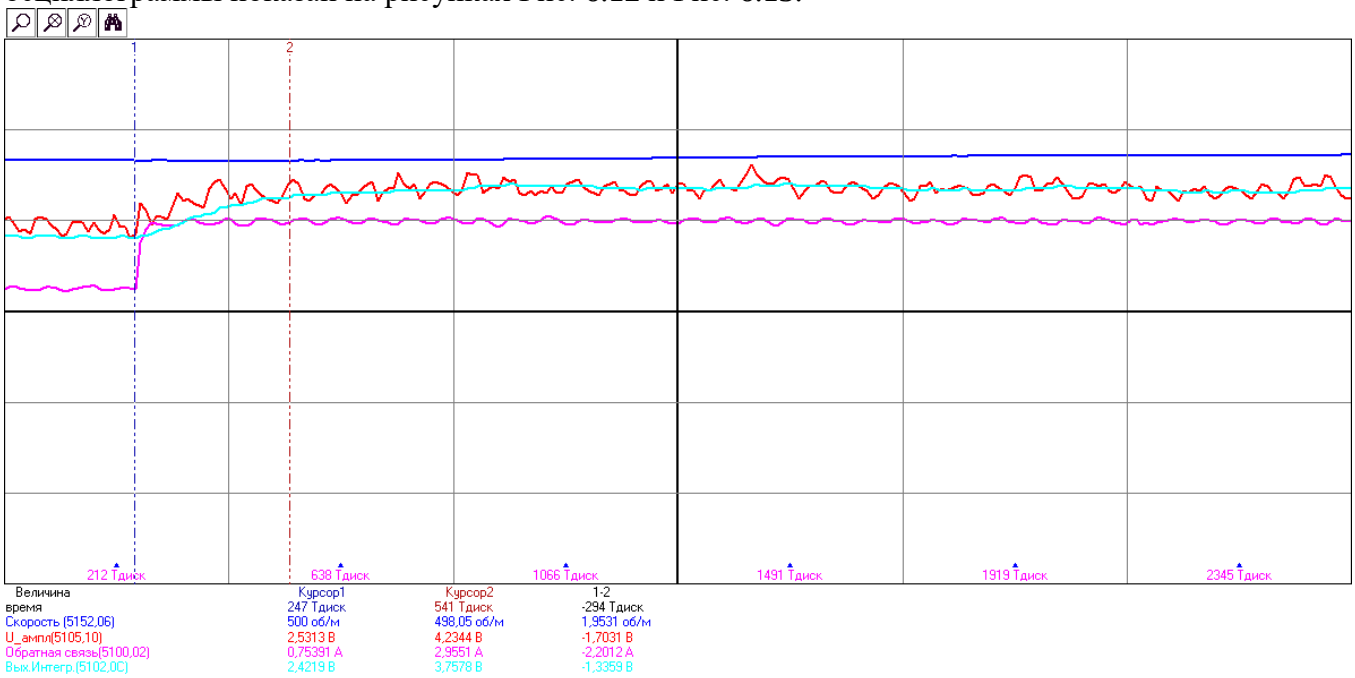


Рис. 6.12 Осциллограмма реакции напряжения на АД при изменении тока оси *x* скачком, снято в режиме Выборка для двигателя с малой постоянной времени ротора

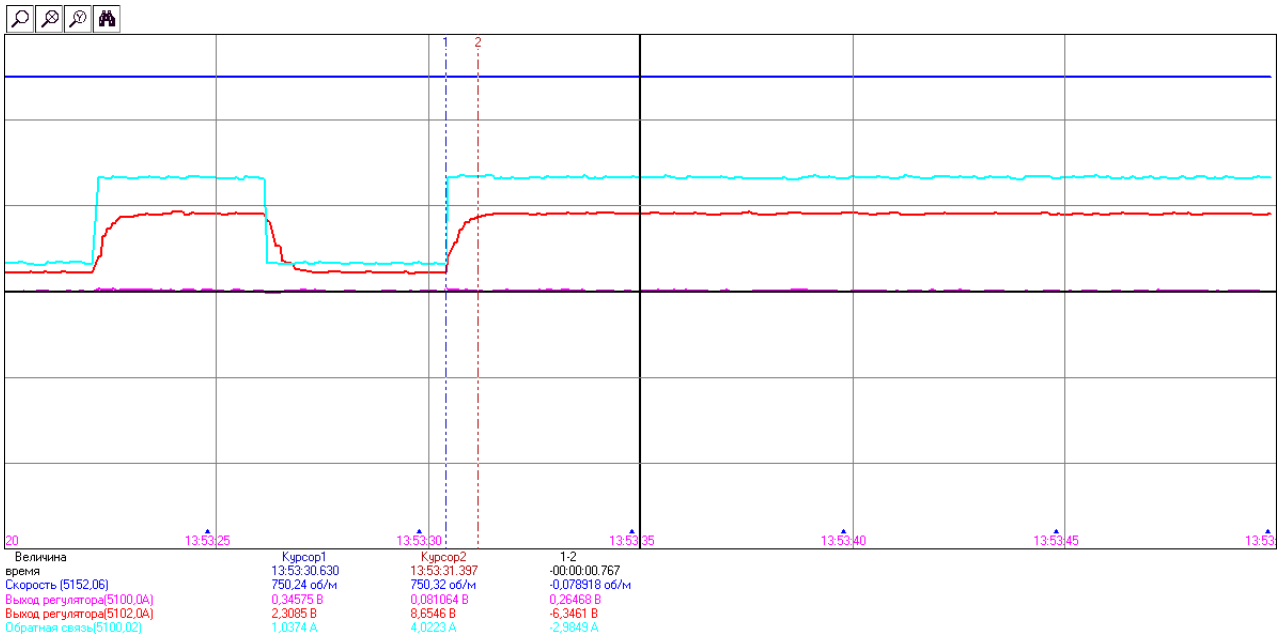


Рис. 6.13 Осциллограмма реакции напряжения на АД при изменении тока оси *x* скачком, снято в непрерывном режиме осциллографа для двигателя с большой постоянной времени ротора

После снятия осциллограммы можно аналогично пересчитать постоянную времени и скорректировать параметр **Наблюдатель ротора АД.Пост. времени ротора [5B04,01]**.

На этом этапе базовая настройка векторной СУ для АД завершена.

6.2.1. Позиционное управление

Для запуска структуры, замкнутой по положению для случая АД необходимо использовать инкрементальный датчик положения ротора. Необходимо пройти все шаги пусконаладочных работ до п.п. 6.2.2 включительно и качественно настроить регулятор скорости в этом пункте. Далее необходимо задать **Настройки СУ.режим работы** = 60, чтобы выбрать замкнутую по положению структуру управления, а затем обнулить параметр **ДПР-Энкодер.Число оборотов**: если он окажется не нулевой, то привод после запуска начнет «отматывать» обороты назад, стремясь к нулевому заданию положения (и нулевому счетчику оборотов). Параметры регулятора положения по умолчанию настроены на достаточно жесткое поддержание положения. Для первого запуска рекомендуется сделать регулятор более мягким, а интегральную составляющую отключить полностью. Первый раз рекомендуется запускать привод с небольшим разрешенным током (**Задания.Макс. ток ВУ**), а также ограничить частоту вращения **Задания.Скорость [5112,01]** на низком уровне порядка 50 об/мин. В параметре **Задания.theta_elec** пользователем задается желаемое положение в электрических градусах, к которому привод должен следовать значением параметра **ДПР-Энкодер.Угол электрический не огранич. [5152h.17h]**. Можно запустить привод и попробовать задать разные положения. На осциллографе в непрерывном режиме интересно смотреть задание углового положения, текущее угловое положение, текущий ток статора и скорость из группы **Наблюдаемые**.

Качественная настройка контура положения достаточно сложная процедура и выходит за рамки данного руководства. Пользователю предлагается изучить данный вопрос самостоятельно, экспериментируя с разными параметрами регулятора скорости и положения. Рекомендуется, как минимум, испытать две схемы с регуляторами: ПИ-регулятор скорости и П регулятор положения, а также П-регулятор скорости и ПИ-регулятор положения (в этом случае значение **Задания.Скорость [5112,01]** должно быть большим, чтобы пропорциональный регулятор скорости мог реализовать полное задание на ток). Регуляторы тока всегда должны быть ПИ.

После завершения настройки можно сохранить все параметры привода в энергонезависимую память. Это делается в программе UniCON путем нажатия на кнопку «Сохранить параметры», расположенной справа сверху. При этом значения всех параметров CANopen, доступных на запись, сохранятся в пользовательской флеш памяти микроконтроллера и будут в дальнейшем восстанавливаться при перезагрузке микроконтроллера в процедуре инициализации драйвера CANopen. Также можно сохранить все настройки привода в файл на компьютер, воспользовавшись меню Сервис-Перенос параметров (подробнее см. руководство пользователя на UniCON).

6.3. Двигатель постоянного тока

Перед работой с реальным оборудованием рекомендуется сначала провести виртуальную лабораторную работу с имитационной моделью двигателя постоянного тока и только затем повторять шаги на реальном оборудовании.

Для выбора имитационной модели двигателя постоянного тока необходимо скомпилировать проект и запрограммировать контроллер для использования имитационных моделей (тип аппаратной части выбирается в файле build.h). Далее в параметре **Модель общие данные.Тип двигателя модели [5181,01]** выбрать «**ДПТ**», а в параметре **Модель общие данные.Параметры двигателя [5181,02]** выбрать набор параметров электродвигателя, наиболее близкий к желаемому. Наборы параметров можно посмотреть в файле V_motorParams_DCM.h. Также можно задать набор параметров равным нулю и задать свои собственные параметры двигателя в группе **Модель ДПТ**. После завершения настройки нужно дать команду сохранения

параметров кнопкой «Сохранить параметры» справа сверху программы UniCON, а затем выполнить перезагрузку микроконтроллера для применения выбранных настроек (**Настройки СУ.Перезагрузка [5114,05]).**

6.3.1. Разомкнутая система управления

1. Для того, чтобы запустить разомкнутую систему управления ДПТ, необходимо открыть группу параметров «***Настройки СУ***» и записать в параметр «***Режим работы [5114.02]***» число 35. Это соответствует выбору состояния ***CTRL_DC_DRIVE*** дискретного автомата в функции «***SM_CTRL_Fast_Calc».***
2. Далее в группе параметров «ДПТ Сист. Упр» необходимо задать желаемое напряжение якоря и обмотки возбуждения (параметры ***«Заданное напряжение якоря [5185.02]»*** и ***«Заданное напряжение ОВ [5185.03]»***). В группе параметров ***«Модель общие данные»*** можно установить необходимую величину момента нагрузки (параметр ***«Симулируемый момент нагрузки [5181.03]»***).
3. После, нажатием кнопки «пуск» можно запустить систему управления.
   Предварительно добавив в осциллограф «***Ток ОЯ [5184.0F]***», «***Ток ОВ [5184.10]***», «***Реализованный электрический момент [5181.04]***» и «***Скорость [5181.05]***» можно наблюдать работу модели и системы управления (см. рис. 6.14).

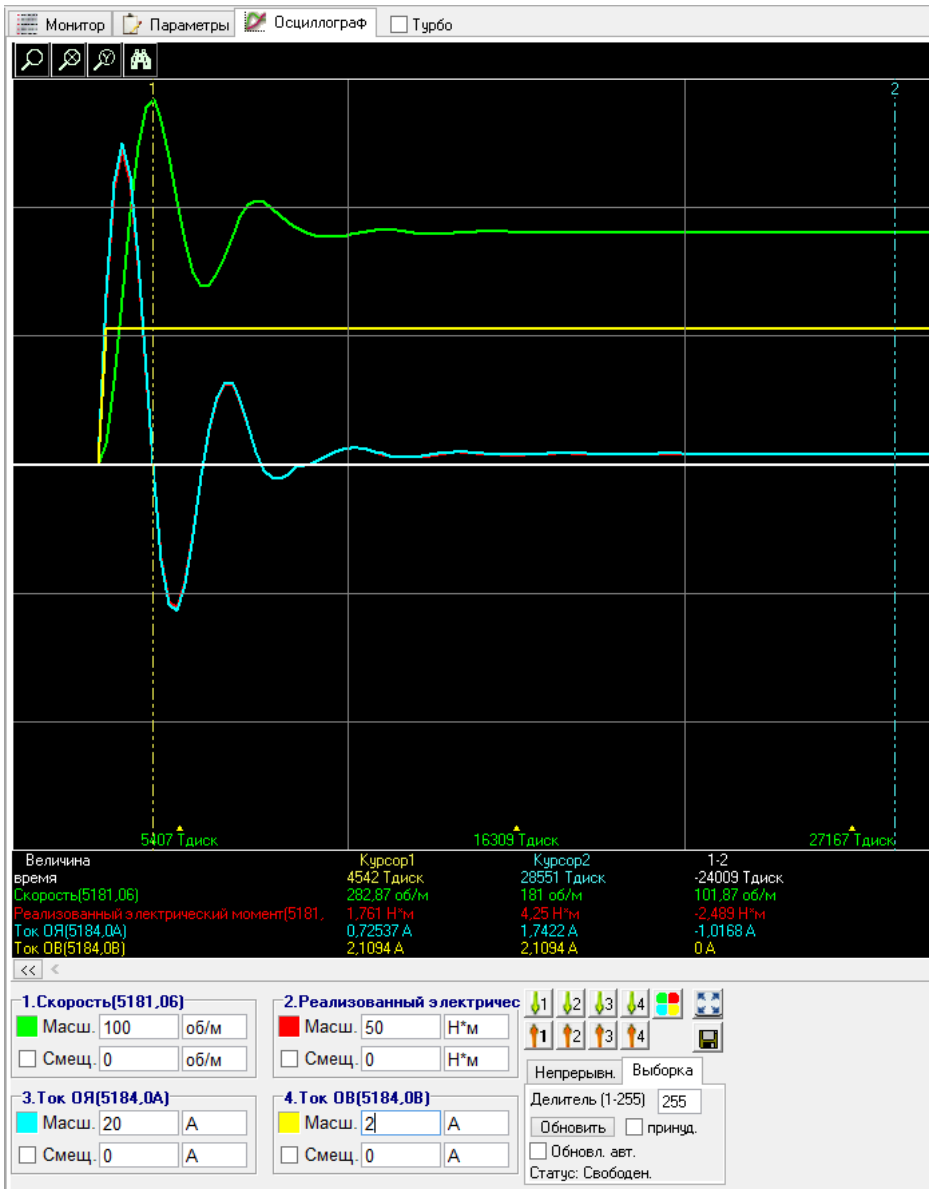


Рис. 6.14 Осциллограмма пуска ДПТ с набором параметров №4 в разомкнутой системе управления

6.3.2. Замкнутая система управления

1. Для того, чтобы запустить замкнутую систему управления ДПТ с контуром тока и скорости, необходимо открыть группу параметров «***Настройки СУ***» и записать в параметр «***Режим работы [5114.02]***» число 36. Это соответствует выбору состояния ***CTRL_DC_DRIVE_FB*** дискретного автомата в функции «***SM_CTRL_Fast_Calc».***
2. Далее в группе параметров «ДПТ Сист. Упр» необходимо задать желаемую скорость работы двигателя и напряжение ОВ (параметр «***Задание скорости (Подч. Рег.) [5185.01]***» и «***Заданное напряжение ОВ [5185.03]***»). В группе параметров «***Модель общие данные***» можно установить необходимую величину момента нагрузки (параметр «***Симулируемый момент нагрузки [5181.03]***»).
3. В группе параметров «***Рег. тока якоря (ДПТ)»*** необходимо установить требуемые настройки регулятора тока якоря, а в группе «***Рег. скорости»*** – требуемые настройки регулятора скорости.
4. После, нажатием кнопки «пуск» можно запустить систему управления.
   Предварительно добавив в осциллограф «***Ток ОЯ [5184.0F]***», «***Ток ОВ [5184.10]***», «***Реализованный электрический момент [5181.04]***» и «***Скорость [5181.05]***» можно наблюдать работу модели и системы управления (см. рис. 6.15).

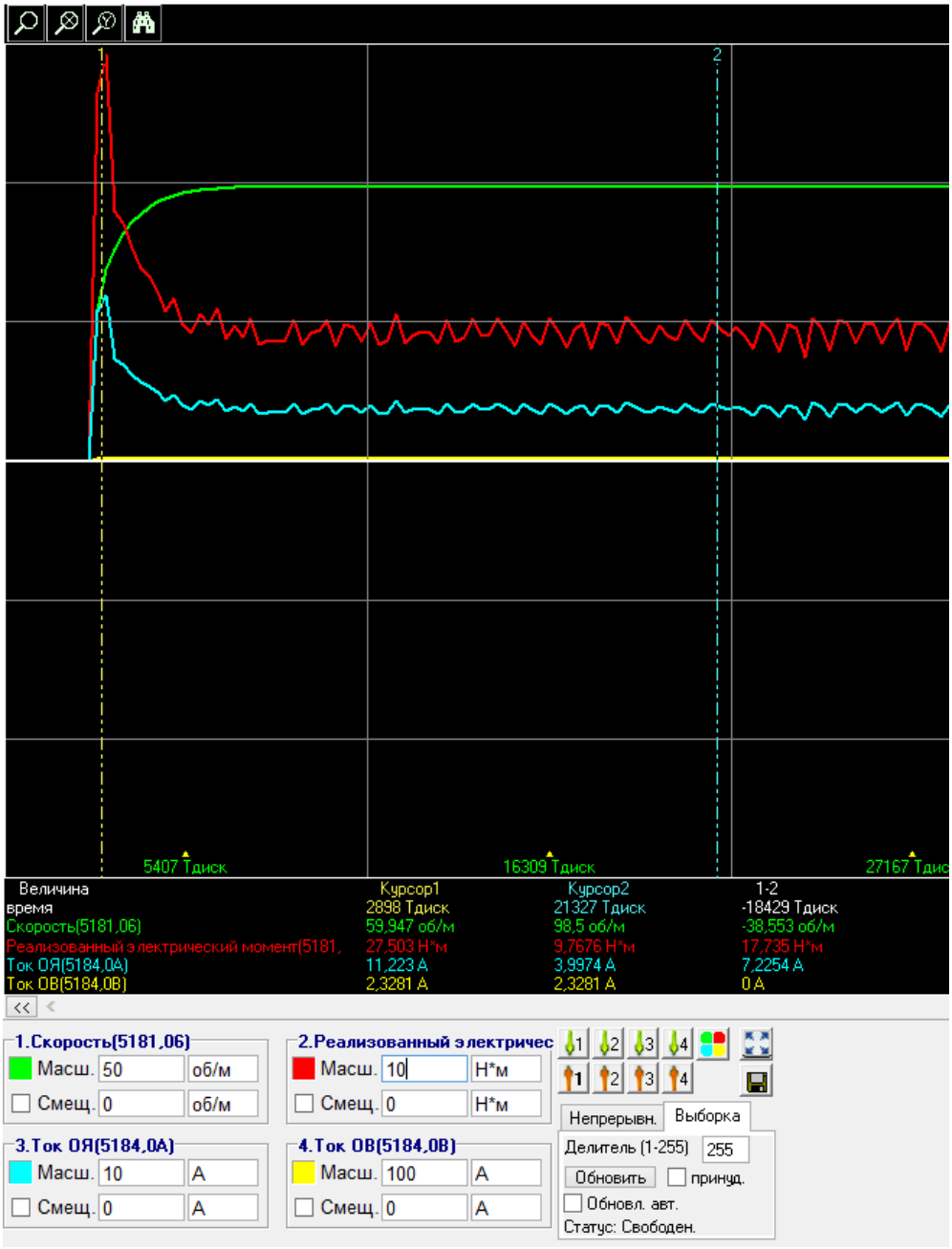


Рис. 6.15 Осциллограмма пуска ДПТ с набором параметров №4 в системе подчиненного регулирования с заданной скоростью 100 об/мин

6.4. Вентильно-индукторный двигатель

Перед работой с реальным оборудованием рекомендуется сначала провести виртуальную лабораторную работу с имитационной моделью вентильно-индукторного электродвигателя и только затем повторять шаги на реальном оборудовании.

Для выбора имитационной модели вентильно-индукторного двигателя необходимо скомпилировать проект и запрограммировать контроллер для использования имитационных моделей (тип аппаратной части выбирается в файле build.h). Далее в параметре **Модель общие данные.Тип двигателя модели [5181,01]** выбрать «**ВИД**», а в параметре **Модель общие данные.Параметры двигателя [5181,02]** выбрать набор параметров электродвигателя, наиболее близкий к желаемому. Наборы параметров можно посмотреть в файле V_motorParams_SRM.h. Также можно задать набор параметров равным нулю и задать свои собственные параметры двигателя в группе **Модель (ВИД)**. После завершения настройки нужно дать команду сохранения параметров кнопкой «Сохранить параметры» справа сверху программы UniCON, а затем

выполнить перезагрузку микроконтроллера для применения выбранных настроек (**Настройки СУ.Перезагрузка [5114,05]).**

6.4.1. Режим автокоммутации

1. Для того чтобы запустить систему управления ВИД с автокоммутацией, необходимо открыть группу параметров «***Настройки СУ***» и записать в параметр «***Режим работы [5114.02]***» число 101. Это соответствует выбору состояния ***CTRL_RUN_AUTOCOMMUTATION*** дискретного автомата в функции «***SM_CTRL_Fast_Calc»***.
2. В группе параметров «***Задания***» в параметре «***Макс. Ток ВУ [5112.02]***» задать уставку токоограничения – максимальное задание тока, выдаваемое регулятором скорости. Там же, в параметре «***Скорость [5112.01]***» необходимо задать желаемую скорость работы двигателя (частоту вращения). В группе параметров «***Модель общие данные***» можно установить необходимую величину момента нагрузки (параметр «***Симулируемый момент нагрузки [5181.03]***»).
3. В группе параметров «***Рег. скорости»*** необходимо установить требуемые настройки регулятора скорости.
4. После, нажатием кнопки «пуск» можно запустить систему управления.

Добавив в осциллограф «***Ток фазы A [5181.0B]***», «***Ток фазы B [5181.0C]***», «***Ток фазы C [5181.0D]***» и «***Скорость [5181.06]***» можно наблюдать работу модели и системы управления (см. рис. 6.16).

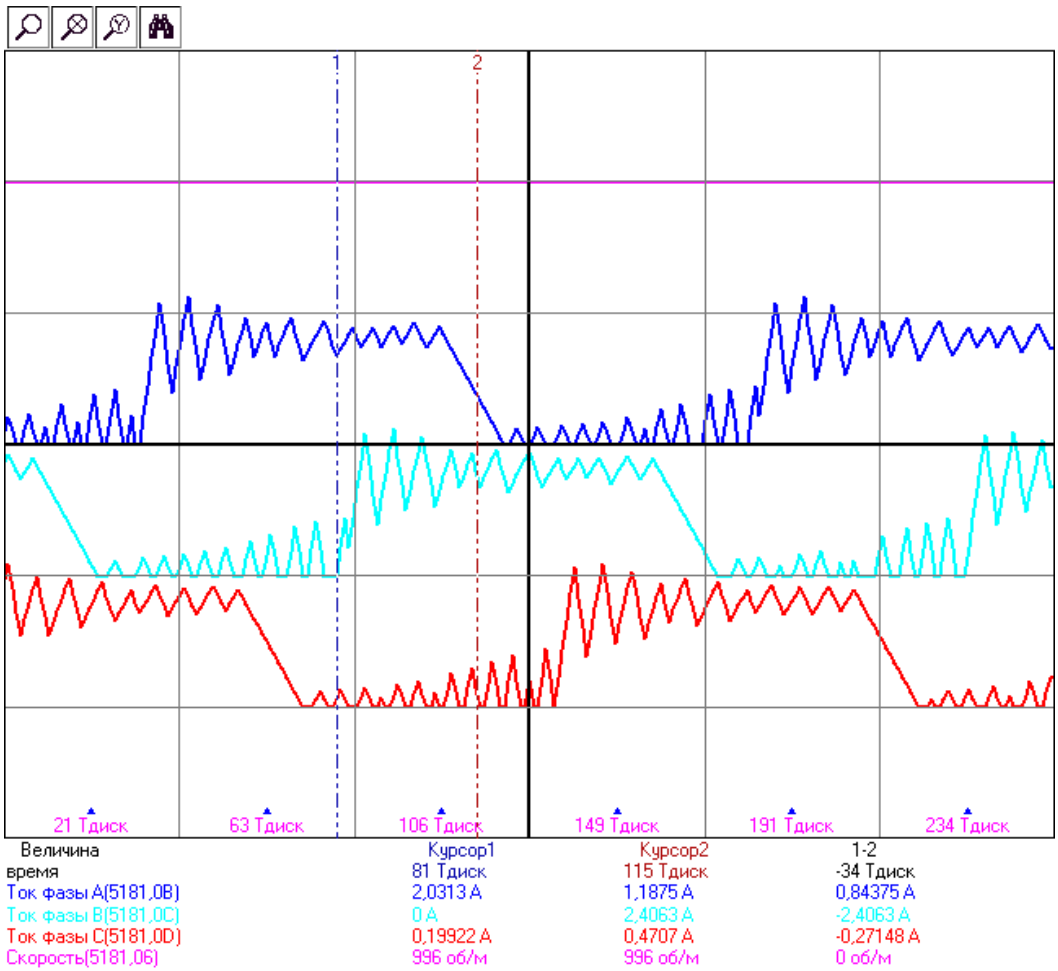
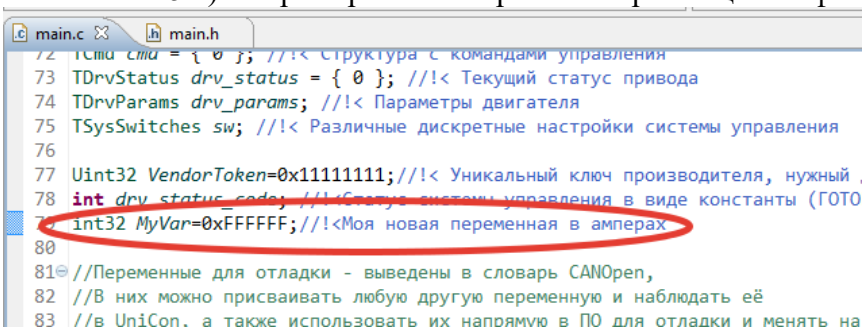


Рис. 6.16 Осциллограмма работы ВИД с набором параметров №1 в режиме автокоммутации с заданной скоростью 1000 об/мин

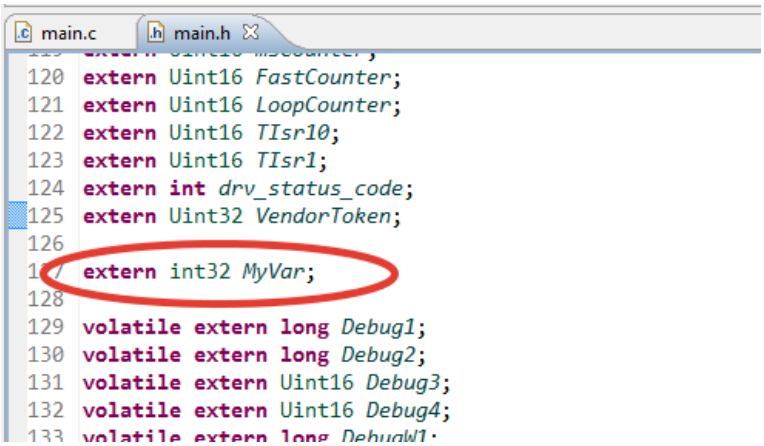
6.5. Добавление параметра CANopen в программе COODEdit

Пусть стоит задача добавить свой новый параметр в словарь объектов CANopen и наблюдать его в UniCON. Для этого нужно проделать ряд шагов:

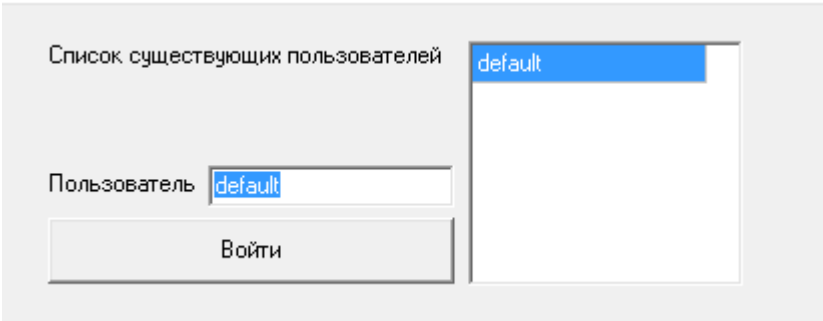
1. Создать новую переменную в проекте. Пусть это будет переменная с именем MyVar типа long (int32) в формате с фиксированной точкой IQ8.24, в которой хранится значение тока в относительных единицах с базой 5A (т.е. величине 1.0 в IQ соответствует значение физической величины 5A). В примере ниже переменная размещена в файле main.c.



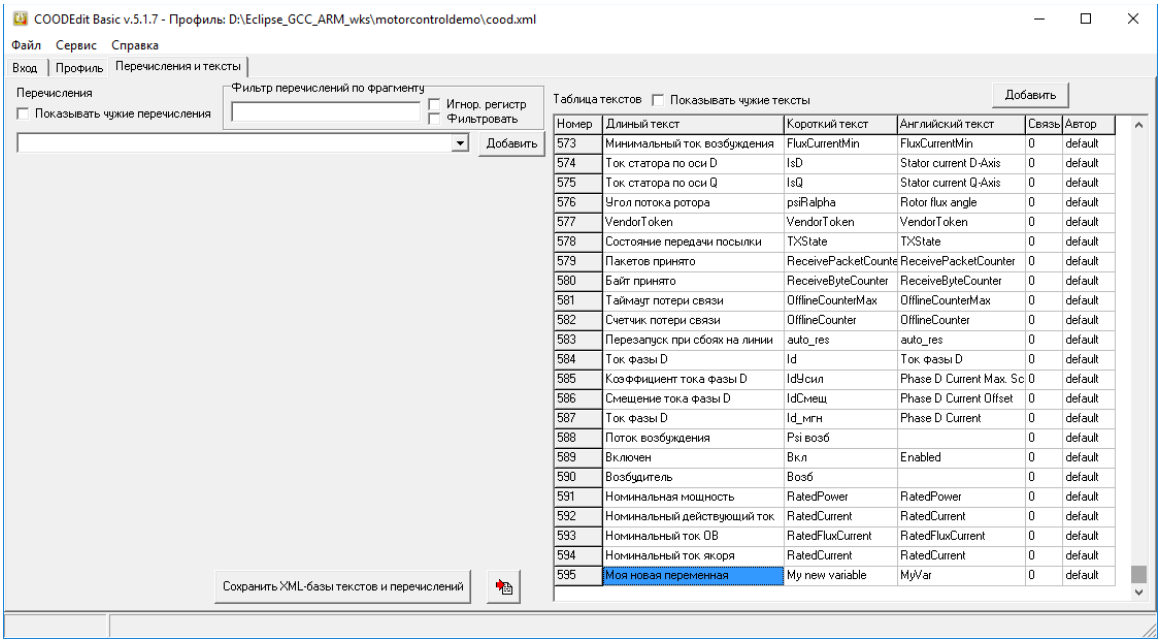
2. Для того, чтобы новая переменная была видимой в других модулях программы (в том числе в модуле CANopen драйвера) необходимо также объявить данную переменную со спецификатором extern в заголовочном файле main.h.



3. Для добавления переменой в словарь необходимо запустить программу COODEdit, которую можно скачать из раздела Downloads репозитория (https://bitbucket.org/niietcm4/motorcontroldemo/downloads/). Перед началом работы рекомендуется ознакомиться с руководством пользователя на программу, которое находится в директории с программой. Следующие шаги будут показывать нужные действия без подробного их пояснения (для подробностей см. руководство пользователя COODEdit).
4. На вкладке «Вход» программы необходимо нажать кнопку «Вход» для пользователя default.

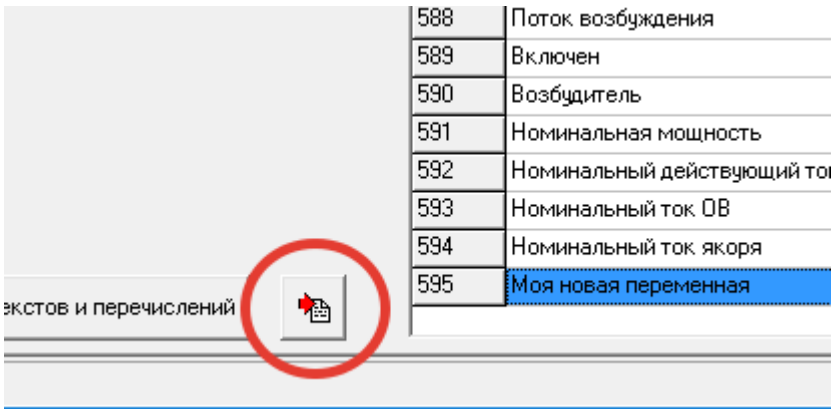


5. После этого нажать в меню **Файл - Открыть** и выбрать файл профиля словаря объектов в директории с проектом MotorControlDemo, называющийся cood.xml.
6. Необходимо создать новый текст для переменной – её название в программе UniCON. Для этого в COODEdit необходимо открыть базу текстов на вкладке «Перечисления и тексты», в правой верхней части окна нажать кнопку «Добавить» для создания новой пустой строки текста. Вписать новый желаемый текст переменной: длинный текст по умолчанию отображается в UniCON, английский текст отображается при выборе английского языка UniCON, а короткий текст может быть использован для дисплеев с малым числом символов (пульты управления). После заполнения каждого поля обязательно нажать Enter!

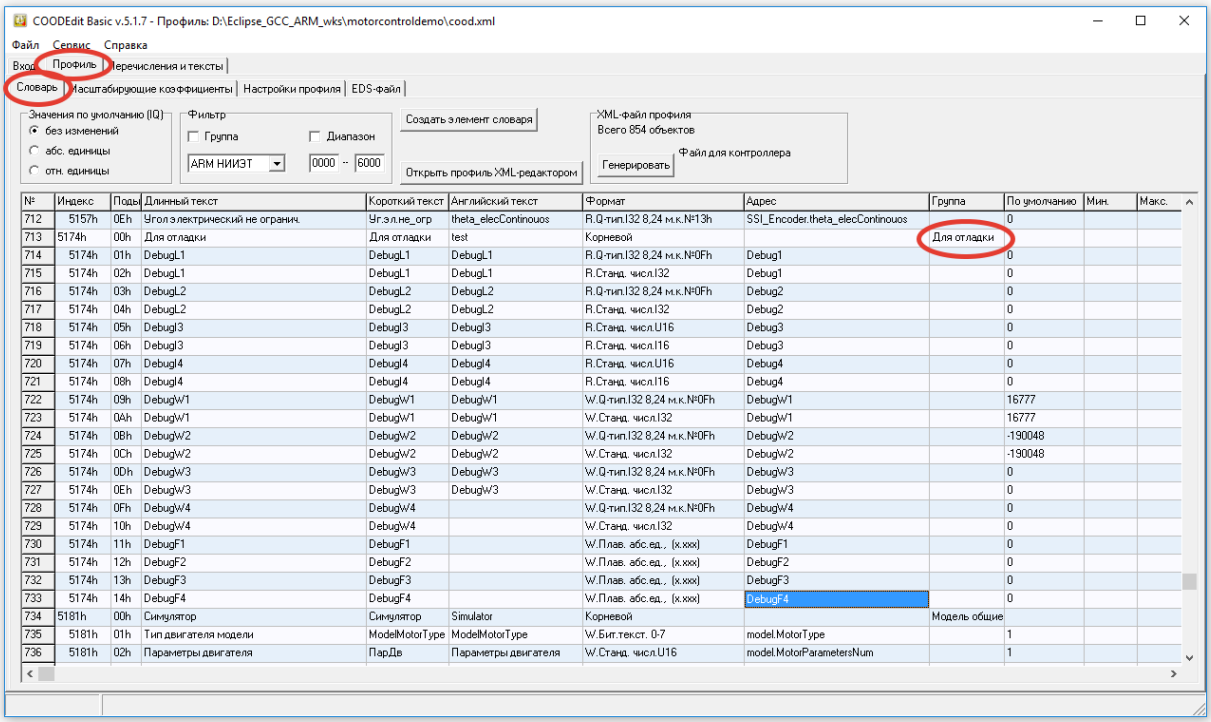


Номер	Длинный текст	Короткий текст	Английский текст	Связь	Автор
573	Минимальный ток возбуждения	FluxCurrentMin	FluxCurrentMin	0	default
574	Ток статора по оси D	IsD	Stator current D-Axis	0	default
575	Ток статора по оси Q	IsQ	Stator current Q-Axis	0	default
576	Угол потока ротора	psiRalpha	Rotor flux angle	0	default
577	VendorToken	VendorToken	VendorToken	0	default
578	Состояние передачи посылки	TXState	TXState	0	default
579	Пакетов принято	ReceivePacketCounte	ReceivePacketCounter	0	default
580	Байт принято	ReceiveByteCounter	ReceiveByteCounter	0	default
581	Таймаут потери связи	OfflineCounterMax	OfflineCounterMax	0	default
582	Счетчик потери связи	OfflineCounter	OfflineCounter	0	default
583	Перезапуск при сбоях на линии	auto_res	auto_res	0	default
584	Ток фазы D	Id	Ток фазы D	0	default
585	Коэффициент тока фазы D	IdУсил	Phase D Current Max. Sc	0	default
586	Смещение тока фазы D	IdСмещ	Phase D Current Offset	0	default
587	Ток фазы D	Id_мгн	Phase D Current	0	default
588	Поток возбуждения	Psi возб		0	default
589	Включен	Вкл	Enabled	0	default
590	Возбудитель	Возб		0	default
591	Номинальная мощность	RatedPower	RatedPower	0	default
592	Номинальный действующий ток	RatedCurrent	RatedCurrent	0	default
593	Номинальный ток ОВ	RatedFluxCurrent	RatedFluxCurrent	0	default
594	Номинальный ток якоря	RatedCurrent	RatedCurrent	0	default
595	Моя новая переменная	My new variable	MyVar	0	default

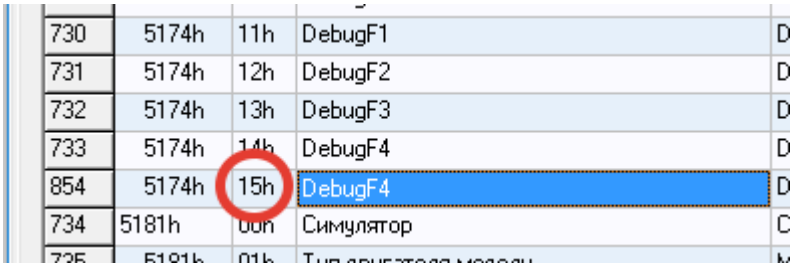
7. Необходимо экспортировать новый список текстов для программы UniCON, чтобы новый параметр мог быть корректно отображен в списке объектов. Для этого необходимо нажать на кнопку с пиктограммой с красной стрелкой в нижней части окна. Выбрать директорию программы UniCON и заменить файл текстов coTextBase_11111111.xml новым. Если UniCON был запущен, необходимо после перезапустить его для использования нового файла текстов.



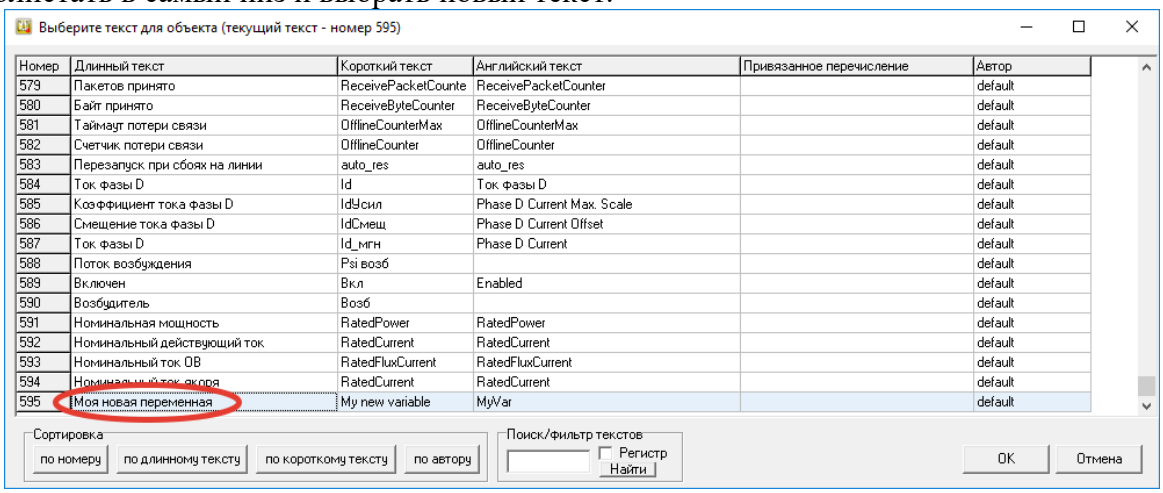
8. Далее для создания переменной необходимо перейти на вкладку «Профиль – Словарь». Пусть добавлять новую переменную мы будем в уже существующую группу параметров «Для отладки». Найдем её в словаре.



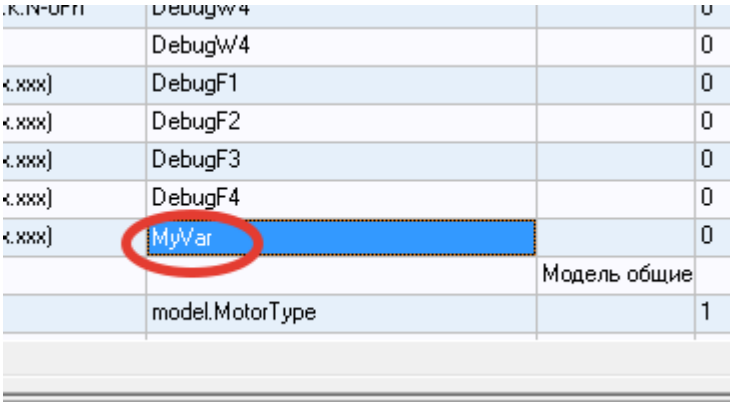
9. Новый параметр проще всего создать, копируя уже существующий. Нажмите правой кнопкой мыши на последнем параметре DebugF4 группы «Для отладки» и выберите «Копировать». Параметр скопируется с тем же именем и индексом/подындексом.
10. Необходимо задать новый номер подындекса, следующий по порядку. Для этого можно кликнуть два раза по ячейке с подындексом и ввести новое значение, а можно нажать правой кнопкой но корневому параметру и выбрать в контекстном меню «Перенумеровать подындексы».



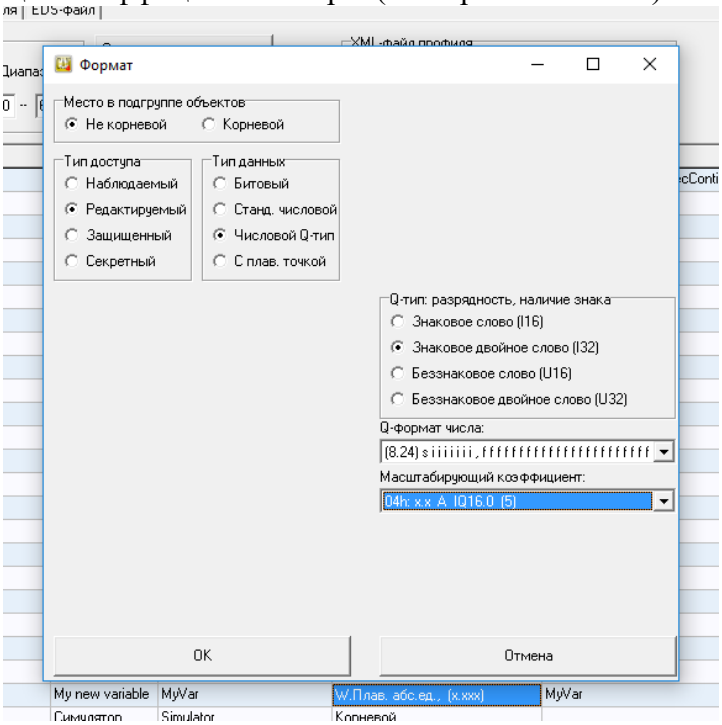
11. Далее необходимо выбрать для параметра новый созданный текст из базы текстов. Для этого нужно два раза кликнуть по ячейке «Длинный текст», в появившемся окне текстов пролистать в самый низ и выбрать новый текст.



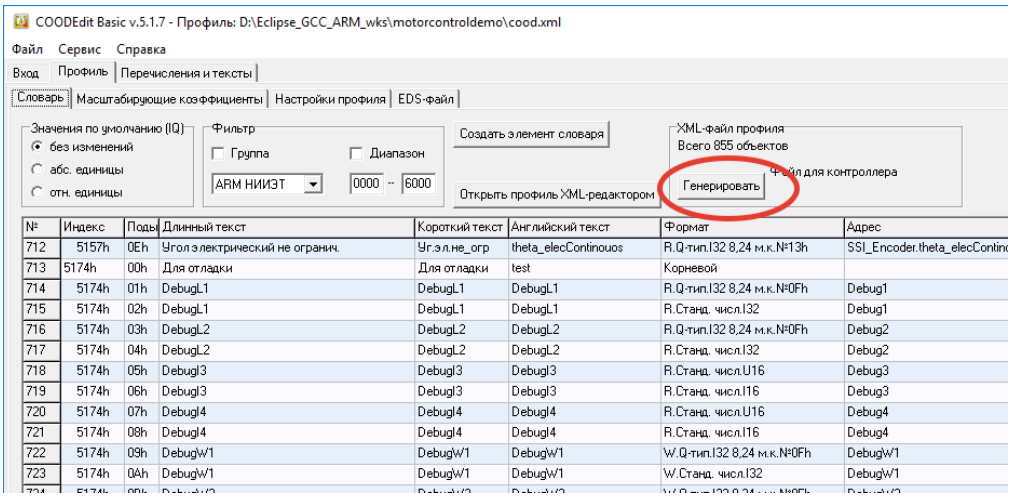
12. Нужно указать имя Си-переменной в проекте микроконтроллера, аналогично вписав его в ячейку «Адрес».



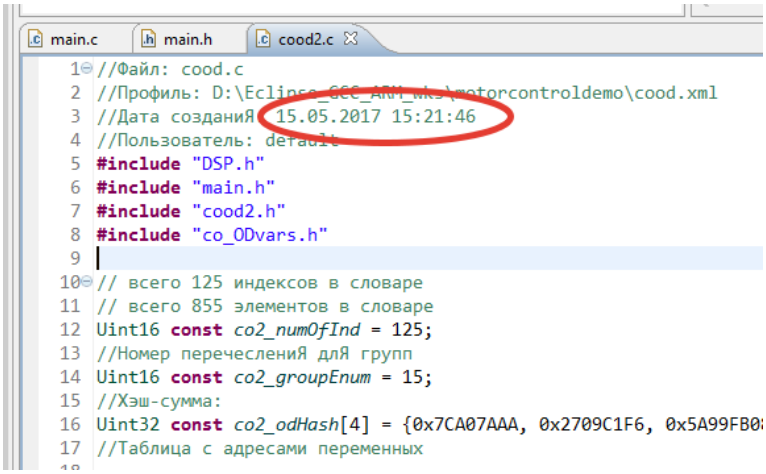
13. Далее нужно сконфигурировать формат переменной. После двойного щелчка по ячейке «Формат» нужно выбрать что это редактируемый параметр, Числовой Q-тип, I32, Q-формат 8.24, масштабирующий коэффициент номер 4 (Амперы с базой 5А). Нажать Ок.



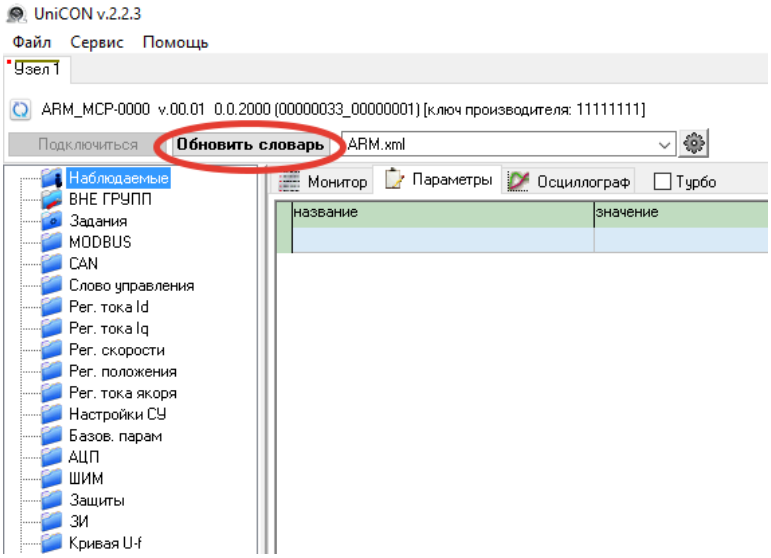
14. При необходимости можно настроить минимум и максимум значений объекта, а также задать значение по умолчанию (им будет проинициализирована переменная после запуска программы, после вызова инициализации драйвера CANopen).
15. На данном этапе настройка словаря закончена, нужно сохранить результат добавления переменной в файл профиля cood.xml из меню **Файл – Сохранить профиль** или сочетанием клавиш Ctrl+S.
16. Далее необходимо сгенерировать новый файл cood.c нажатием кнопки «Генерировать» сверху справа. Данная процедура сгенерирует файл дли микроконтроллера, содержащий словарь объектов на языке Си. Содержимое файла будет автоматически скопировано в буфер обмена.



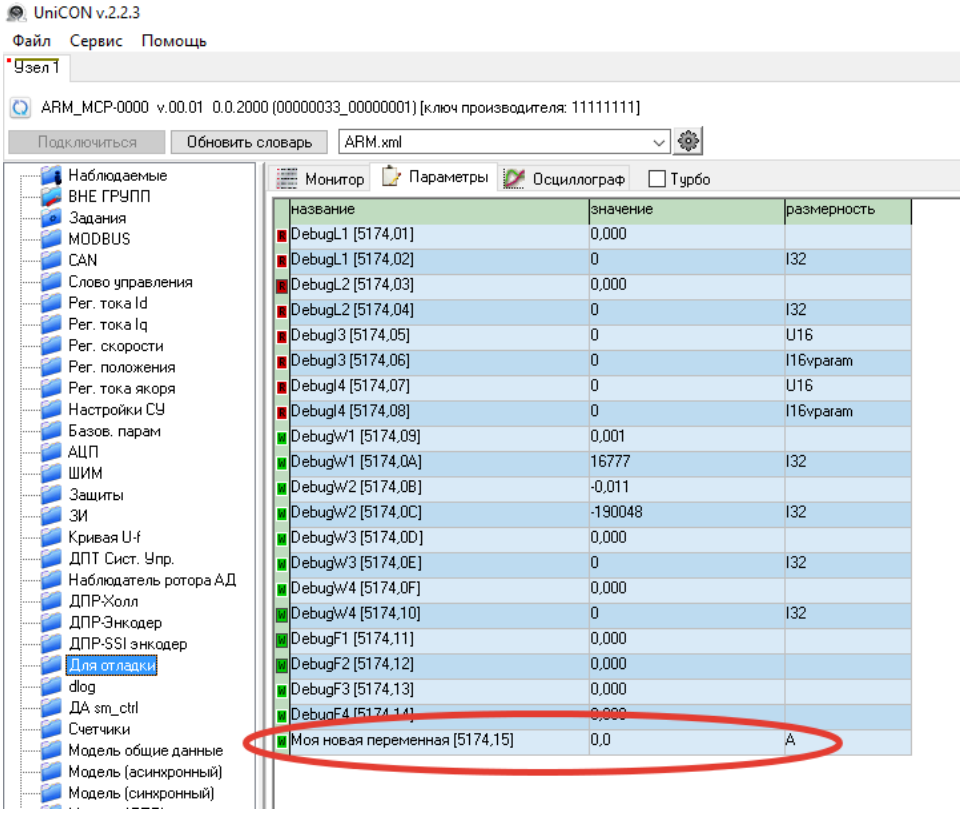
17. В проекте микроконтроллера следует открыть файл cood2.c и заменить все его содержимое на содержимое буфера обмена (Ctrl+A, Ctrl+V, Ctrl+S). Убедиться в текущей дате и времени генерации файла.



18. Запустить сборку (компиляцию) проекта. Убедиться, что проект собрался без ошибок. Перед программированием микроконтроллера желательно сохранить все текущие значения параметров привода в файл на компьютере посредством опции «Перенос параметров» UniCON, так как при добавлении в словарь нового редактируемого параметра все значения параметров сбросятся на значения по умолчанию, так как у драйвера CANopen не сойдется контрольная сумма при загрузке сохраненных значений параметров из flash-памяти. Если никаких ценных настроек не было сделано, сохранять значения не обязательно. После необходимо запрограммировать микроконтроллера новым ПО и запустить его.
19. Запустить программу UniCON. После подключения к устройству должна начать мигать кнопка «Обновить словарь», оповещая о том, что хэш словаря объектов микроконтроллера не соответствует хэшу словаря объектов сохраненному в UniCON. Необходимо нажать на эту кнопку.



20. После завершения обновления необходимо открыть группу параметров «Для отладки» и найти внизу новый созданный параметр. Следует отметить, что значение параметра равно нулю, так как заданное по умолчанию в COODEdit значение параметра равно нулю, несмотря на присвоенный переменной инициализатор 0xFFFFFF.



21. Новый параметр успешно добавлен в словарь объектов и доступен для наблюдения, редактирования, осциллографирования и т.п. Для более подробного понимания происходящего необходимо ознакомиться с руководствами пользователя на пакет программ UniCON, COODEdit, драйвер CANopen.

7. Заключение

Данный документ постоянно дорабатывается и обновляется.

Последнюю на данный момент версию документации Вы можете найти на нашем сайте: http://motorcontrol.ru/production/controlcards/controlcard_nt32m4f1/.

По любым проблемам, вопросам и предложениям обращайтесь:

- Интернет-форум ОА «НИИЭТ» http://forum.niiet.ru/viewtopic.php?f=37&t=536
- интернет-форум ООО «НПФ ВЕКТОР» http://motorcontrol.ru/forum/
- motorcontrol@niiet.ru
- Система отслеживания ошибок Bitbucket https://bitbucket.org/niietcm4/motorcontroldemo/issues

Если вы нашли ошибку в программном обеспечении или документации просьба по возможности оповестить об этом разработчиков любым удобным способом.